



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Java Basics

FEBRUARY, 2026



Contents

- Introduction to Object Oriented Programming(OOP)
- Java Basics

Introduction to Object Oriented Programming (OOP)



Contents

- Programming Paradigm
- Object Oriented Programming

Programming Paradigm



Programming Paradigm

Overview

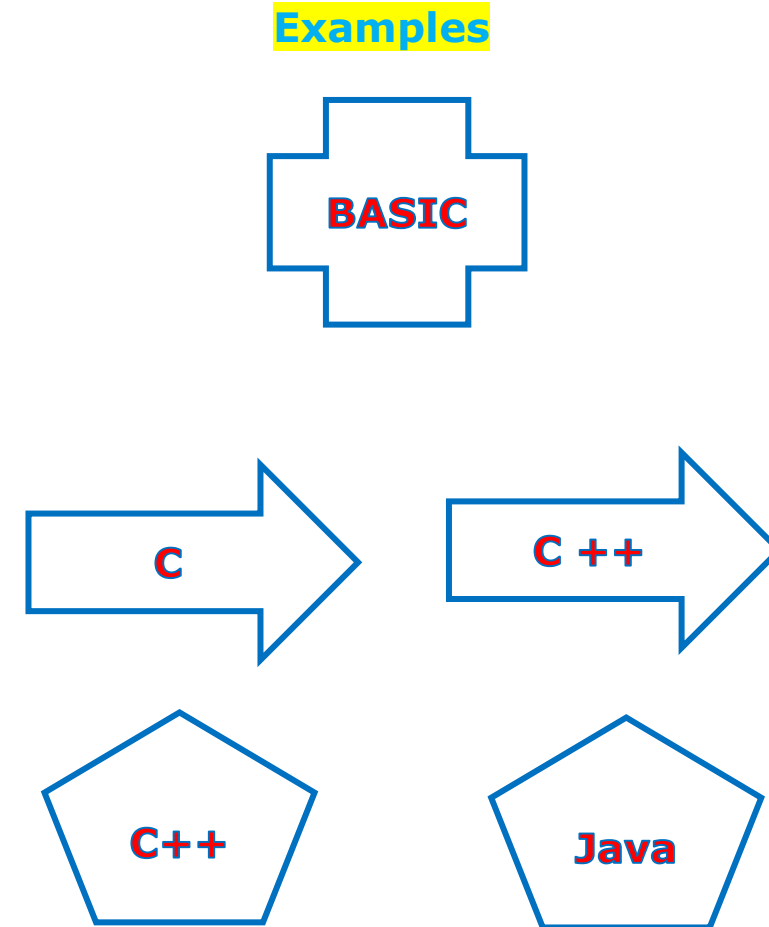
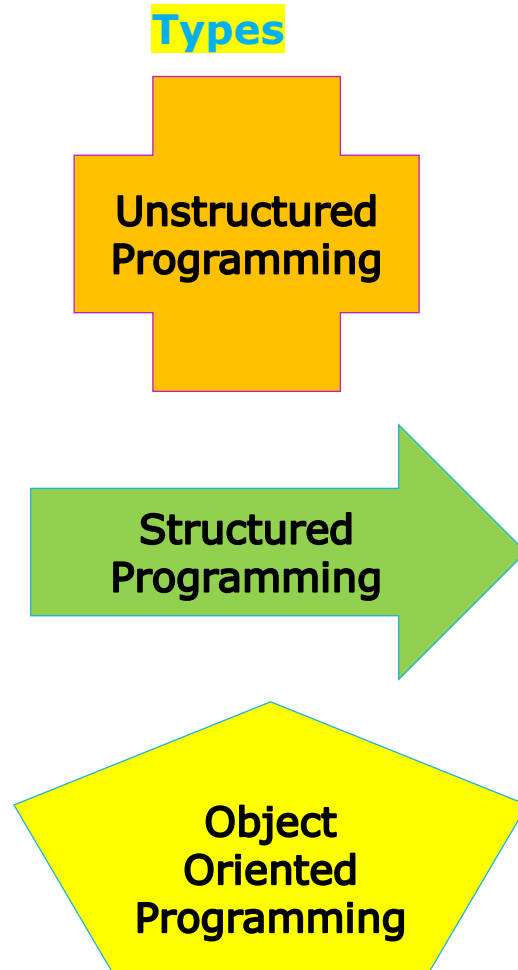
- Programming paradigm is a **way or style** of programming.
- It defines the **methodology of designing** and **implementing programs**.

Three basic types

1. **Unstructured Programming**: It contains only **one section**, i.e., **main**. The main consists of sequence of commands and statements i.e. unable to use **selection and repetition control statement** constructs.
2. **Structured Programming**: It uses the structured control flow constructs of **selection** (if/then/else) and **repetition** (while and for), block structures, and **subroutines**.
3. **Object Oriented Programming**: Organized **around objects**. It combine **data** and **action** together.

Programming Paradigm

Overview



Unstructured Programming

- **Unstructured programming** is the historically earliest programming paradigm capable of creating Turing-complete algorithms.
- It uses unstructured **jumps** to labels or instruction addresses with the use of unstructured control flow using **goto** statements or equivalent.
- Unstructured programming has been heavily criticized for producing **hardly-readable** ("spaghetti") code.

Programming Paradigm

Unstructured Programming

Advantages

- Simple and easy to practice by novice programmer
- Good for small program; no lengthy planning needed.
- Speed and flexible
- Provides full freedom to programmers to program as they want.

Disadvantages

- No code reusability
- Repetition of code
- Cumbersome Maintenance
- Debugging is tough

Programming Paradigm

Structured Programming

- This programming paradigm aimed at **improving the clarity, quality, and development time** of a computer program by making extensive use of the structured control flow constructs of selection (if / then / else) and repetition (while and for), block structures, and **subroutines**.
- *Structured programming* enforces a logical structure on the program being written to make it more **efficient** and **easier to understand** and modify.
- The problem is **bifurcated into number of small problems** known as procedures (Also alternatively referred as functions or methods).

Programming Paradigm

Structured Programming

Advantages

- Complexity can be reduced using modularity (divide and conquer)
- Logical structures ensure clear flow of control.
- Modules can be re-used many times; thus, it saves time and increase reliability.
- Easier to update/fix the program by replacing individual modules rather than larger amount of code.

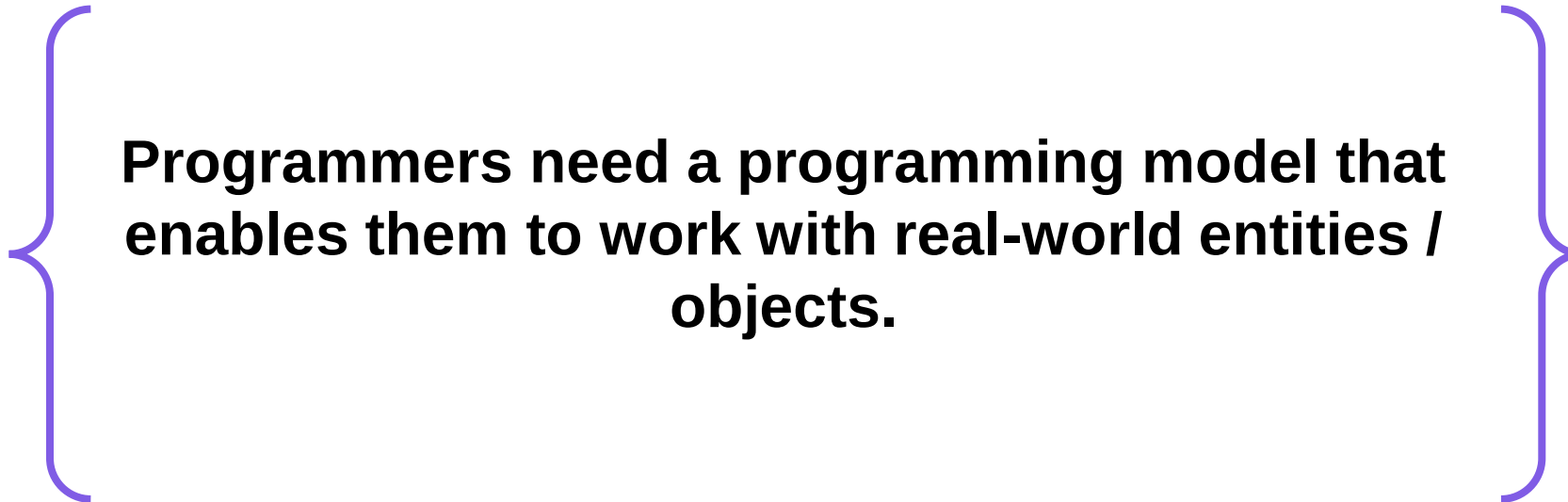
Disadvantages

- Data is not secure
- Code reusability is less
- Can support the software development projects easily up to a certain level of complexity. If complexity of the project goes beyond a limit, it becomes difficult to manage.

Object Oriented Programming



Need



Programmers need a programming model that enables them to work with real-world entities / objects.

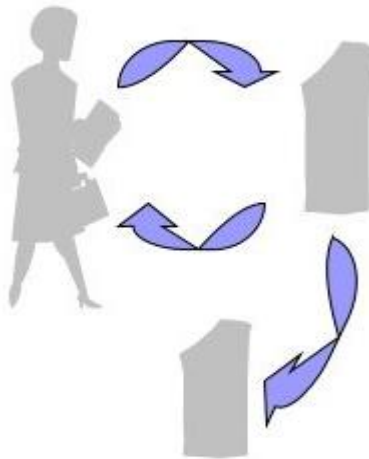
- People in real life have knowledge and can do a variety of tasks.
- Objects in OOP include fields to store **knowledge/state/data** and **methods** to do various tasks.

Object Oriented Programming

Structured Vs Object Oriented - Overview

Break down a programming task into a collection of **variables, data structures, and subroutines**.

Structured



Subroutines:
Withdraw, Deposit, Transfer

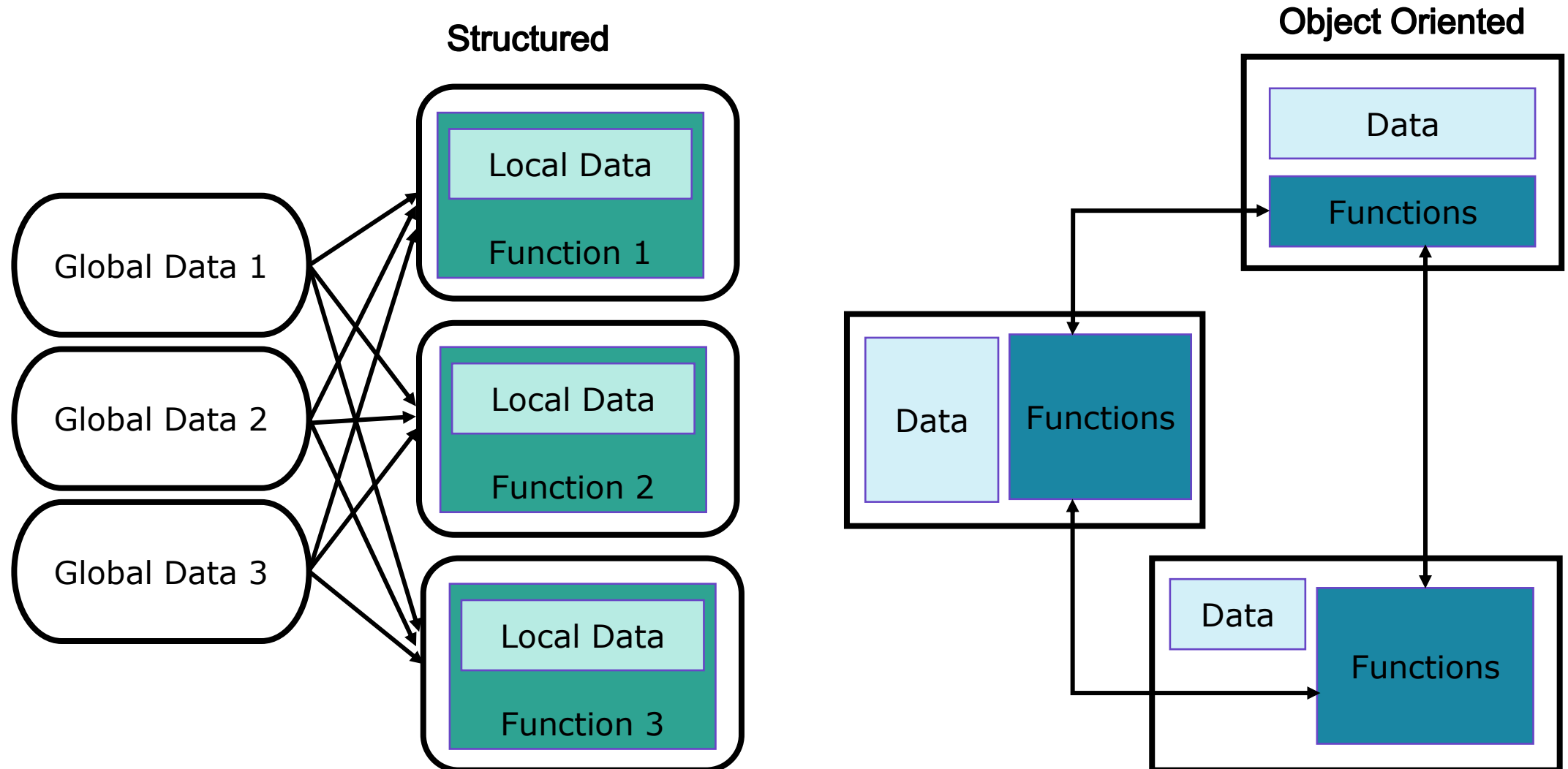
Break down a programming task into **objects** that expose behaviour (methods) and data (members or attributes).

Object Oriented



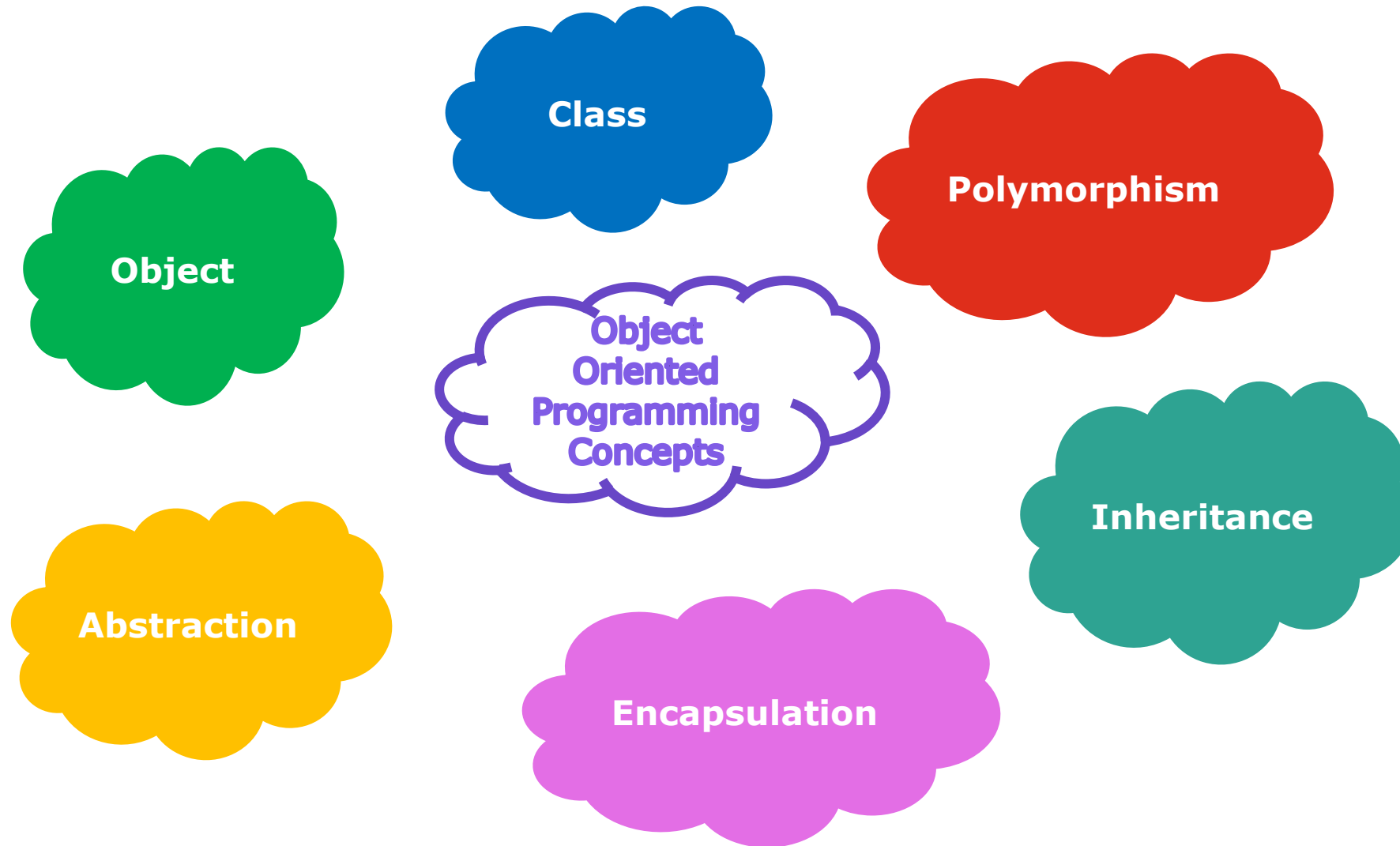
Objects:
Customer, Money, Account

Structured Vs Object Oriented - Overview



Object Oriented Programming

Features



Object Oriented Programming

Object - What it is?

- Object is the **key element** to understand *object-oriented* technology.
- It is a **real-world entity**.

Real world Objects



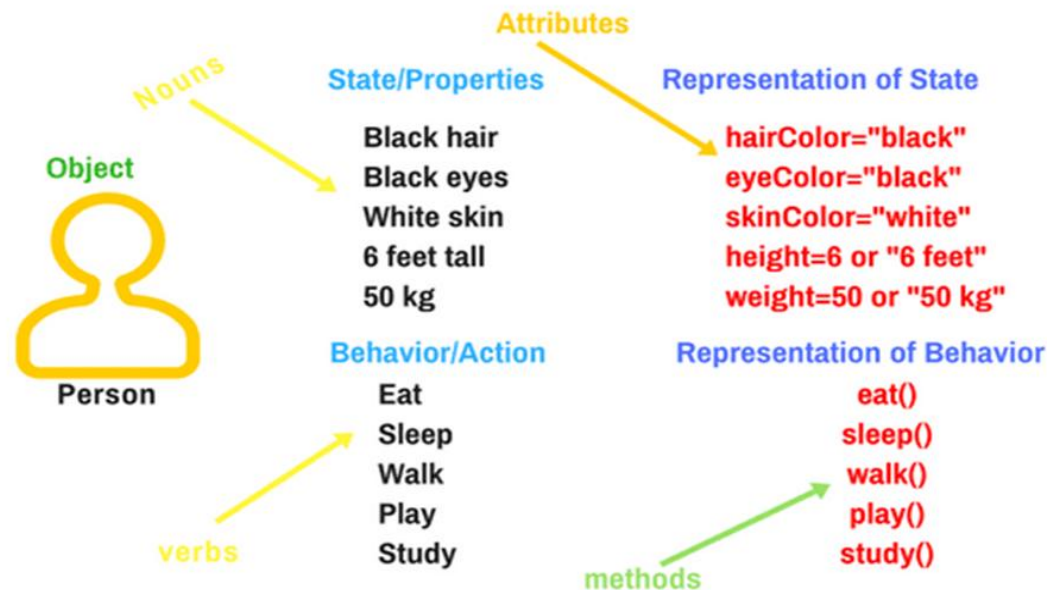
(Everything is an object in the real world)



Object Oriented Programming

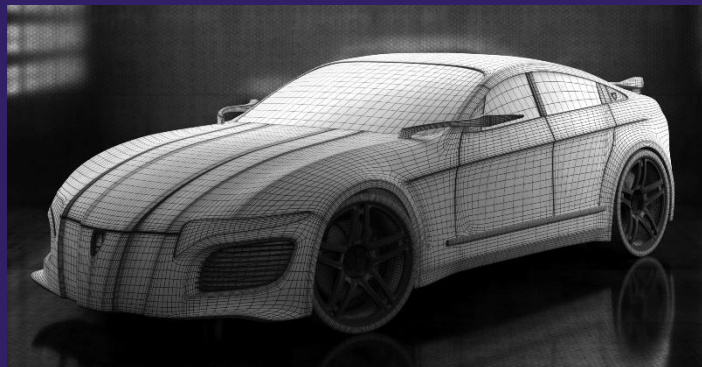
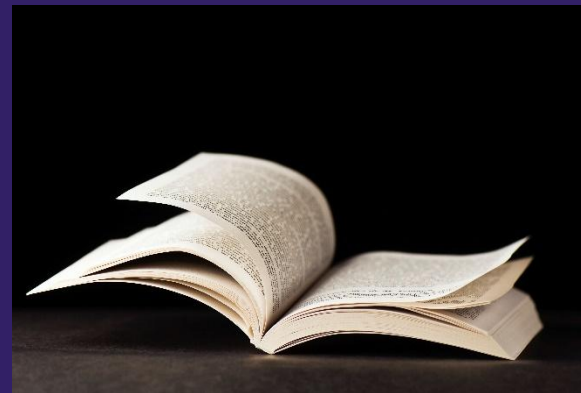
Object - What will it have?

- Real-world objects share **three characteristics**: State, Behavior and Identity.
 - The **state** of an object can be described as a set of attributes and their values.
 - The **behaviour /operation/action** of an object refers to the changes that occur in its attributes over a period.
 - Identity** is the name that identifies the object.



Activity

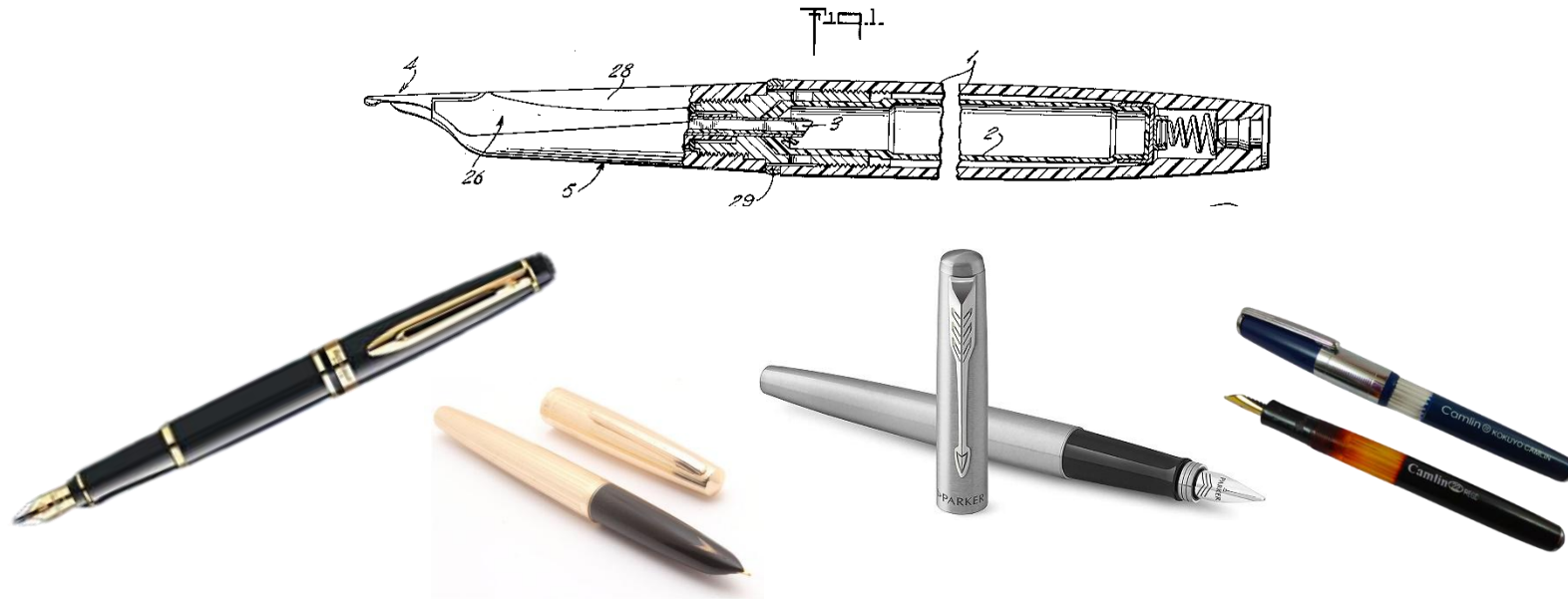
1. Identify the **STATE**, **BEHAVIOUR** and **IDENTITY** of the following Objects



Object Oriented Programming

Class

- Class is a **group of objects with similar properties**, common **behaviour**, common **relationships** with other objects, and common **semantics**.
- A class is the **template / blueprint** from which individual objects are created.



Object Oriented Programming

Object and Class

Objects

Class – Sportsman

States

Name
Sport
Gender
Height
Weight
rank

Behaviors

Profiling
BMI



Name: Cristiano Ronaldo

Sport: Football

Gender: Male

Height: 1.87m

Weight: 83kg

Rank: 6

Mr. Cristiano Ronaldo is a Football player with the international rank 6.

BMI is 23.7



Name: Rafael Nadal

Sport: Tennis

Gender: Male

Height: 1.85m

Weight: 85kg

Rank: 4

Mr. Rafael Nadal is a Tennis player with the international rank 4.

BMI is 24.6



Name: Mirabai Chanu

Sport: Weightlifting

Gender: Female

Height: 1.50m

Weight: 49kg

Rank: 2

Ms. Mirabai Chanu is a Weightlifting player with the international rank 2.

BMI is 21.8

Object Oriented Programming

Object – Instance of a class



Name: Cristiano Ronaldo
Sport: Football
Gender: Male
Height: 1.87m
Weight: 83kg
Rank: 6



Name: Rafael Nadal
Sport: Tennis
Gender: Male
Height: 1.85m
Weight: 85kg
Rank: 4



Name: Mirabai Chanu
Sport: Weightlifting
Gender: Female
Height: 1.50m
Weight: 49kg
Rank: 2

Instances of Sportsman class

Sports person 1

Sports person 2

Sports person 3

States

Name
Sport
Gender
Height
Weight
rank

Behaviors

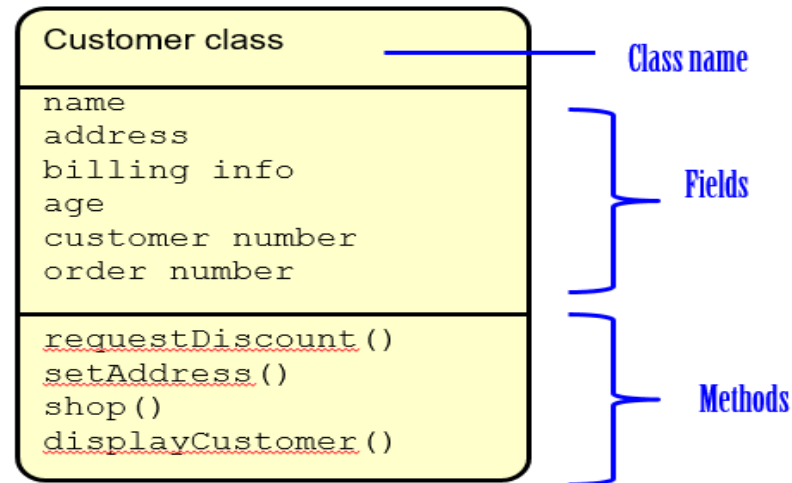
Profiling()
BMI()

Sportsman

Object Oriented Programming

Class diagram

Properties and Behaviors of 'Customer' class



Properties:

- name
- address
- age
- order number
- customer number

Behaviors:

- shop()
- setAddress()
- requestDiscont()
- displayCustomer()

Abstraction

- Data abstraction is **providing** only what is **relevant/essential** information about the data to the outside world, **hiding** the **irrelevant/background details/implementation**.
 - **For Example**, when you **send an email** to someone you just click send and you get the success message.
 - **What actually happens when you click send?**
 - **How data is transmitted over network to the recipient?**
 - All these are hidden from you (**because it is irrelevant to you**).

Object Oriented Programming

Abstraction



ATM user need not to know what is the process behind the screen.

♫ Musician need not to know how the sound is produced. All they need to know is which key to press



Note: An abstraction includes the essential details relative to the perspective of the user.

Object Oriented Programming

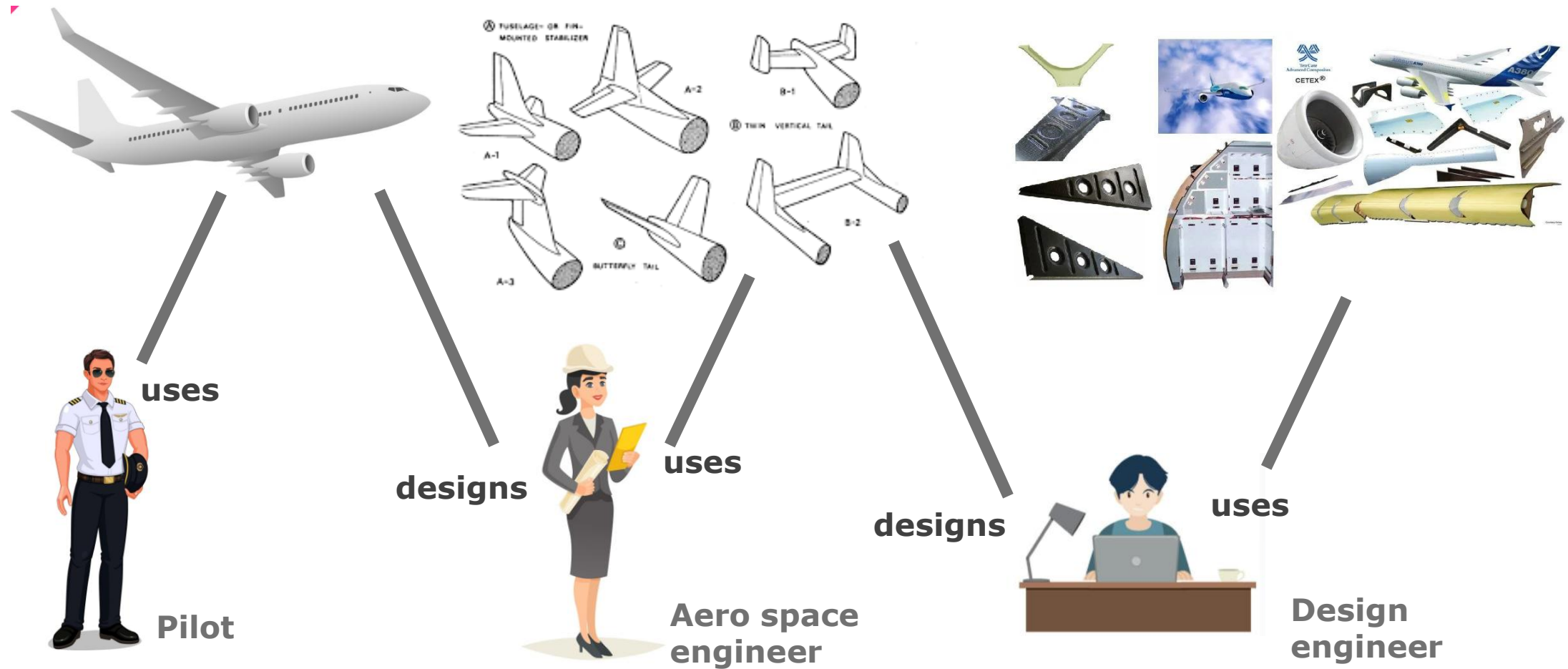
Abstraction



Note: An abstraction includes the essential details relative to the perspective of the user.

Object Oriented Programming

Abstraction



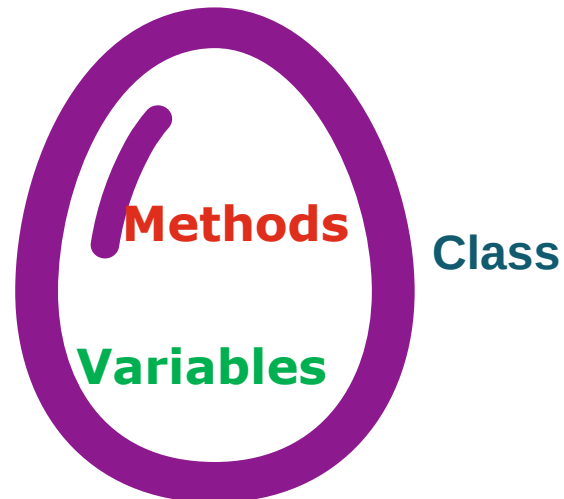
Note: An abstraction includes the essential details relative to the perspective of the user.

Encapsulation

- It refers to the **bundling of data with the methods** that operate on that data, or the restricting of direct access to some of an object's components.
- The internal representation of an object is generally **hidden from view** outside of the object's definition.
- It can reduce system complexity, and thus increase the robustness.



Medicines are not directly
accessible / protected
from outside



Encapsulation Vs Abstraction



Encapsulation

Abstraction



Note: **Abstraction** says what details to be made visible where as **Encapsulation** provides the level of access right to that visible details.

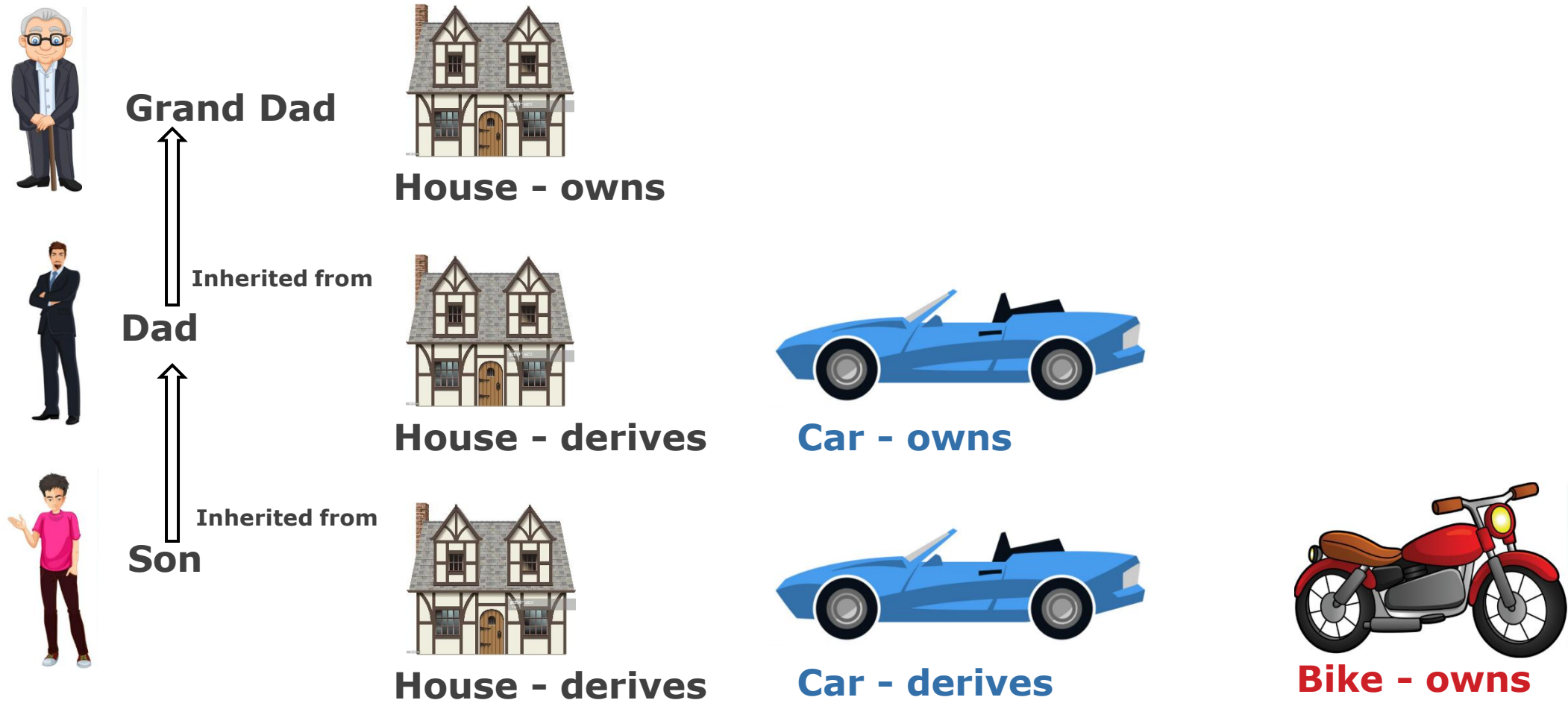
Inheritance

- Inheritance is a process in which **one object gets** the capability to acquires all the **properties and behaviours** of its parent object automatically.
- As a result, there is no need to rewrite the member again and again. This '**Code reusability**' feature avoids the **code redundancy**.



Object Oriented Programming

Inheritance



Polymorphism

- Polymorphism is the concept where an **object behaves differently in different situations**.
- More precisely we say it as '**many forms of single entity**'.



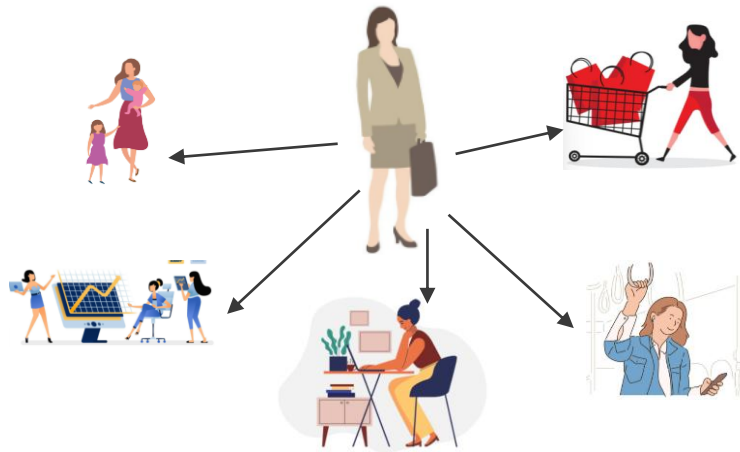
Object Oriented Programming

Polymorphism

Example 1: we are turn on the computer by one button at the same time we can turn off the computer by the same button.

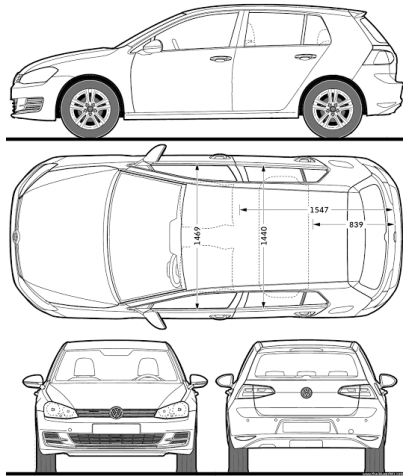


Example 2: A person might have a **variety of characteristics**. He is a man, and he can also be a father, a husband, or an employee. As a result, the same person behaves differently in different situations.



Object Oriented Programming

Summary: Car Object



CLASS



OBJECT



ENCAPSULATION



INHERITANCE



ABSTRACTION



POLYMORPHISM

Object Oriented Programming

Procedure Oriented Vs Object Oriented

POP

The program is divided into functions.

Top-down approach

Every function has different data, so there's no control over it.

Parts of a program are linked through parameter passing.

Expanding data and function is not easy.

Inheritance is not supported.

OOP

The program is divided into objects.

Bottom-Up approach

Data in each object is controlled on its own.

Object functions are linked through message passing.

Adding new data and functions is easy.

Inheritance is supported in public, private & protected modes.

Procedure Oriented Vs Object Oriented

POP

No access modifiers supported.

No data hiding. Data is accessible globally.

Overloading is not possible.

No friend function.

No virtual classes or functions.

No code reusability.

Not suitable for solving big problems.

OOP

Access control is done with access modifiers.

Data can be hidden using Encapsulation.

Overloading can be done.

No friend function except C++.

The virtual function appears during inheritance.

The existing code can be reused.

Used for solving big problems.

Quiz



1. Identify equivalent OOP Principle for the following scenarios.

A person who understands more than two languages yet can converse in any of them depending on the situation.

Polymorphism

Quiz



2. Identify equivalent OOP Principle for the following scenarios.

A scientific calculator is a more advanced version of a regular calculator.

Inheritance

Quiz



3. Identify equivalent OOP Principle for the following scenarios.

When you go to the bank and tell the banker(Object) to make a deposit(Method) of X dollars into your account(object), the banker will ask for your account number as well as the amount of cash you want to deposit. After a few moments, the banker informs you. Hey, your deposit was completed successfully, and your receipt is attached.

Abstraction

Object Oriented Programming

Quiz



4. How many objects can you identify in the below picture?



a) 16

b) 2

c) 3

d) 8

Ans: a) 16

Quiz



5. Is 'Sound' an object?

a) Yes

b) No

Ans: a) Yes

Quiz



6. Class is a collection of object

a) True

b) False

Ans: b) False

Quiz



7. Which of the following statement is correct?

a) Class is an instance of object.

b) Object is an instance of a class.

c) Class is an instance of data type.

d) Object is an instance of data type.

Ans: b)

Quiz



8. Which of the following concepts means wrapping up of data and functions together?

a) Abstraction

b) Encapsulation

c) Inheritance

d) Polymorphism

Ans: b)

Java Basics

History ,Introduction to Java



Contents

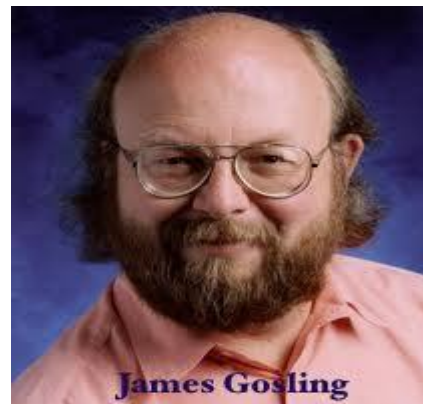
- Introduction to Java
- Project Scenario
- Java Tokens
- Read User Input
- Control Flow Statements
- Arrays
- Functions

Introduction to Java



What is Java?

- Java is a **general purpose, object-oriented, concurrent, and secured** programming language.
- It is a **widely used robust technology**.
- It is a **platform** – It has a runtime environment (**JRE**) and **API**.
- **James Gosling** – the father of Java.



History

- Java initial work started in **1990** by **Sun Microsystems** engineer **Patrick Naughton** as a part of the **Stealth Project**.
- The Stealth Project soon changed to the **Green Project**, with **Mike Sheridan** and **James Gosling** joining the ranks, and the group began developing new technology for programming next-generation smart appliances.
- **James Gosling**, **Mike Sheridan**, and **Patrick Naughton** initiated the Java language project in **June 1991**.
- Gosling attempted to modify and **extend C++**, but quickly abandoned this approach in favour of creating an entirely new language.

History

- Firstly, it was called "**Greentalk**" by James Gosling, and file extension was .gt. After that, it was called **Oak**, named after the tree that stood outside his office.
- Originally **designed for small, embedded systems in electronic appliances** like set-top boxes. Then incorporate some changes based on emergence of **World Wide Web**, which demanded **portable programs**.
- In 1995, Oak was renamed as "**Java**" because it was already a trademark by Oak Technologies.
- The first publicly available version of **Java (Java 1.0)** was released in 1995.
- In 2006 Sun started to make Java available under the **GNU General Public License (GPL)**. Sun Microsystems was acquired by the **Oracle Corporation in 2010**. Oracle continues this project called OpenJDK.

Where it is used?

- Java is **almost everywhere** and more than **3 billion devices** run java.
- It is used create desktop Applications, Web Applications, Enterprise Applications such as banking applications, mobile applications.
- Web services and Cloud based applications.
- Embedded System
- Games etc.,

Editions

J2SE

- **Java 2 Standard Edition**
- Client side standalone applications

J2ME

- **Java 2 Micro Edition**
- Applications for mobile Devices

J2EE

- **Jakarta EE/Java 2 Enterprise Edition**
- Applications for Enterprise

Introduction to Java

Version

Version	Release date	End of Free Public Updates ^{[1][5][6][7]}	Extended Support Until
JDK Beta	1995	?	?
JDK 1.0	January 1996	?	?
JDK 1.1	February 1997	?	?
J2SE 1.2	December 1998	?	?
J2SE 1.3	May 2000	?	?
J2SE 1.4	February 2002	October 2008	February 2013
J2SE 5.0	September 2004	November 2009	April 2015
Java SE 6	December 2006	April 2013	December 2018 December 2023, paid support for Zulu ^[8]
Java SE 7	July 2011	April 2015	July 2022
Java SE 8 (LTS)	March 2014	January 2019 for Oracle (commercial) December 2030 for Oracle (non-commercial) December 2030 for Zulu At least May 2026 for AdoptOpenJDK At least May 2026 for Amazon Corretto	December 2030
Java SE 9	September 2017	March 2018 for OpenJDK	N/A
Java SE 10	March 2018	September 2018 for OpenJDK	N/A
Java SE 11 (LTS)	September 2018	September 2027 for Zulu At least October 2024 for AdoptOpenJDK At least September 2027 for Amazon Corretto	September 2026, or September 2027 for e.g. Zulu ^[8]
Java SE 12	March 2019	September 2019 for OpenJDK	N/A
Java SE 13	September 2019	March 2020 for OpenJDK	N/A
Java SE 14	March 2020	September 2020 for OpenJDK	N/A
Java SE 15	September 2020	March 2021 for OpenJDK, March 2023 for Zulu ^[8]	N/A
Java SE 16	March 2021	September 2021 for OpenJDK	N/A
Java SE 17 (LTS)	September 2021	September 2030 for Zulu	TBA
Legend: ■ Old version ■ Older version, still maintained ■ Latest version ■ Future release			

Environment

- Java includes many development tools, classes and methods.
- **Development tools** are part of Java Development Kit (**JDK**)
- The classes and methods are part of Java Standard Library (**JSL**), also known as Application Programming Interface (**API**).
- It includes **hundreds of classes** and **methods grouped** into **several packages** according to their functionality.

Environment

- **Java Development Kit (JDK)** – Development environment to build java applications

JDK = Java Runtime Environment (JRE) + Development Tool

- **Java Runtime Environment (JRE)** – It provides the minimum requirements for executing a java application

JRE = Java Virtual Machine (JVM) + Library Classes

Environment

- **Java Virtual Machine (JVM)** – **Code execution component** of java application. It **provides runtime environment** in which **java byte code** can be executed.

Java Interpreter + Just-In-Time Compiler

Note

- JVM, JRE and JDK are **ported to different platforms** to provide **hardware and operating system-independence**.

Environment



Features

- **Simple** – Java syntax based on earlier languages like C and C++. Designed considering the **pitfalls** of earlier languages
- **Object Oriented** – Java supports all the **Object Oriented features**.
- **Secured** – No explicit **pointer** support, run inside the java **virtual machine sandbox**. The **class loader, byte code verifier and security manager** component of JVM enable the secured environment. Java also supports application developer to use other security mechanism like **SSL, JAAS, Cryptography**, etc.

Features

- **Robust** – Java have strong memory management, lack of pointers that avoids security problems, automatic garbage collection, exception handling and the type checking.
- **Platform Independent** - Once compiled ,code will be run on any platform without recompiling or any kind of modification -“**Write Once Run Anywhere(WORA)**”. This is made possible by making use of a **Java Virtual Machine(JVM)**.

Note

- JVM, JRE and JDK are **ported to different platforms** to provide **hardware and operating system-independence**.

Features

- **Architecture-neutral** – the size of **primitive types** is fixed
- **Distributed** – Java enable to create distributed application with the help of RMI and EJB
- **Multi-threaded** – Java run application concurrently.

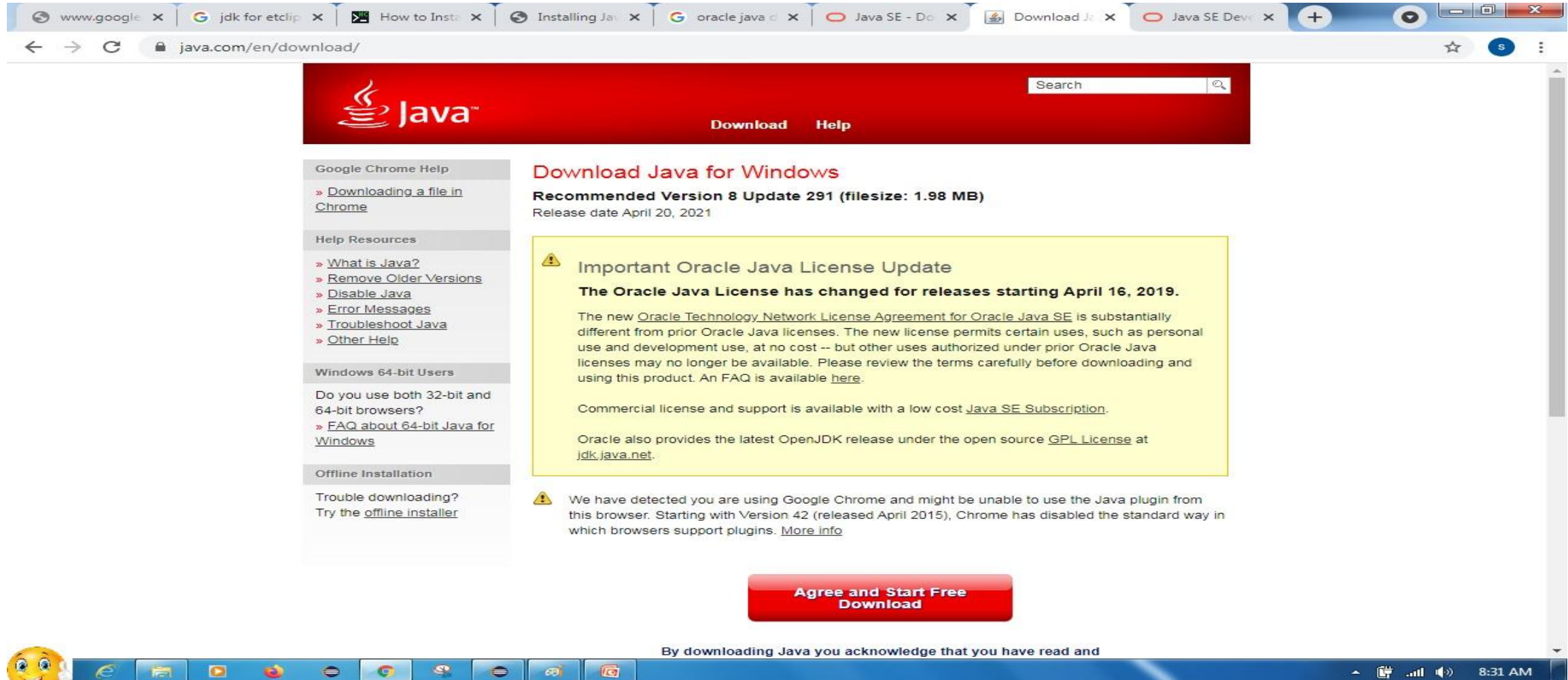
Environmental Setup

- To **run java applications**, we need to prepare the **environmental setup**. For installing Java we need the following:
- The Java Runtime Environment (**JRE**) (Includes Java Virtual Machine (JVM))
- The Java Developer Kit (**JDK**) (Includes the Java Compiler)
- A text editor
- If JDK is installed, you automatically get the JRE
 - It can be **downloaded** here:

<https://www.oracle.com/in/java/technologies/javase-downloads.html>

Introduction to Java

Environmental Setup



The screenshot shows a web browser window with multiple tabs open. The active tab is 'java.com/en/download/'. The page features a red header with the Java logo and navigation links for 'Download' and 'Help'. A search bar is also present. On the left, there is a sidebar with 'Google Chrome Help' and 'Help Resources' sections. The main content area is titled 'Download Java for Windows' and highlights the 'Recommended Version 8 Update 291 (filesize: 1.98 MB)' with a release date of April 20, 2021. A prominent yellow box contains an 'Important Oracle Java License Update' notice, stating that the Oracle Java License has changed for releases starting April 16, 2019. Below this, a red button reads 'Agree and Start Free Download'. At the bottom, a blue bar contains the text: 'By downloading Java you acknowledge that you have read and'.

Java Installation Steps

1. Go to the Java Downloads Page and click on the option of **Download**.
2. After clicking **Download**, you will be redirected to a page, select the **Accept License Agreement** button.

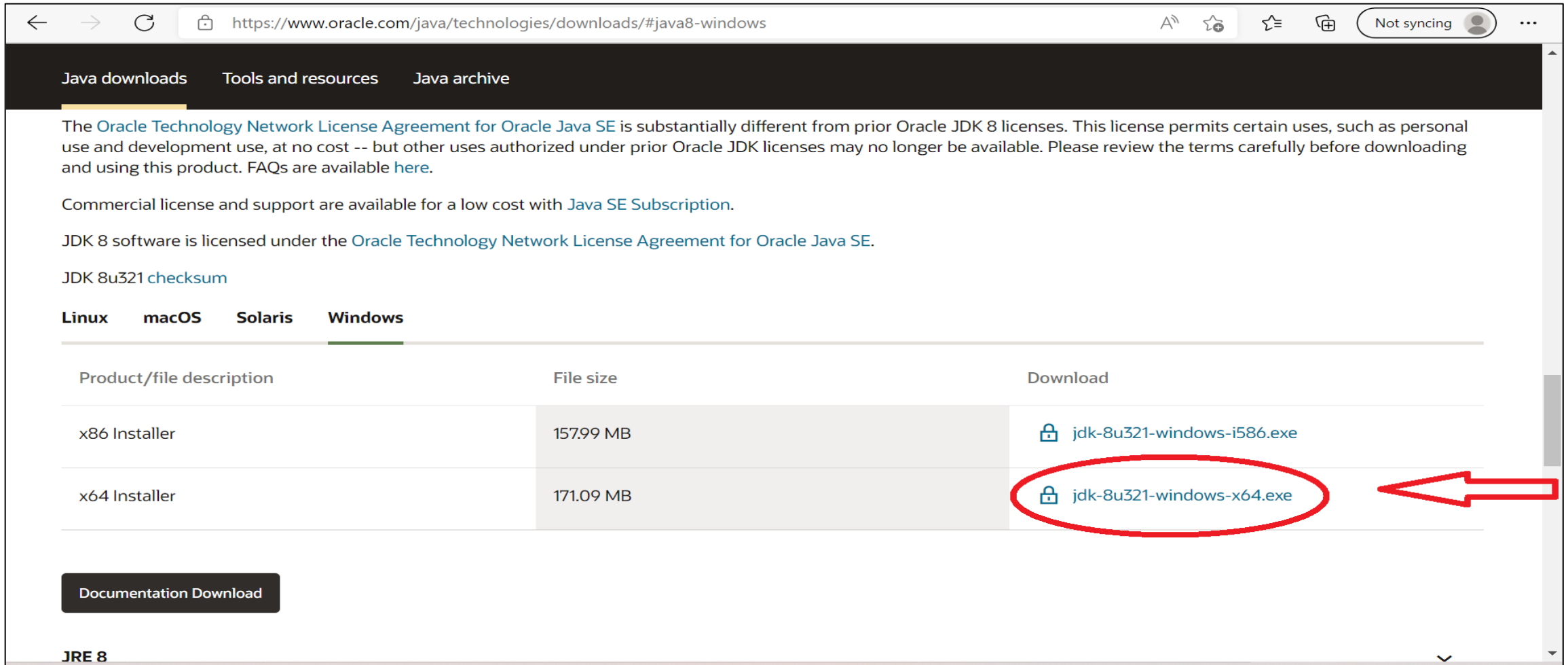
Choose a version based on the operating system

Here, I have chosen **jdk-8u211-windows-x64.exe**

3. Now, once the file is downloaded, run the installer and keep clicking on **Next**, till you get a message of finished downloading.

Introduction to Java

Java Installation Steps



The [Oracle Technology Network License Agreement for Oracle Java SE](#) is substantially different from prior Oracle JDK 8 licenses. This license permits certain uses, such as personal use and development use, at no cost -- but other uses authorized under prior Oracle JDK licenses may no longer be available. Please review the terms carefully before downloading and using this product. FAQs are available [here](#).

Commercial license and support are available for a low cost with [Java SE Subscription](#).

JDK 8 software is licensed under the [Oracle Technology Network License Agreement for Oracle Java SE](#).

JDK 8u321 [checksum](#)

Linux **macOS** **Solaris** **Windows**

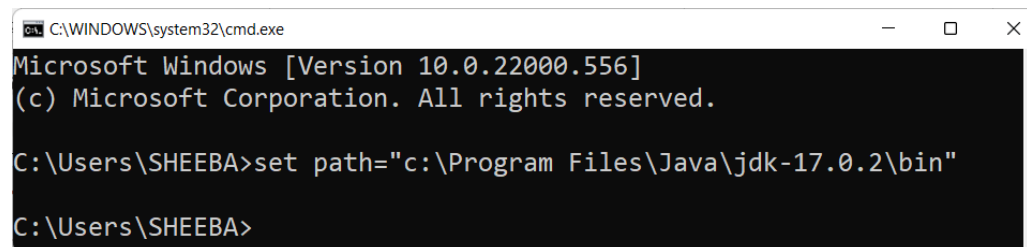
Product/file description	File size	Download
x86 Installer	157.99 MB	 jdk-8u321-windows-i586.exe
x64 Installer	171.09 MB	 jdk-8u321-windows-x64.exe

[Documentation Download](#)

JRE 8

First Java Program in Command Prompt

- You can write the code using any one of the text editor like **notepad** and **execute in Command prompt**,
- Before executing the code, need to set the path for the Java file.
- The path is required to be set for using tools such as javac, java, etc.
- If we are saving the **Java source file inside the JDK/bin directory**, the path is not required to be set because all the tools will be available in the current directory.
- If we have a **Java file outside the JDK/bin folder**, it is necessary to set the path of JDK.
- We can set the path of Java file in two ways as follows
 - **Temporary path**: Open CMD, copy the path of the JDK/Bin directory and write in the command prompt: `set path = copied_path( path where your software has installed)`.



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.22000.556]
(c) Microsoft Corporation. All rights reserved.

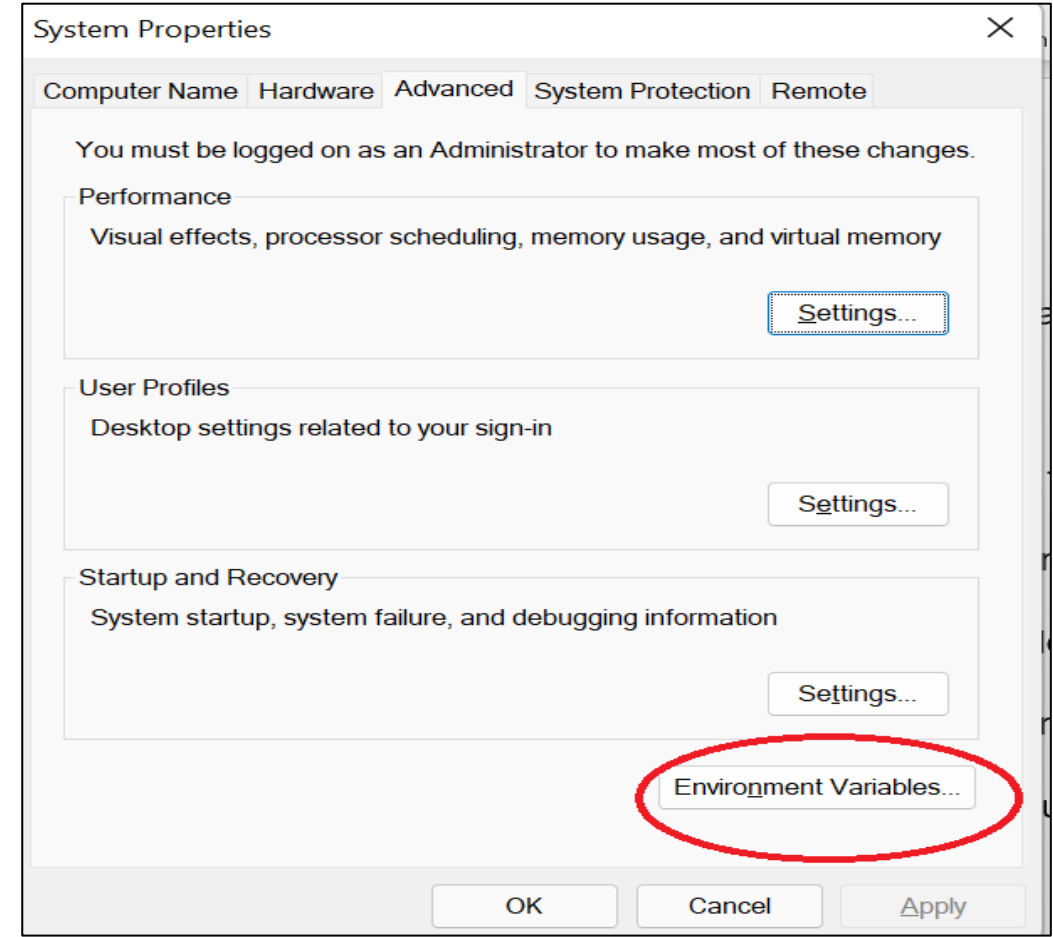
C:\Users\SHEEBA>set path="c:\Program Files\Java\jdk-17.0.2\bin"

C:\Users\SHEEBA>
```

First Java Program in Command Prompt

Permanent Path:

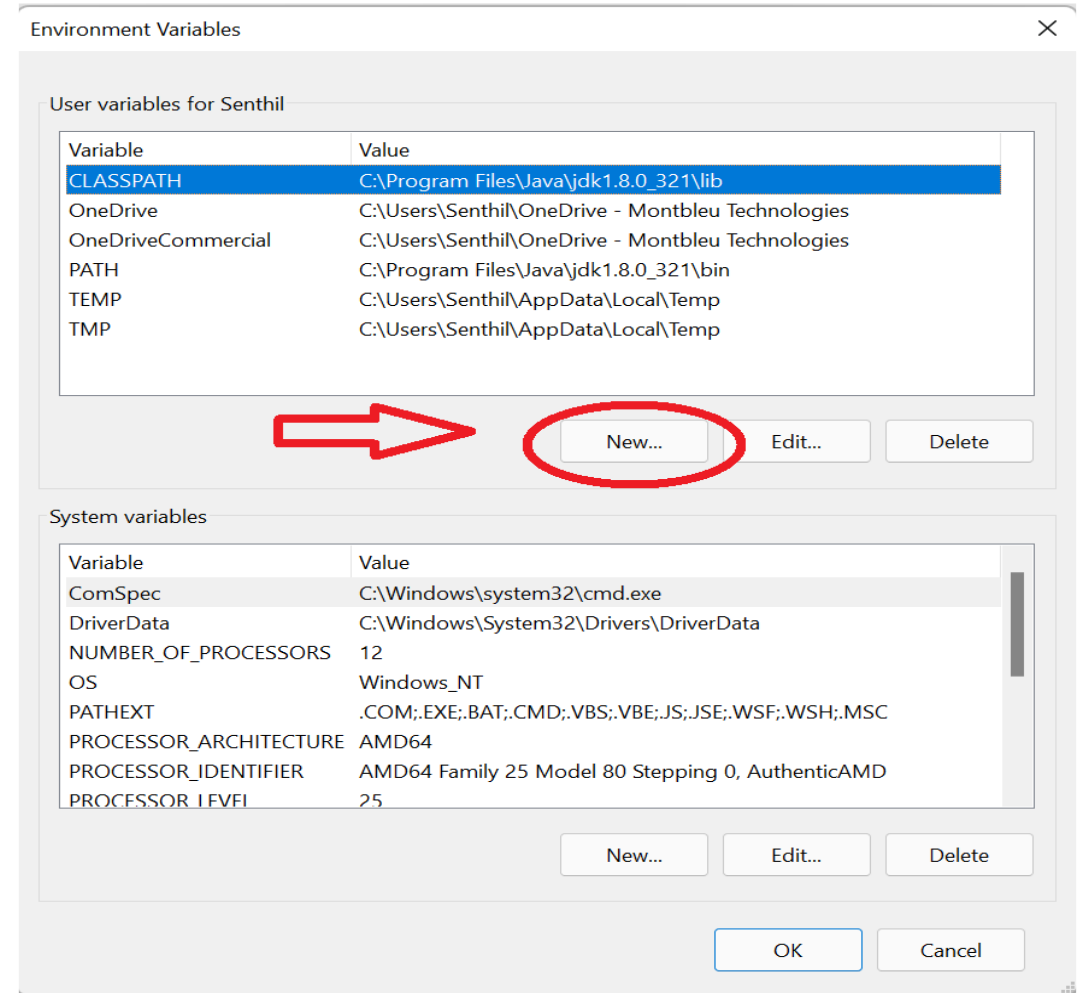
- Go to Start and search for '**System**'. Then, click on 'System' and go to Advanced System Settings.
- Now, click on '**Environment Variables**' under the 'Advanced' tab as shown here



Introduction to Java

Fist Java Program in Command Prompt

- Next, under **User Variables** choose New.
- Enter the variable name as '**JAVA_HOME**' and the full path to Java installation



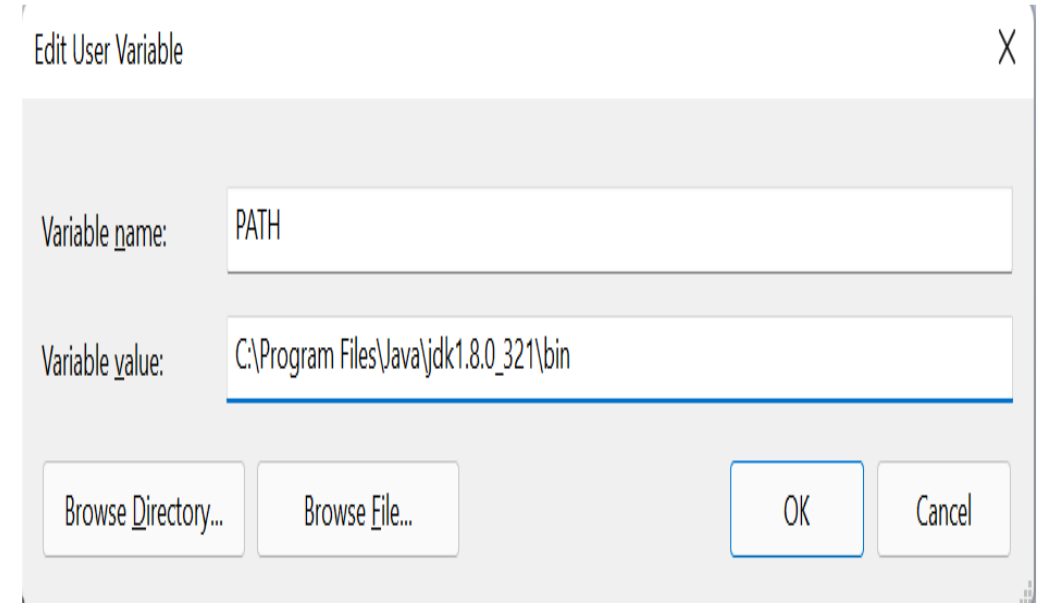
Introduction to Java

First Java Program in Command Prompt

- Next, **edit the path** of the system variable as shown here.
- In the row of 'Variable value', enter the path of the folder(enter the path of the software have installed).

```
path="c:\Program Files\Java\jdk-17.0.2\bin"
```

- Click OK .



Edit User Variable

Variable name: PATH

Variable value: C:\Program Files\Java\jdk1.8.0_321\bin

Browse Directory... Browse File... OK Cancel

Introduction to Java

First Java Program in Command Prompt

- You can either use temporary or permanent methods to set the path.
- Once have you completed, check the software have installed and the path is properly set by typing the command in CMD ***“java -version”***.
- This command will display the **installed version of Java** in your system.

```
C:\>javac -version
javac 17.0.1
C:\>
```

First Java Program in Command Prompt

Note:

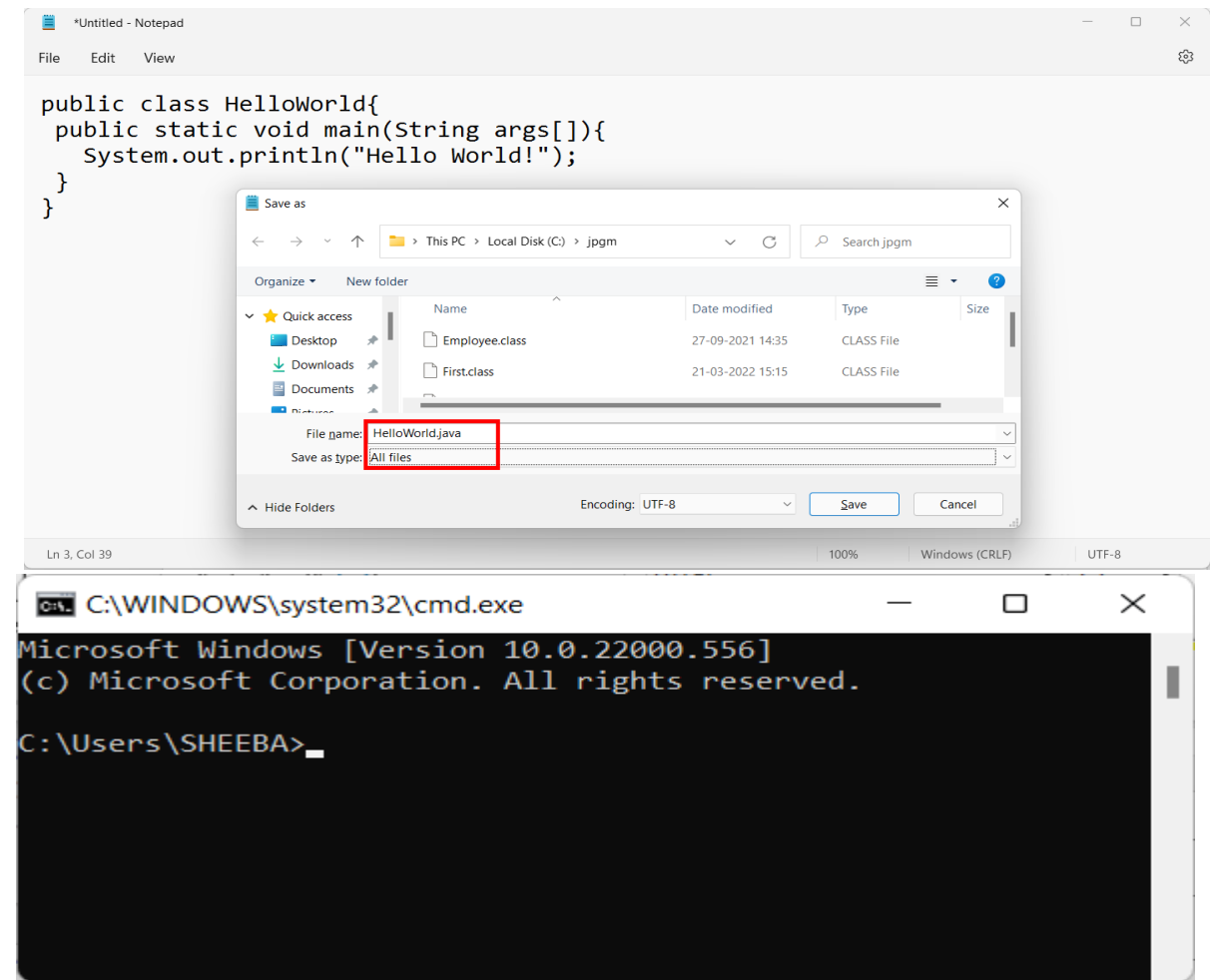
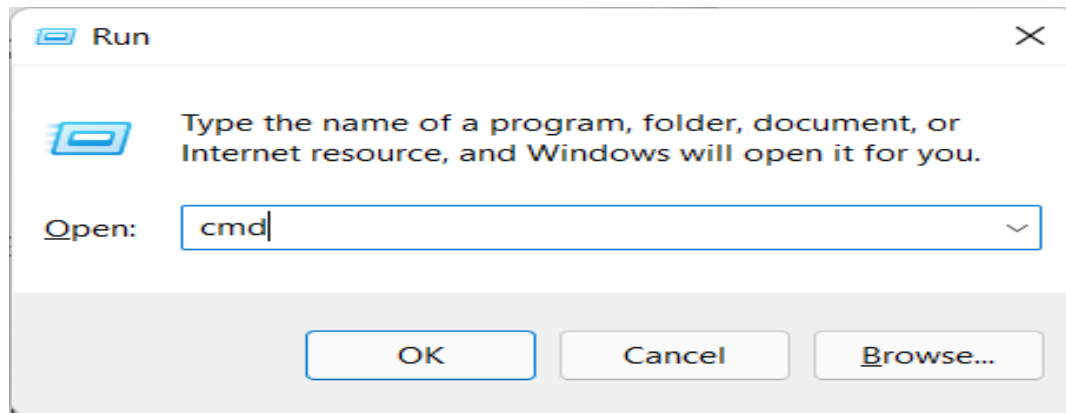
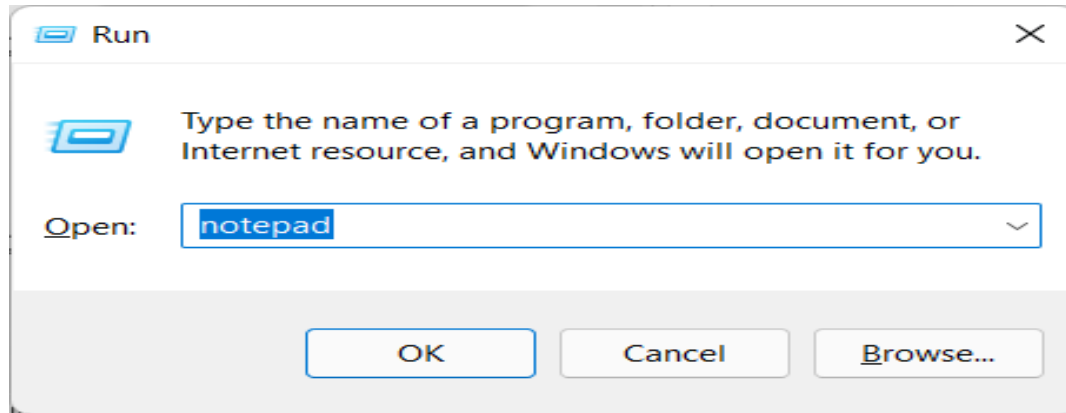
- If you set the path with the **permanent method**, then **no need to set the path for every individual java program.**
- But In the case of the **temporary method**, whenever you are opening the CMD you need to set the path.
- Once you have completed the path set continue with the execution of your code.

First Java Program in Command Prompt

- To execute the Java program, the following steps need to be followed
 - **Step 1:** Open the notepad or press the *Windows + R*, type notepad, and click Ok.
 - **Step 2:** Write a Java program that you want to compile and run in the notepad.
 - **Step 3:** To save a Java program press Ctrl + S key and provide the file name. Remember that the file name must be the same as the **class name** followed by the **.java** extension.
 - **Example:** Writing the java program in the notepad with the class name “*HelloWorld*”
in notepad and saving the file as **HelloWorld.Java**.
 - **Step 4:** To compile and run the program, open the **Command Prompt** by pressing **Windows Key + R**, type **CMD**, and press **enter** key or click on the **Ok** button.

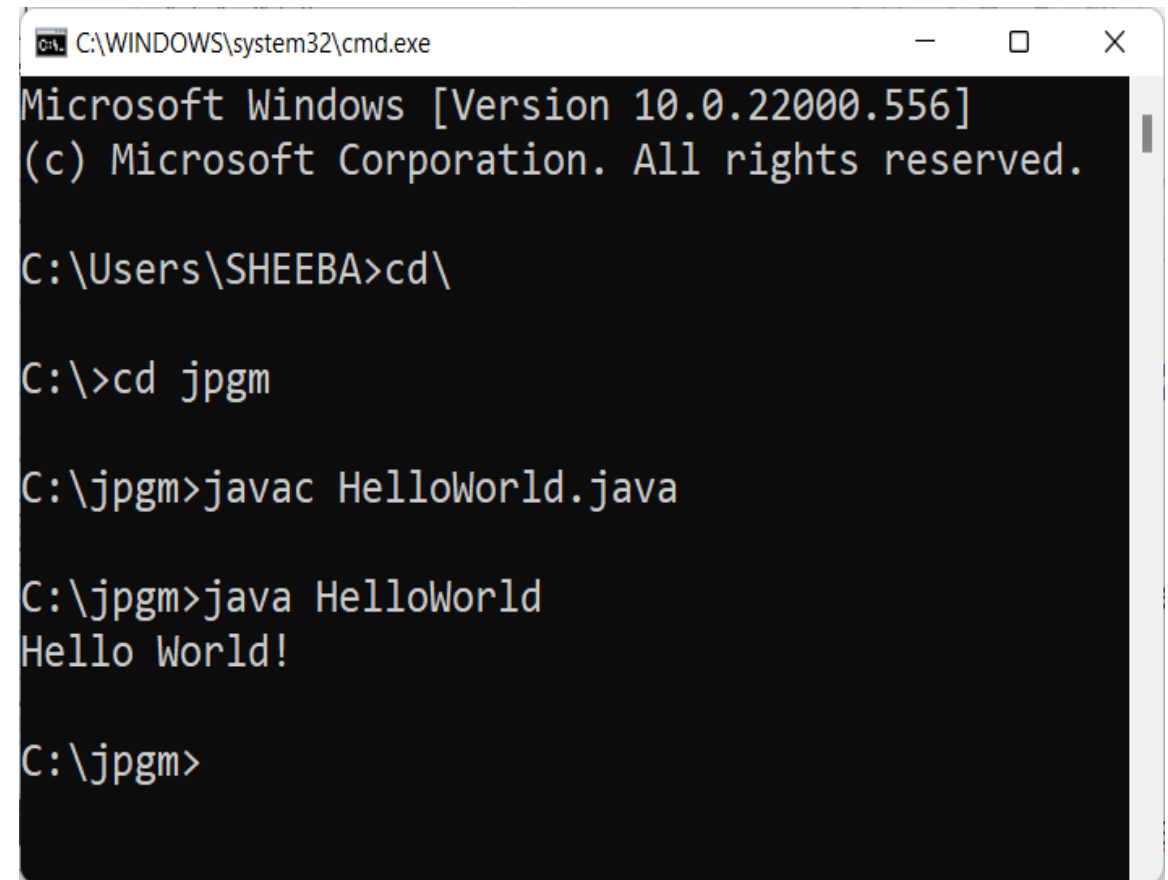
Introduction to Java

First Java Program in Command Prompt



First Java Program in Command Prompt

- **Step 5:** To compile the Java program type the following comment
 - **Example:** **javac** HelloWorld.java
- **Step 6:** To execute the Java program type the following comment
 - **Example:** **java** HelloWorld



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.22000.556]
(c) Microsoft Corporation. All rights reserved.

C:\Users\SHEEBA>cd\

C:\>cd jpgm

C:\jpgm>javac HelloWorld.java

C:\jpgm>java HelloWorld
Hello World!

C:\jpgm>
```

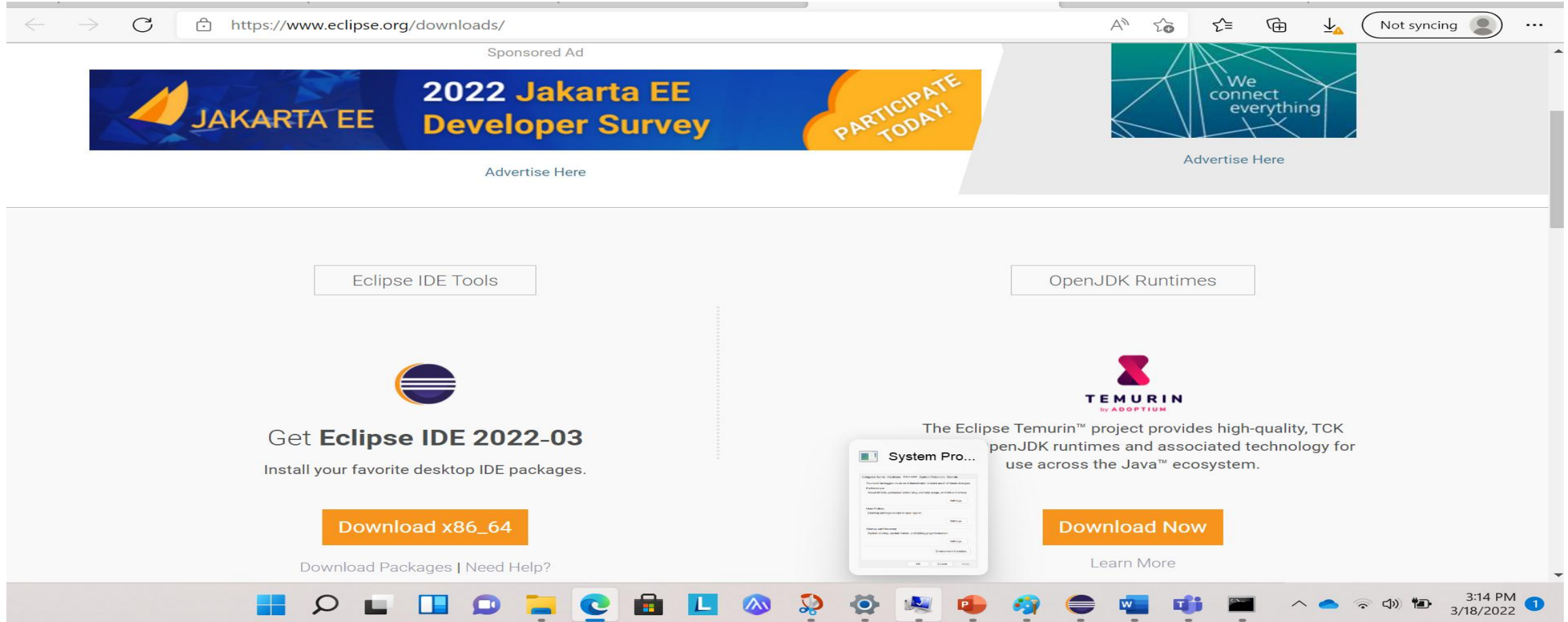
Eclipse IDE

- Using a text editor is the basic way of creating a Java program, but there are tools available to make it easier to create Java programs called **IDEs** like **Eclipse**.
- **Eclipse** is a **third-party open-source IDE** that is very powerful and popular.
 - It can be downloaded here: <http://www.eclipse.org/downloads/>

Introduction to Java

Eclipse IDE

Eclipse download page.



The screenshot shows the Eclipse download page in a web browser. The address bar displays `https://www.eclipse.org/downloads/`. At the top, there is a sponsored advertisement for the "2022 Jakarta EE Developer Survey" with a "PARTICIPATE TODAY!" button. Below the ad, the page is divided into two main sections. The left section, titled "Eclipse IDE Tools", features the Eclipse logo and the text "Get **Eclipse IDE 2022-03** Install your favorite desktop IDE packages." Below this is a large orange button labeled "Download x86_64" and a link "Download Packages | Need Help?". The right section, titled "OpenJDK Runtimes", features the "TEMURIN by ADOPTIUM" logo and the text "The Eclipse Temurin™ project provides high-quality, TCK openJDK runtimes and associated technology for use across the Java™ ecosystem." Below this is a large orange button labeled "Download Now" and a link "Learn More". The browser's taskbar at the bottom shows various application icons, including Windows, Edge, and several development tools. The system clock in the bottom right corner indicates the time is 3:14 PM on 3/18/2022.

Introduction to Java

Eclipse IDE

Choose an option from the list

Home / Downloads / Packages / Release / Eclipse Photon / R

Eclipse InstallerEclipse PackagesEclipse Developer Builds

Eclipse Photon R Packages

 Yatta Launcher for Eclipse Install, launch, and share your Eclipse IDE. Stop configuring. Start Coding.		
 Eclipse IDE for Eclipse Committers 188 MB 138,423 DOWNLOADS Package suited for development of Eclipse itself at Eclipse.org, based on the Eclipse Platform adding PDE, Git, Marketplace Client, source code and developer documentation. Click here to file a bug against Eclipse Platform. Click here to file a bug against Eclipse Git team provider.		Windows 32-bit 64-bit Mac Cocoa 64-bit Linux 32-bit 64-bit
 Eclipse IDE for C/C++ Developers 222 MB 122,593 DOWNLOADS An IDE for C/C++ developers with Mylyn integration.		Windows 32-bit 64-bit Mac Cocoa 64-bit Linux 32-bit 64-bit
 Eclipse IDE for Java and DSL Developers 345 MB 82,548 DOWNLOADS The essential tools for Java and DSL developers, including a Java & Xtext IDE, a DSL Framework (Ptext), a Git client, XML Editor, and Maven integration.		Windows 32-bit 64-bit Mac Cocoa 64-bit
 Eclipse IDE for Java Developers 193 MB 354,799 DOWNLOADS The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Mylyn, Maven and Gradle integration.		Windows 32-bit 64-bit Mac Cocoa 64-bit Linux 32-bit 64-bit
 Eclipse IDE for JavaScript and Web Developers 372 MB 20,303 DOWNLOADS The essential tools for JavaScript and Web developers, including a JavaScript IDE, a Git client, XML Editor, Mylyn, Maven and Gradle integration.		Windows 32-bit 64-bit Mac Cocoa 64-bit Linux 32-bit 64-bit




Get Eclipse PHOTON
 Install your favorite Eclipse packages.
[Download 32-bit](#) [Download 64-bit](#)
[Download Packages](#) | [Need Help?](#)

RELATED LINKS

- Compare & Combine Packages
- New and Noteworthy
- Install Guide
- Documentation
- Updating Eclipse
- Forums

Other versions

MORE DOWNLOADS

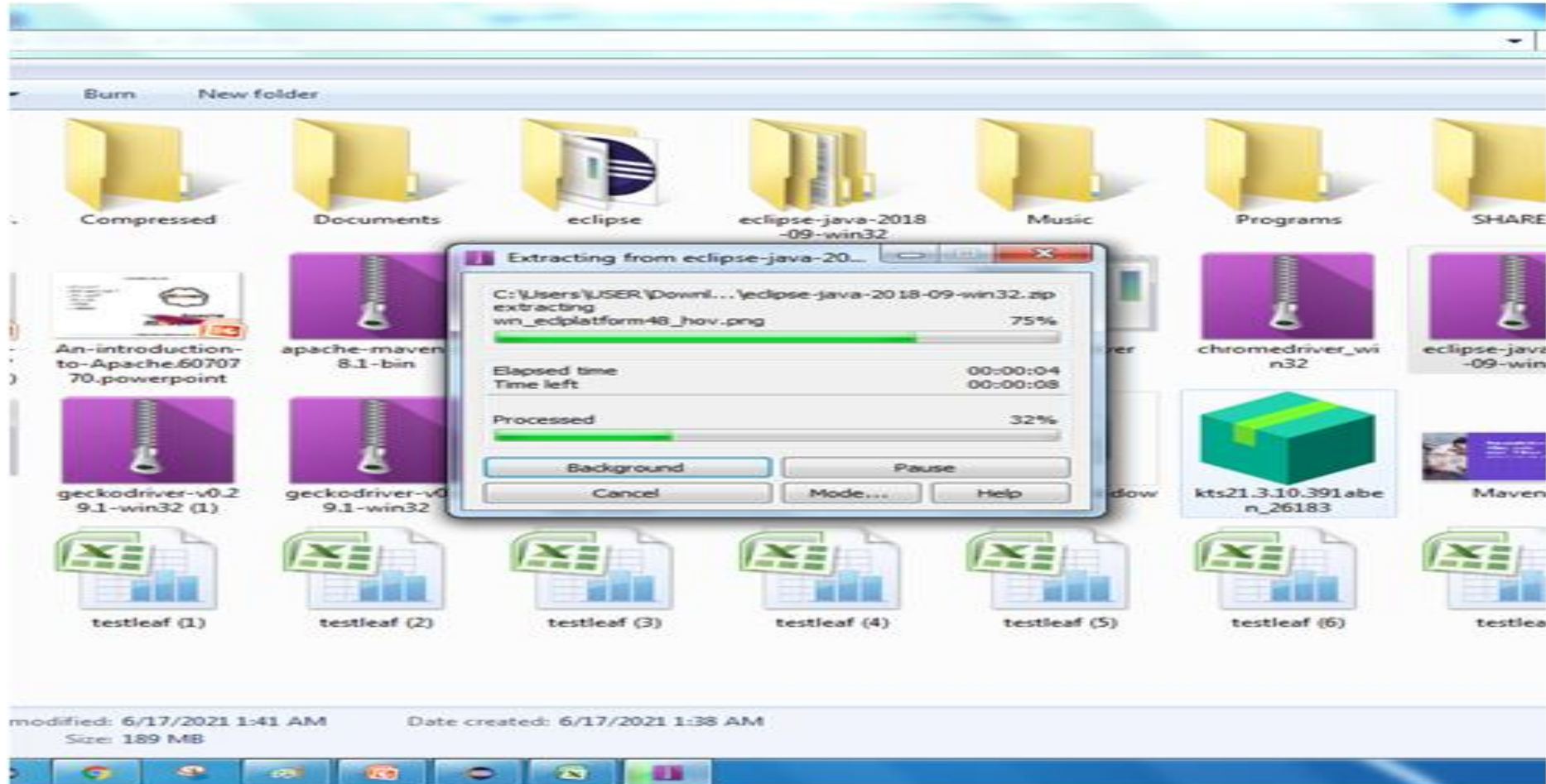
- Other builds
- Eclipse Photon [4.8]
- Eclipse Oxygen [4.7]
- Eclipse Neon [4.6]
- Eclipse Mars [4.5]
- Eclipse Luna [4.4]

The IDE you need

Introduction to Java

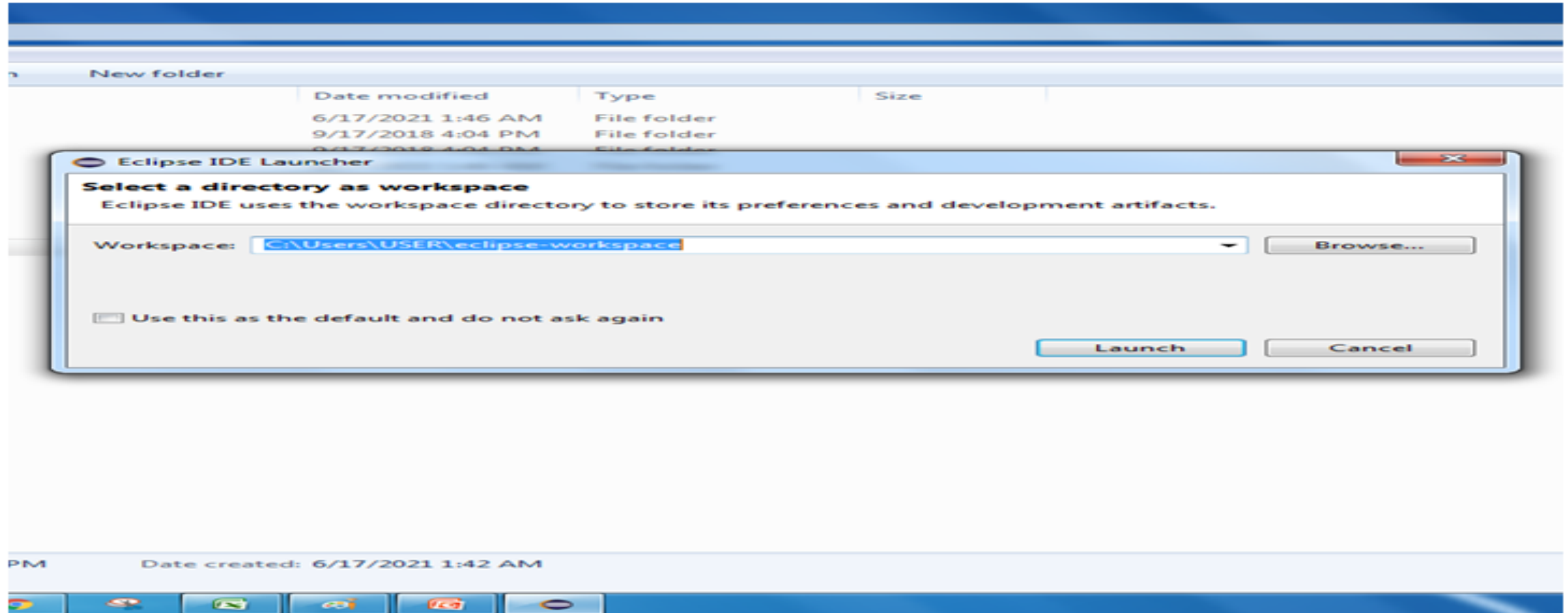
Eclipse IDE

After download, right click and extract the eclipse file



Eclipse IDE

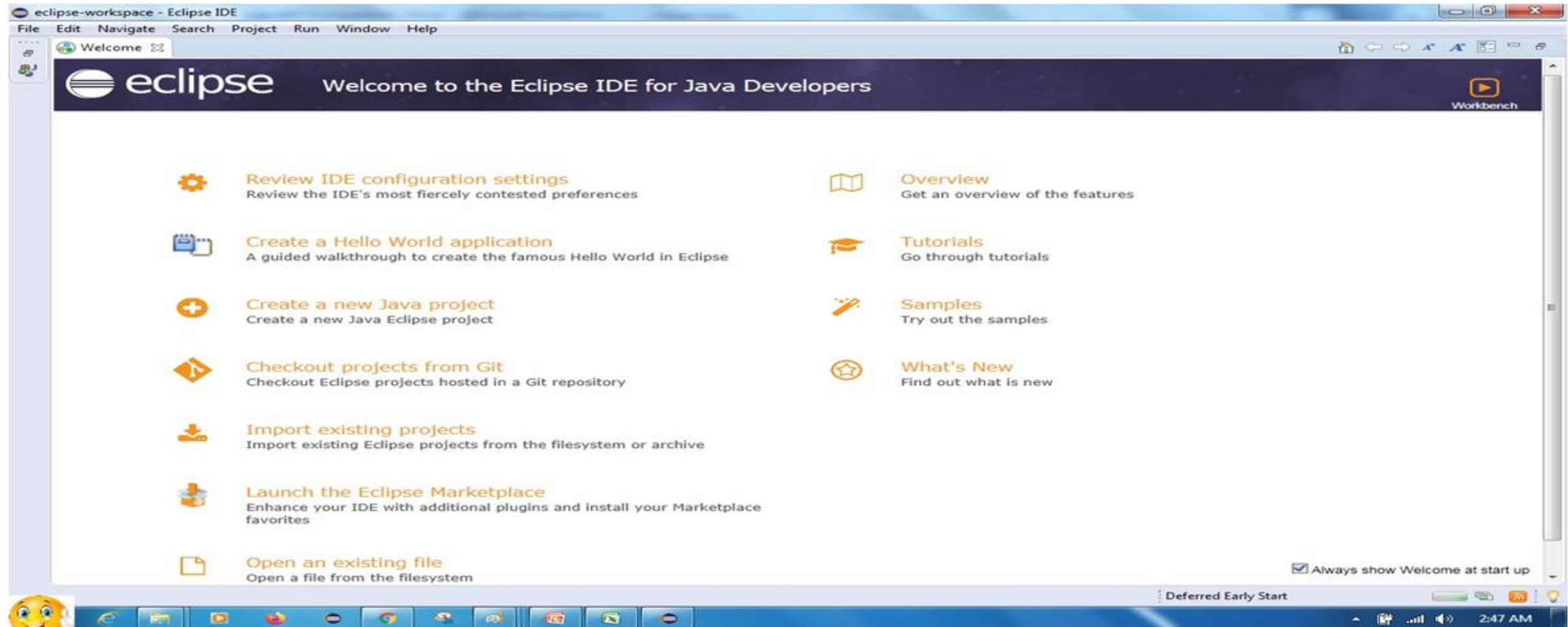
Configuring Eclipse: **Select a directory to create a workspace for projects**



Introduction to Java

Eclipse IDE

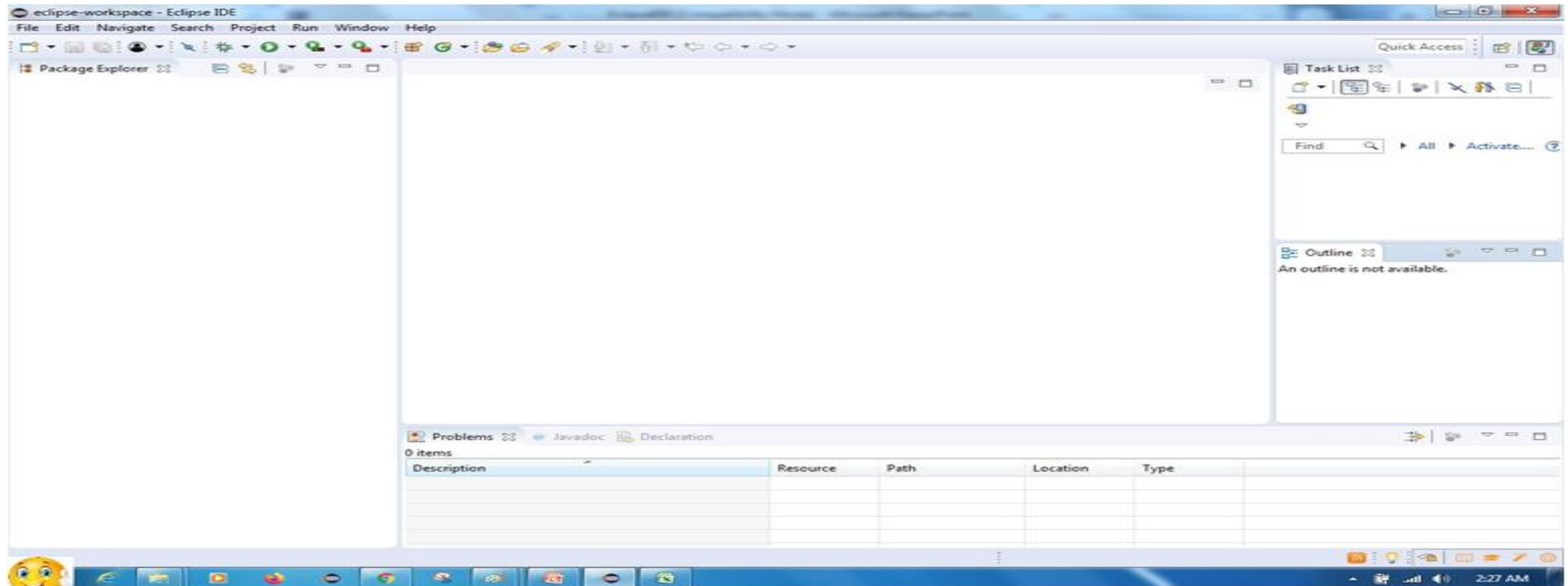
Once the workspace is chosen, you will get the welcome page



Introduction to Java

Eclipse IDE

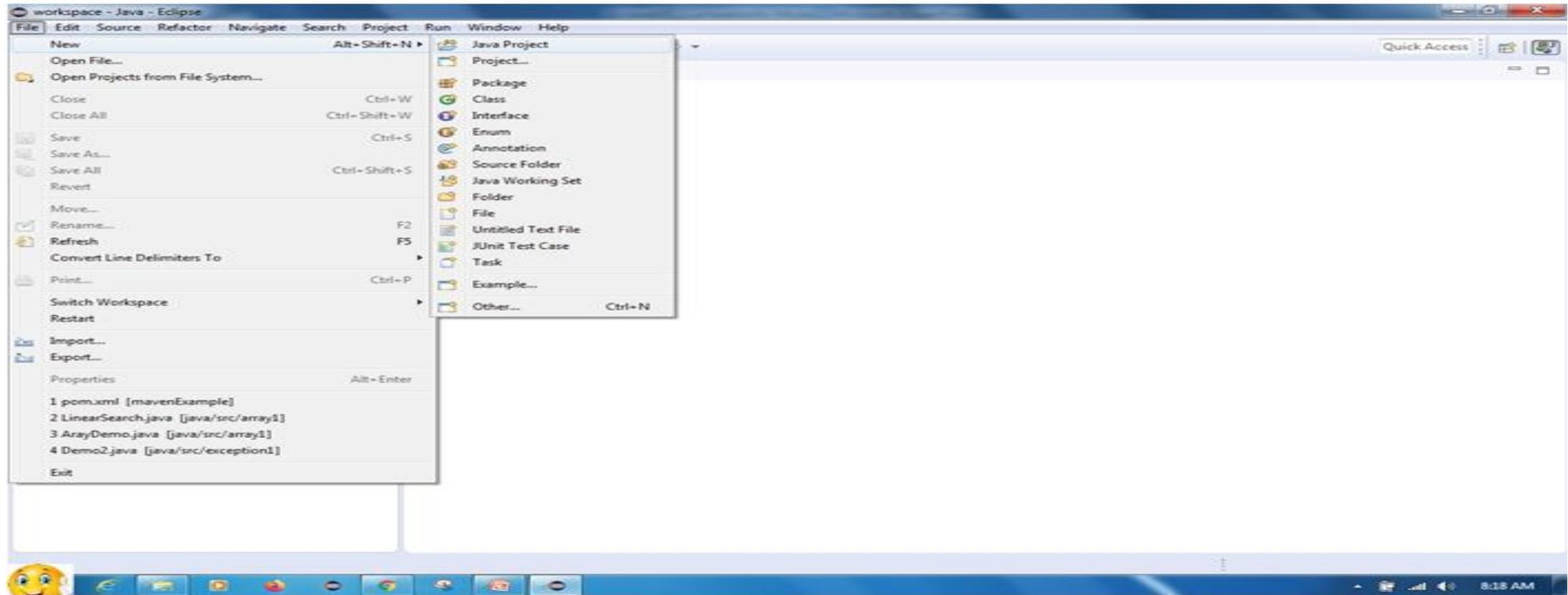
After closing the welcome screen ,the project explorer and other options will be shown up.
 Now you ready are to start working.



Introduction to Java

Eclipse IDE

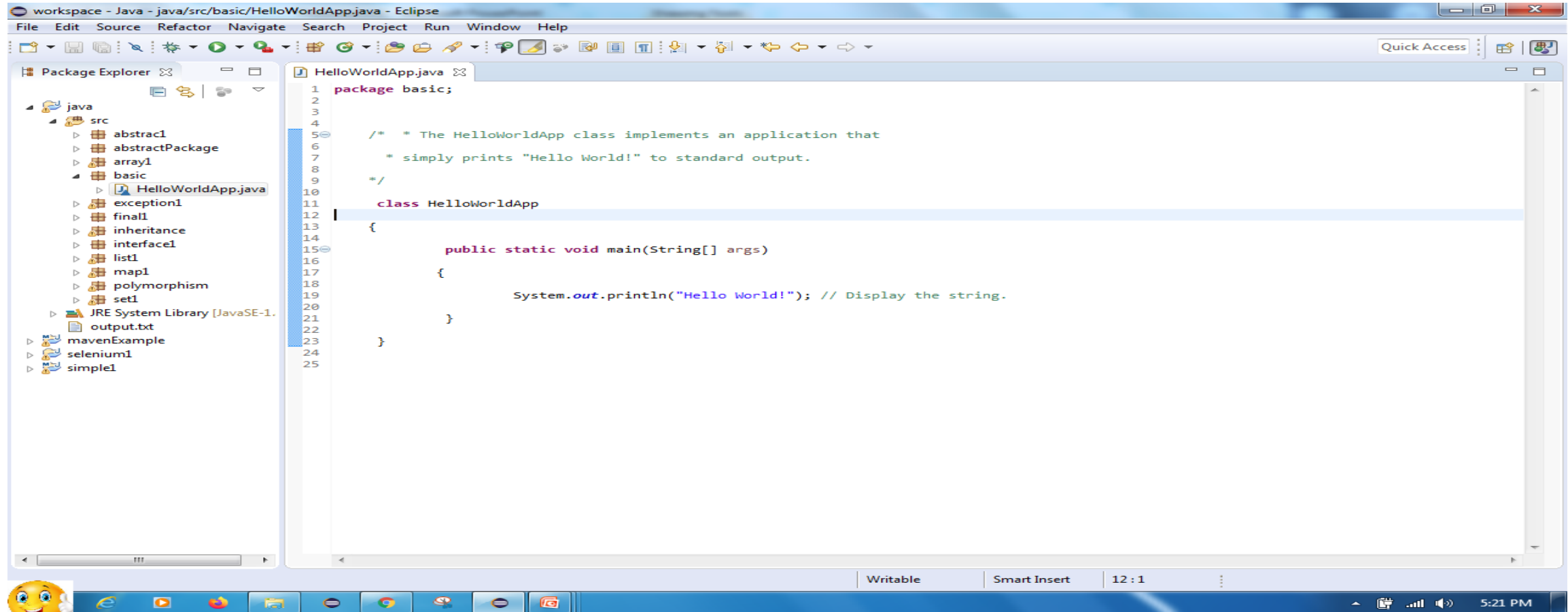
Start working by creating the Java Projects



Introduction to Java

Eclipse IDE

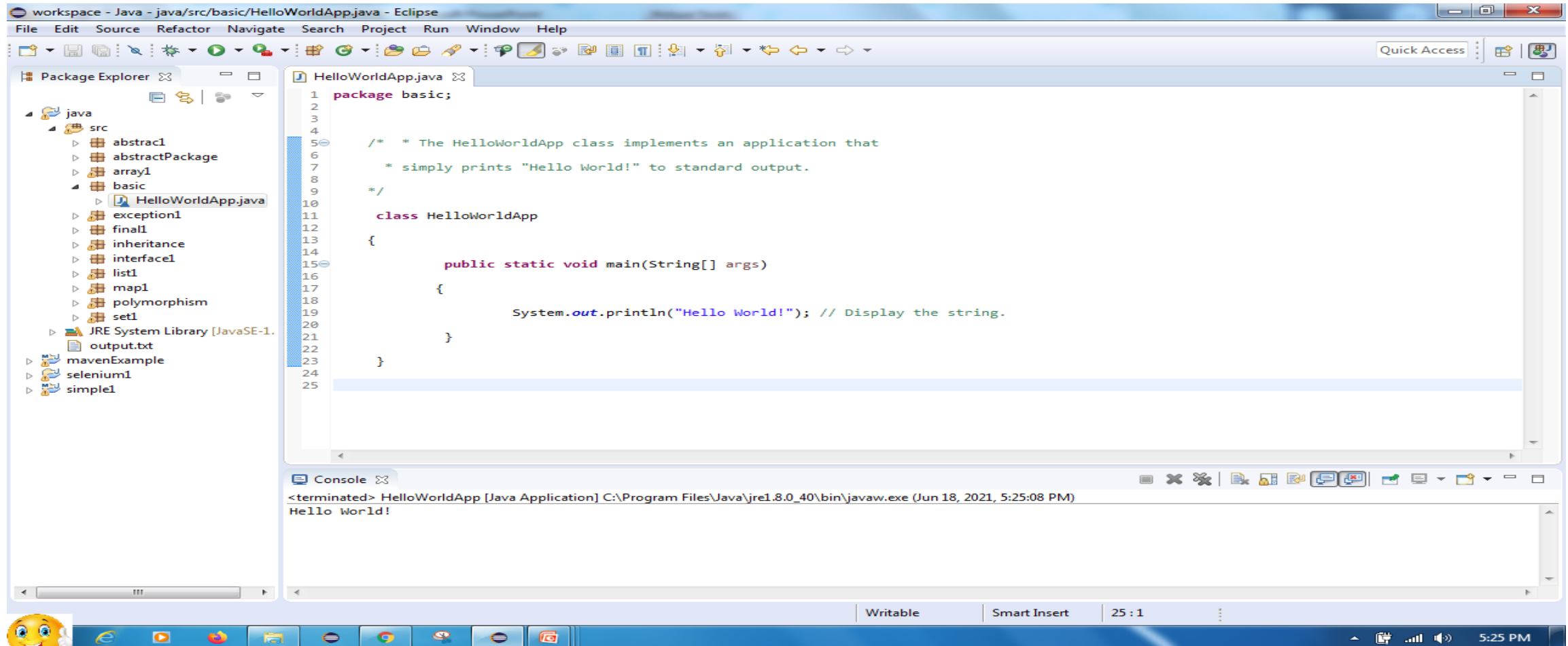
Create First Java Program with file extension .java Example: HelloWorldApp.java



Introduction to Java

Eclipse IDE

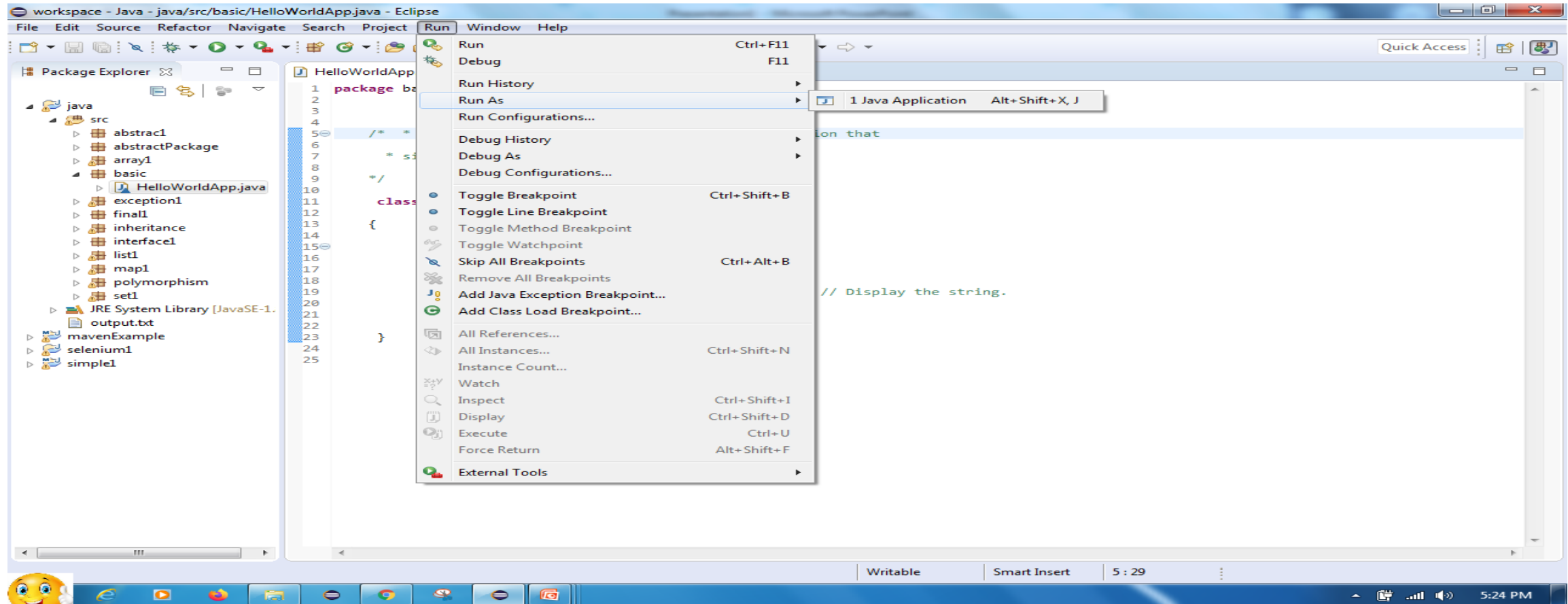
After run Print Hello World!



Introduction to Java

Eclipse IDE

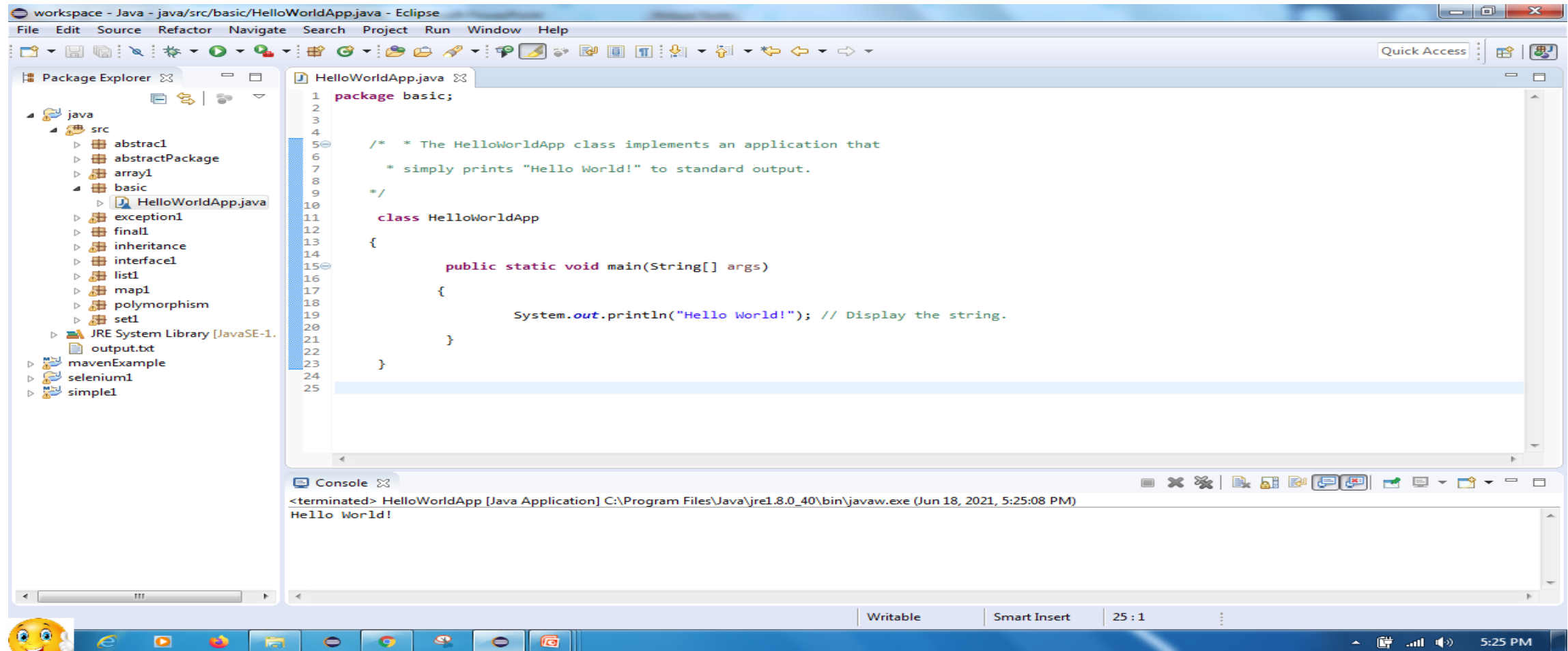
Create First Java Program with file extension .java Example: HelloWorldApp.java



Introduction to Java

Eclipse IDE

After run Print Hello World!



First Java Program

```
/**  
  
* The HelloWorldApp class implements an application that  
  
* simply prints "Hello World!" to standard output.  
  
*/  
  
class HelloWorldApp {  
  
    public static void main(String[] args){  
  
        System.out.println("Hello World!"); // Display the Hello World!  
  
    }  
  
}
```

Need of Coding standards

- Code conventions are important to the programmer for the following reasons
 - To facilitate the **copying, changing, and maintenance** of the code.
 - To maintain the **readability** of the code.
 - Helps to **understand the code quickly** and thoroughly.
 - Helps to **well package the code** when deploying as the product.

General coding practice

- All code should have the following attribute:
 - **Simplicity:** Code should be easily understood and be concise.
 - **Readability:** Describe the code with comments and name the Identifiers relative to the objective of the code and ensure other people can understand the code.
 - **Modularity:** When there is a need to redo the small fraction of code then practice using the existing or reusing the code.
 - **Efficiency:** Code should be fast and economical. When using data files, read a value once and store it in a variable – don't go back and forward for the same value. Close connections if they are not required. Do not hold onto references to variables if not required, so as not to impose a memory leak.

Java source files

- Source file of java can be saved with the extension or suffixes “**Filename.java**”.
- **Java source file has the following order:**
 - Comments
 - Package and import statements
 - Class and interface declaration

Indentation

- In Java, the exact construction of the indentation is **unspecified**.
- so, set the tabs exactly for every **8 spaces**.
- The line length should **not exceed 80 characters**.
- When the statement is not fit in a single line, then wrap the lines based on the following principles.
 - Break the lines **after a comma**.

Example:

```
public void Cart(string product_name, double cost,  
                int quantity, string description)
```

Indentation

- Break the line **before an operator**.
- Prefer **higher-level breaks** to lower-level breaks.

Example: //Breaking an arithmetic expression before an operator and outside the parenthesized expression at a high level.

```
Calculation = a*(b + c - d)  
              + 4 * b
```

- Align the **new line** with the beginning of the expression at the same level on the previous line.
- If the above rules lead to confusing code or to code that's squished up against the right margin, just **indent 8 spaces** instead.

Indentation

Example: //8 space indentation is used to breaking the if statements

```
if ((condition1 && condition2)
    || (condition3 && condition4)
    ||!(condition5 && condition6)) {
    Method();
}
```

- The **ternary expression** can be formatted as below:

```
Variable = Expression1 ? Expression 2 : Expression 3;
```

```
Variable = Expression1 ? Expression 2
                        : Expression 3;
```

```
Variable = Expression1
        ? Expression 2
        : Expression 3;
```

Indentation

Blank Lines:

- Blank lines improve **readability**.
- **Two blank lines** should always be used in the following circumstances:
 - Between sections of a source file.
 - Between class and interface definitions
- **One blank line** should always be used in the following circumstances:
 - Between methods
 - Between the local variables in a method and its first statement
 - Before a block or single-line comment.
 - Between logical sections inside a method.

Indentation

Blank Spaces:

Blank Spaces are used to improve the **reliability of the code** and can be used in the following circumstances

- A keyword followed by a **parenthesis should be separated by a space.**

Example:

```
while (False){  
    }
```

- It should **not** be used between a **method name and its opening parenthesis**. This helps to distinguish keywords from method calls.
- This can be included after **commas in an argument list**.

Indentation

- **Except ‘.’ and unary operators** all the binary operators should be separated their **operands by spaces**.
- **Expression** in the **for statement** should be separated using **space**.

Comments

- Comments are the **overview** or **description** of the code.
- In Java, the comments can be implemented in **four styles** as follows
- **Block comments:**
 - Block comments (**`/**`**) are used to provide **descriptions of files, methods, data structures, and algorithms.**
 - Block comments may be used at the **beginning** of each file and before each method and also within methods.
 - Block comments **inside** a function or method should be indented to the **same level** as the code they describe.

Example:

```
/*  
*Here is a block comment  
*/
```

Comments

- **Single-Line comments:**

- **Short description of a block of code** can be given using single-line comments `(/*..*/)`.
- If a comment can't be written in a single line, it should follow the block comment format.

Example:

```
if(condition) {  
    /* Description about block of code in a single line */  
}
```

Comments

- **Trailing comments:**
 - Trailing comments (**`/*..*/`**) are **very short comments** that describe the statements.
 - These comments can be included in the **same line** of the statement but should be **separated from the statement using spaces**.
 - If more than **one short comment** appears in a code, they should all be **indented to the same tab**.

Example:

```
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");    /* Display the Hello World! */  
    }  
}
```

Comments

- **End - Of - Line comments:**
 - This comment line (**//**) can be used as a **single-line comment** or short comment for a statement.
 - These comments can be included in the **same line** of the statement but should be **separated** from the statement using **spaces**.
 - This can be used in **consecutive multiple lines** for commenting out sections of code.

Example:

```
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");    // Display the Hello World!  
    }  
}
```


Documentation Comments

- Documentation comments are generally used when **writing code for a project/software package**.
- It helps to generate a **documentation page for reference**, which can be used for getting information about the present method, its parameters, class, variables, etc..,
- The **JavaDoc** tool is used to process the doc comments that come with JDK and it is used for generating Java code documentation in HTML(HyperText Markup Language) format from Java source code, which requires documentation in a predefined format.
- It is made up of **two parts: a description and Javadoc tags**.

Documentation Comments

Javadoc Tag:

- **Special tag** embedded within a Java doc comment.
- These doc tags enable you **to autogenerate** a complete, **well-formatted API from your source code**.
- The tags **start** with an "at" sign (@) and are **case-sensitive** (i.e), they must be in upper and lowercase letters as shown in the below table.
- A tag must start at the beginning of a line (after any leading spaces and an optional asterisk) or it is treated as normal text.
- By convention, tags with the **same name are grouped together**.

Documentation Comments

Two Types of Javadoc tag :

- **Block tags** - Can be placed only in the **tag section** that follows the main description.
 - **Example:** @tag
- **Inline tags** - Can be placed anywhere in the main description or in the comments for block tags and

Inline tags are denoted by curly braces

- **Example:** {@tag}

Documentation Comments

Javadoc Tags:

- Below table describe the **Javadoc tag** that is used in documentation comments.

Tag	Description	Syntax
@author	Adds the author of a class.	@author name-text
{@code}	Displays text in code font without interpreting the text as HTML markup or nested javadoc tags.	{@code text}
{@docRoot}	Represents the relative path to the generated document's root directory from any generated page.	{@docRoot}
@deprecated	Adds a comment indicating that this API should no longer be used.	@deprecated deprecatedtext
@exception	Adds a Throws subheading to the generated documentation, with the classname and description text.	@exception class-name description

Documentation Comments

Tag	Description	Syntax
<code>{@inheritDoc}</code>	Inherits a comment from the nearest inheritable class or implementable interface.	Inherits a comment from the immediate superclass.
<code>{@link}</code>	Inserts an in-line link with the visible text label that points to the documentation for the specified package, class, or member name of a referenced class.	<code>{@link package.class#member label}</code>
<code>{@linkplain}</code>	Identical to <code>{@link}</code> , except the link's label is displayed in plain text than code font.	<code>{@linkplain package.class#member label}</code>
<code>@param</code>	Adds a parameter with the specified parameter-name followed by the specified description to the "Parameters" section.	<code>@param parameter-name description</code>
<code>@return</code>	Adds a "Returns" section with the description text.	<code>@return description</code>

Documentation Comments

Tag	Description	Syntax
@see	Adds a "See Also" heading with a link or text entry that points to reference.	@see reference
@serial	Used in the doc comment for a default serializable field.	@serial field-description include exclude
@serialData	Documents the data written by the writeObject() or writeExternal() methods.	@serialData data-description
@serialField	Documents an ObjectOutputStreamField component.	@serialField field-name field-type field-description
@since	Adds a "Since" heading with the specified since-text to the generated documentation.	@since release
@throws	The @throws and @exception tags are synonyms.	@throws class-name description

Documentation Comments

Tag	Description	Syntax
<code>{@value}</code>	When <code>{@value}</code> is used in the doc comment of a static field, it displays the value of that constant.	<code>{@value package.class#field}</code>
<code>@version</code>	Adds a "Version" subheading with the specified version-text to the generated docs when the -version option is used.	<code>@version version-text</code>

Documentation Comments

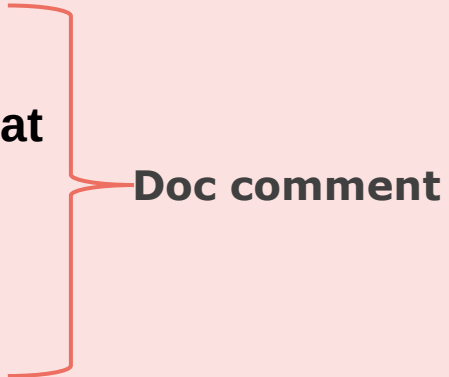
- **Format of Documentation comments.**
 - Each **line** above is **indented** to align with the **code below the comment**.
 - The **first line** contains the **begin-comment delimiter (/**)**.
 - Write the **first sentence** as a **short summary of the method**, as Javadoc automatically places it in the method summary table (and index).
 - The **inline tag** can be used **anywhere** that a comment can be written, such as in the text following block tags.

Documentation Comments

- If we have **more than one paragraph** in the doc comment, separate the paragraphs with a `<p>` **paragraph tag**.
- Insert a **blank comment line** between the **description** and the **list of tags**.
- The first line that begins with an "@" character **ends the description**.
- There is only **one description block** per doc comment; we **cannot continue the description following block tags**.
- The **last line** contains the **end-comment delimiter** `(*/)`.

First Java Program using Documentation Comments

```
/**  
 * The HelloWorldApp class implements an application that  
 * simply prints "Hello World!" to standard output.  
 * @author SmartCliff  
*/  
  
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");    // Display the Hello World!  
    }  
}
```



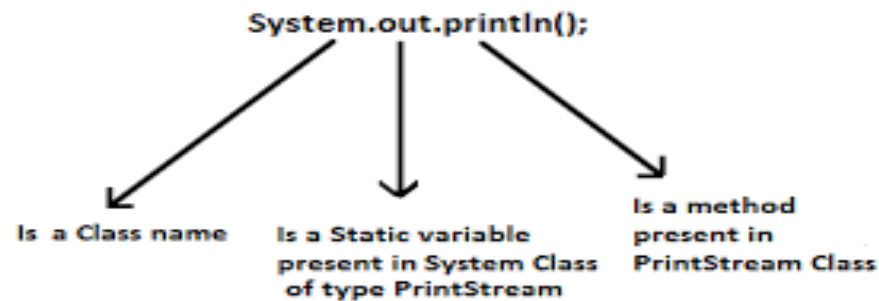
Doc comment

First Java Program in Detail

- **Class:** keyword is used to declare a class in java.
- **public** keyword is an access modifier which represents visibility, it means it is visible to all.
- **static** is a keyword, if we declare any method as static, it is known as static method.
- The core **advantage of static method** is that there is **no need to create object** to invoke the static method.
- The **main method is executed by the JVM**, so it doesn't require to create object to invoke the main method. So it saves memory.
- **void** is the return type of the method, it means it doesn't return any value.

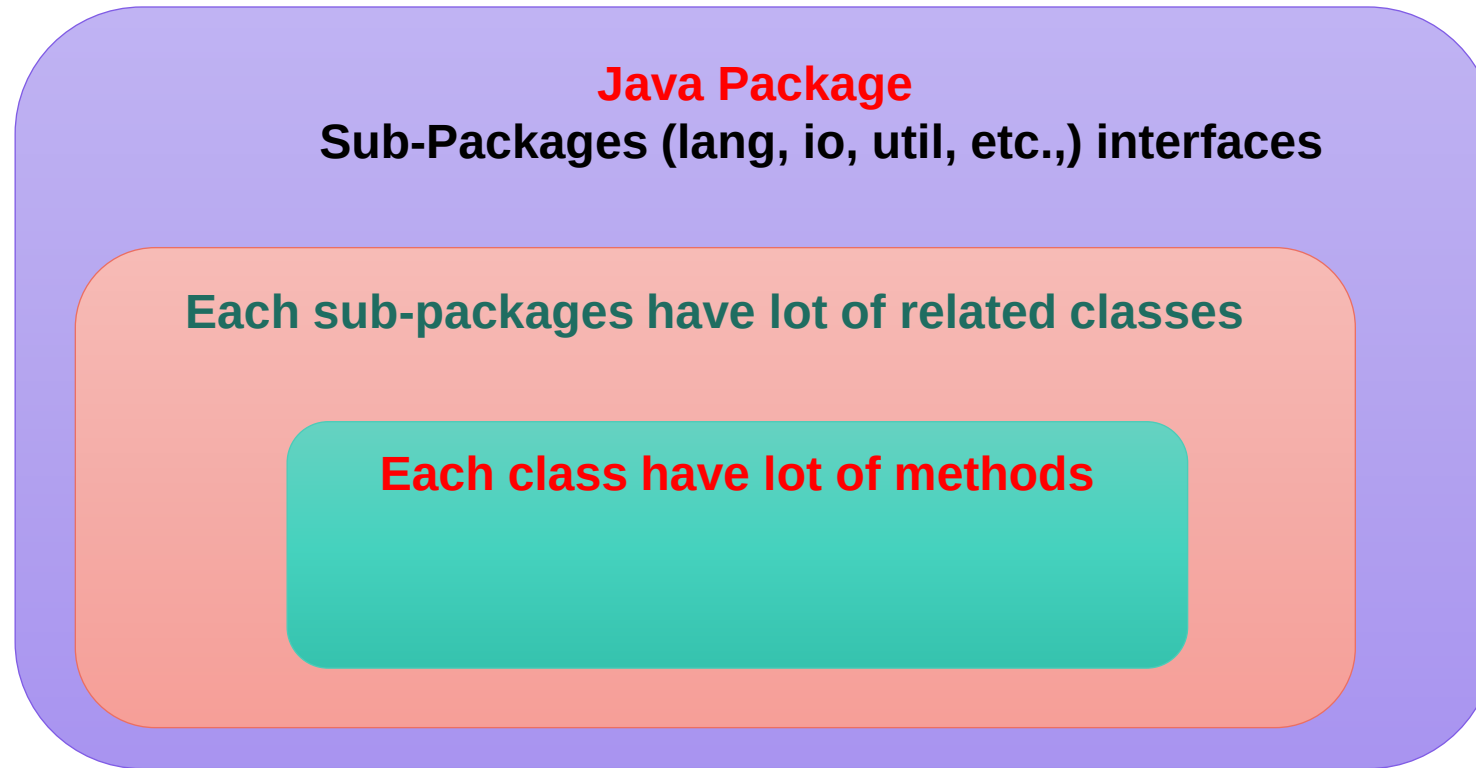
First Java Program in Detail

- **main** represents startup of the program.
- **String[] args** is used for command line argument.
- **System.out.println()** is used print statement.
 - System is a final class defined in the **java.lang** package.



Package: Introduction

- **Package:** It is a mechanism to **organize related classes, interfaces, and sub-packages** according to their functionality. It is like a folders in a file directory.



Package: Introduction

There are two types of Packages:

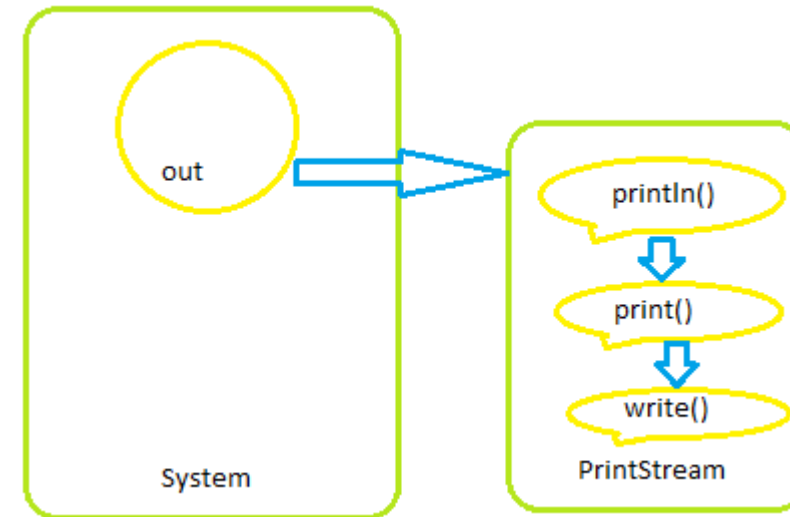
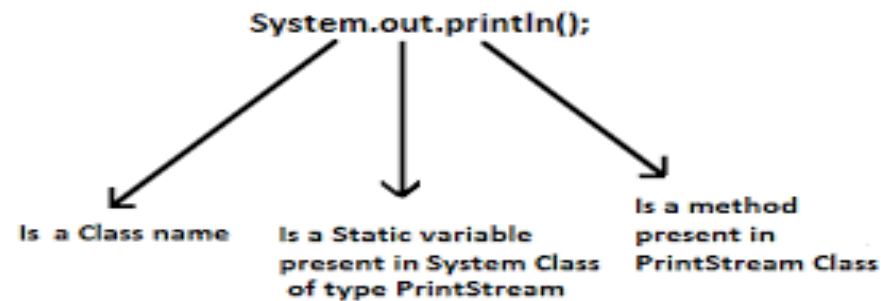
1. **Built-in Packages:** The already defined package in Java API like **java.io.***, **java.lang.*** etc. are known as built-in packages. It is simply **import** based on your application needs.
2. **User-defined Packages:** The package created by user and use based on application needs is called user-defined package.

Note:

- Programmers typically **use packages** to **organize classes** belonging to the **same category** or providing **similar functionality**.

Package: Introduction

- **Example:** `java.lang.System` (System class is a one of the class of lang package)



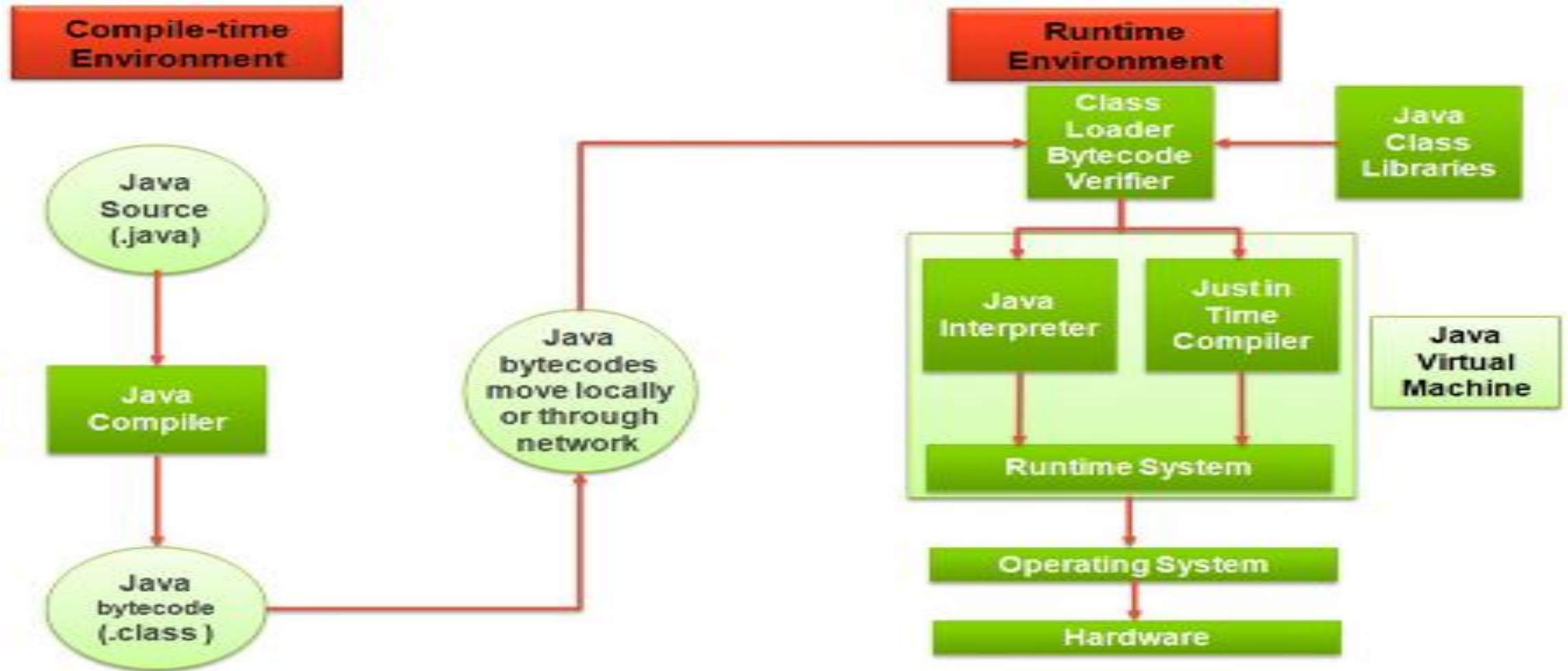
Note:

- As per Java 1.8 standard version, Java have **14 predefined packages**, **150 sub packages**, **7000 classes** and **7 lakh methods**.

Compiling and Executing

- To run java application both **compiler and interpreter** involves.
- The source code of Java will be created with file extension .java, **Example:** HelloworldApp.java
- The java Compiler compiles a java file and convert into **bytecode / classfile**.
- The byte code will be in a file with extension .class
- The .class file is **interpreted** by the JVM and morphed into **machine specific code**.

Compiling and Executing



Just-In-Time (JIT) Compiler

- The **Just-In-Time (JIT) compiler** is one of the integral parts of the Java Runtime Environment.
- It improves the **performance of Java applications** by compiling byte codes to native machine code at run time.
- The JIT compiler is **enabled by default**. When a method has been compiled, the JVM calls the compiled code of that method directly instead of interpreting it.
- Theoretically, if compilation did not require processor time and memory usage, compiling every method could allow **the speed of the Java program** to approach that of a native application.

Project Scenario



Project Scenario

Project Scenario :Movie Ticket Booking

Problem statement:

- E-ticket or electronic tickets are movie tickets that are booked online and are a digital alternative to printed tickets. The “Online movie ticket booking system” is an automated system that helps to book the ticket online. It reduces the hassles of the manual ticket booking process which involves real-time human interaction and the time consumed.
- This project will give a real-life understanding of how online movie ticket booking and activities are performed. This project will detail the automation process for movie ticket booking while providing enhanced techniques for maintaining the required information up to date.

Project Scenario

Project Requirements

- System should be able to list the cities where selected theatres are located
- Each theatre can have multiple halls and each hall can run one movie show at a time
- Each Movie will have multiple shows
- A customer should be able to search movies by their title, language, genre, theatre and city name
- Once the customer selects a movie, the service should display the theatre running that movie and its shows

Project Scenario

Project Requirements

- The customer should be able to select a show at a particular theatre and book their tickets
- The system should show the seating arrangement of the theater hall and the customer should be able to select multiple seats according to his/her preference
- The customer should be able to distinguish between available seats and booked ones
- Customers of our system should be able to pay with credit/debit cards, cash or UPI
- The system should ensure that no two customers can reserve the same seat

Project Scenario

Major Users in Movie Ticket Booking

- **Administrator** is the one who is responsible for adding new movies, showtime, cancel any movie or show, blocking/unblocking customers, etc.
- **Customer** is the end user who needs a ticket to watch the movie.
- **Officer** is one who can book or cancel ticket(s) on behalf of the customer directly.
- **Guest** is someone who can search for movie(s) but to book ticket(s) he/she should become a registered user.

Project Scenario

Administrator Modules and Functionalities

Login Module

- **Login** – Administrator can enter the username and password to login
- **Change Password** – Administrator can change the password whenever they want

Movie Management Module

- **AddMovie** – Adding the Movie detail if any new Movie released in general or any theatre decided to release the movie at any time
- **UpdateMovie** – Changing the Movie detail in the specific theatre
- **DeleteMovie** – remove the Movie detail from the list

Project Scenario

Administrator Modules and Functionalities

Profile Management Module

- **Registration** – Administrator should be registered
- **Update** – Updating their details

Report Management Module

- **Movie details** – get the list of Movies that are currently running in the theatres
- **Customer details** – get the list of customers who have registered

Project Scenario

Officer Modules and Functionalities

Booking Module

- **Booking Tickets** – Booking the tickets for the customer who has arrived directly at the theatre
- **View Movie List** – Before booking the ticket the customer should be able to view for the list of movies, he/she wants to see
- **Search Movie** – Before booking the ticket the customer should be able to search for the list of movies, he/she wants to see
- **Cancelling Tickets** – Any customer who has decided not to see the Movie can approach the officer to cancel the booked ticket / the cancellation charges and rules are to be displayed on the screen as well on the ticket

Project Scenario

Customer Modules and Functionalities

Login Module

- **Login** - Customer can enter the username and password to login
- **Change Password** - Customer can change the password whenever they want

Booking Module

- **Booking Tickets** – Customers can book a ticket for the movie they want to see
- **View Movie List** – Customers can view the Movie list at anytime
- **Search Movie** - Before ticket booking a customer can search for a specific movie from the Movie list
- **Cancelling Tickets** - Any customer who has decided not to see a Movie should be able to cancel the booked ticket

Project Scenario

Customer Modules and Functionalities

Profile Module

- **Registration** – Customer should be a registered user for booking a movie ticket
- **Update** – Customers can update their details whenever they want
- **View** – Customers can view their details whenever they want

Payment Module

- **Credit/Debit card payment** – Once a customer books ticket, he/she can make the payment by credit/debit card
- **Cash payment** – Once a customer books ticket, he/she can make the payment by cash to the officer
- **UPI payment** - Once a customer books ticket, he/she can make the payment through UPI

Project Scenario

Guest Functionalities

- **Search for a movie** – A guest user can only search and view the movie list

Java Tokens



Introduction

- The tokens are **the small building blocks** of a Java program that are meaningful to the Java compiler.
- The Java **compiler breaks the line** of code into text (words) is called **Java tokens**.
- The Java compiler identified these **words as tokens**.
- These tokens are **separated** by the **delimiters** and delimiters are not part of the Java tokens.
- It is useful for **compilers to detect errors**.

Example: In program, we will be using many statements and expressions to perform the operation. These statements and expressions are made up of tokens.

Java Tokens

Introduction

- Java supports **5 types of tokens** as follows:
 1. Keywords
 2. Identifiers
 3. Literals
 4. Operators
 5. Special symbols

Keywords

- Keywords **are predefined or reserved words** that have special meaning to the Java compiler.
- Each keyword is assigned a **special task** or function and cannot be changed by the user.
- We **cannot** use keywords as **variables or identifiers** as they are a part of Java syntax itself.
- A keyword should always be written in **lowercase** as Java is a case-sensitive language.

Java Tokens

Keywords

- They are **some available keywords** in Java

Keywords				
abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Identifiers

- An identifier is the **name given by the user** for the **various programming elements like variables, classes, methods, interface, etc.**
- **Rules for defining Identifiers**
 - Allowed characters for identifiers are all **alphanumeric characters**([A-Z],[a-z],[0-9]), '\$'(dollar sign) and '_' (underscore).
 - **Example:** “blue@” is not a valid as it contain '@' special character.
 - Identifiers should not start with digits([0-9]).
 - **Example:** “123blue” is a not a valid
 - Java identifiers are **case-sensitive**.
 - **Example:** test and Test both are different

Java Tokens

Identifiers

- There is no limit on the length of the identifier, but it is advisable to use an optimum length of **4 – 15 letters** only.
- Reserved Words can't be used as an identifier.
 - **Example:** `int while = 20;` is an **invalid statement** as **while** is a reserved word.

- **Some Valid Identifiers**

test, Test, _name, name100

ABC, File_100, _abc_

Some Invalid Identifiers

int, 100name, test@123

Test.txt,

Naming Convention of Identifiers

- All the **identifiers** such as **classes, interfaces, packages, methods, and fields** of Java programming language are given according to the Java **naming convention**.
- By using this naming convention, we can achieve **readability** and can also easily **understand the code**.
- In programming, we often **remove the spaces** between words because programs of different sorts reserve the space (' ') character for special purposes.
- Because the **space character is reserved**, we cannot use it to represent a concept that we express in our human language with multiple word

Naming Convention: Identifiers

Example

- That is in programming we cannot refer, *user login count=5*.
- We can represent as `userLoginCount=5`.
- Java follows the **CamelCase** for identifiers naming conventions.

CamelCase

- **Combines the compound words** and removes the space between the words.
- **Two types:** UpperCamelCase and lowerCamelCase.
- **UpperCamelCase:** The first letter of each word is capitalized.

Example: `Product`, `ProductDescription`, `CountValue`.

- **lowerCamelCase :** The first letter of the compound words is lowercase.

Example: `product`, `iPad`, `countValue`, `productDescription`

Naming Convention : Variables

- A variable's name can be **any legal identifier** i.e., begins with a letter, the dollar sign "\$", or the underscore character "_".
- The variable name should be **short** and **meaningful**.
- The choice of a variable name should be **mnemonic**- that is, designed to indicate to the casual observer the intent of its use.
- **One-character variable** names should be **avoided** except for temporary variables.
- The **temporary variables** can be **i,j,k,m, and n** for **integer** and **c,d,e** for **characters**.
- The variable name should be in **lowerCamelCase**.

Example:

```
String firstName;  
int orderNumber ;  
int count;  
float price;
```

Naming Convention : Variables

- **Private class variables** should have an **underscore prefix**.
- Apart from its name and its type, the scope of a variable is its most important feature.
- Indicating class scope by using underscore makes it easy to **distinguish class variables** from local scratch variables.
- This is important because class variables are considered to have higher significance than method variables, and should be treated with special care by the programmer

Example:

```
class Login{  
private String _userName;  
...  
}
```


Naming Convention : Constants and Methods

Constants:

- The names of variables declared class constants and should be in **uppercase**.
- If we specify the constant with **two words**, then separated by **underscores** ("_").

Example:

```
static final int MAX_HEIGHT = 70;
```

```
static final int MAX_WIDTH = 100;
```

```
static final int LENGTH = 20;
```

Naming Convention : Constants and Methods

Methods

- Method should be in the verb.
- Method name should be **lowerCamelCase**.

Example:

```
void calculateTax()
```

```
String getSurname()
```

```
void draw()
```

Naming Convention : Package and Import statements

- The first non-comment line of the java source file is a **package followed by import statements**.
- The prefix of a unique package name is always written in **all-lowercase ASCII letters**.

Example:

```
package java.util.*;  
import java.util.Scanner;
```

Naming Convention: Class and interface

- **Class names** should be **nouns** and must be **simple** and **descriptive**.
- Use whole words-**avoid acronyms** and **abbreviations**.
- Class name should be in **UpperCamelCase**.

Example:

```
class Customer
```

```
class CustomerAccount
```

```
class Login
```

Naming Convention: Class and interface

- **Interface** tends to have a name that describes an operation that a class can do.
- Name of the interface should be **UpperCamelCase**.

Example:

interface **E**numerable

interface **C**ompEnumerable

interface **L**ogin

Data Types

- Data types defines the type data a variable can hold. It specify the different sizes and values that can be stored in the variable.
- Java is a **strongly typed language**
 - All variables must be declared before its use.
- **Two Types of Data types**
 - 1. Primitive data type**
 - byte, short, int, long, float, double, char, boolean
 - 2. Non Primitive data type**
 - Classes, interfaces, arrays, strings

Data Types

- **byte: (1 byte):**
 - The byte data type is an 8-bit signed two's complement integer.
 - It has a minimum value of -128 and a maximum value of 127 (inclusive).
 - Useful for saving memory in large arrays.
 - Default value : 0**
- **short: (2byte)**
 - The short data type is a 16-bit signed two's complement integer.
 - It has a minimum value of -32,768 and a maximum value of 32,767 (inclusive).As byte ,can use a short to save memory in large arrays
 - Default value : 0**

Data Types

- **int: (4 byte)**

- By default, the int data type is a 32-bit signed two's complement integer, which has a minimum value of -2^{31} and a maximum value of $2^{31} - 1$.
- **Default value** : 0

- **long: (8 byte)**

- The long data type is a 64-bit two's complement integer.
- The signed long has a minimum value of -2^{63} and a maximum value of $2^{63} - 1$
- **Default value** : 0L

- **float: (4 byte)**

- The float data type is a single-precision 32-bit IEEE 754 floating point.
- **Default value** : 0.0f

-

Data Types

- **double: (8 byte)**

- The double data type is a double-precision 64-bit IEEE 754 floating point
- **Default value** : 0.0d

- **boolean: (1 bit)**

- The boolean data type has only two possible values: true and false
- **Default value** : false

- **char: (2 byte)**

- single 16-bit **Unicode** character. It has a minimum value of '\u0000' (or 0) and a maximum value of '\uffff'
- **Default value** : '\u0000'

Variables

- A **symbolic name associated with a value** and whose associated value may be changed.
- A variable is defined by the combination of an **identifier, a type and an optional initializer**.
- All variables have a **scope, which defines their visibility, and a lifetime**.
- There are **three types** of variables:
 1. Local Variables
 2. Instance Variables (Non-Static Fields)
 3. Class Variables (Static Fields)

Variables

Local Variables

- Variable that's declared within the body of a method.
- Will be used only within that method.
- Other methods in the class aren't even aware that the variable exists.
- A method will often store its state in local variables.
- A local variable **cannot** be defined with "**static**" keyword.

Variables

Instance Variables (Non-Static Variables)

- A variable declared **inside the class but outside the body of the method**, is called instance variable.
- It is **not declared as static**.
- Instance variables are created when the objects are instantiated and therefore they are **associated with the objects**.

Variables

Class Variables (Static Variables) :

- A variable which is **declared as static is called static variable**. It is also called as a **class variable**
- It **cannot be local**.
- You can create a single copy of static variable and share among all the instances of the class.
- Memory allocation for static variable **happens only once** when the class is loaded in the memory.

Variables

```
/**  
* The VariableApp class implements an application that  
* illustrate different Java variable  
*/  
class VariableApp {  
    int mark = 95 ;//instance variable  
    static char grade = 'S';    // static variable  
    public static void main(String[] args) {  
        float average=95.0 // local variable  
    }  
}
```

Literals

- A **fixed value** assigned to a variable
- Different **types of literals** are as follows:

Types	Examples
Integer Literals 1. Decimal 2. Hexa Decimal 3. Binary	<pre>int decValue=56; int hexaValue=0x10; int binVal=0b11010 ;</pre>
Floating-Point Literals	<pre>double d1 = 123.4; double d2 = 1.234e2; float f1 = 123.4f;</pre>
Character Literals	<pre>char chrlit='0108';</pre>
String Literals	<pre>String check="Test" ;</pre>
Boolean Literals	<pre>boolean result=true;</pre>

Literals

- **Using Underscore Characters in Numeric Literals**

- In **Java SE 7 and later**, any number of **underscore characters** (`_`) can **appear anywhere** between digits in a numerical literal.

- Enables **to separate groups of digits in numeric literals**, which can improve the readability of your code.

Example:

```
long creditCardNumber = 1234_5678_9012_3456L;
```

```
long socialSecurityNumber = 999_99_9999L;
```

```
long bytes = 0b11010010_01101001_10010100_10010010;
```


Literals

- You can **place underscores** only between digits; you **cannot place underscores** in the following places:
 - At the beginning or end of a number
 - Adjacent to a decimal point in a floating point literal
 - Prior to an F or L suffix
 - In positions where a string of digits is expected

Unicode

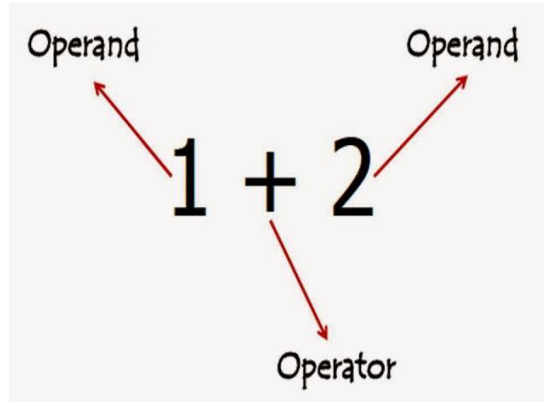
- It is Computing industry standard designed to **encode characters of the world's written languages**.
- **Unicode System?**
 - Before Unicode, there were many language standards:
 - ASCII (American Standard Code for Information Interchange) for the United States.
 - ISO 8859-1 for Western European Language.
 - KOI-8 for Russian.
 - GB18030 and BIG-5 for Chinese, and so on.

Unicode

- **Problems:**
 - A particular code value corresponds to different letters in the **various language standards**.
 - The encodings for languages with large character sets have variable length.
 - Some common characters are encoded as single bytes, other require two or more byte.
- **Solution:**
 - To solve these problems, a new language standard was developed i.e. Unicode System.
 - In **Unicode, character holds 2 byte**, so java also uses 2 byte for characters.
 - The **range of a char is 0 to 65,536**

Operators

- An operator in Java is a **special symbol** that signifies the **compiler** to perform **some specific mathematical or non-mathematical operations on one or more operands**.
- Value that the operator operates on is called operand.



- Java supports **8 types of operators**.

Java Tokens

Expression

- An expression in Java is **any valid combination of tokens** like variables, constants and operators.
- An expression may consist of **one or more operands, and zero or more operators** to produce a value.

Examples:

- $a+b*c$
- $(a*b)/(c+d)$
- $10-4*5$
- Etc.,

Operators & Expressions

- Below table we have listed down all the operators, along with their expressions:

Type	Operators	Expressions
Unary Operator	++,--,+(unary),-(unary), ~,!	a++,--a, -a, ~a, !a
Arithmetic Operator	+, -, %, *, /	a=b+c, a=b%d
Shift Operator	<<, >>, >>>	a=a>>2
Relational Operator	>, >=, <, <=, ==, !=, instance of	a=10>5, 5!=10, 8==8
Bitwise Operator	&, ^,	a=2&3
Logical Operator	&&,	(2>5)&&(10>8)
Ternary Operator	? :	a=(b>c)?1:0
Assignment Operator	=, +=, -=, *=, /=, %=, .&=, ^=, = <<=, >>=, >>>	a=b a+=b a^=b

Precedence and Associativity

Precedence

- Operator precedence determines the **order** in which the operators in an expression are evaluated.

Example: `int num = 6 - 3 * 8;`

- To evaluate the above expression Java, consider the precedence of the operator .Here, Multiplication (*) has highest precedence than subtraction (-).So multiplication will be performed before the subtraction.

Associativity

- If an **expression has two operators with similar precedence**, the expression is evaluated according to its **associativity** .
- That is either **left to right**, or **right to left**.

Example: `X=Y=Z;` Operator has same precedence. Here, the value of Z is assigned to variable Y. Then the value of Y is assigned of variable X because the associativity of = operator is from right to left

Precedence and Associativity

Operators	Precedence	Associativity
postfix increment and decrement	++ --	left to right
prefix increment and decrement, and unary	++ -- + - ~ !	right to left
multiplicative	* / %	left to right
additive	+ -	left to right
shift	<< >> >>>	left to right
relational	< > <= >= instanceof	left to right
equality	== !=	left to right

Precedence and Associativity

Operators	Precedence	Associativity
bitwise AND	&	left to right
bitwise exclusive OR	^	left to right
bitwise inclusive OR		left to right
logical AND	&&	left to right
logical OR		left to right
ternary	? :	right to left
assignment	= += -= *= /= %= &= ^= = <<= >>= >>>=	right to left

Java Tokens

Expression Evaluation

$10 - 3 \% 8 + 6 / 4$
 $10 - 3 + 6 / 4$
 $10 - 3 + 1$
 $7 + 1$
8

Example 1

$6 - (5 - 3) + 10$
 $= 6 - 2 + 10$
 $= 4 + 10$
 $= 14$

Example 2

$3 + 4 * 4 > 5 * (4 + 3) - 1$
 (1) inside parentheses first
 $3 + 4 * 4 > 5 * 7 - 1$
 (2) multiplication
 $3 + 16 > 5 * 7 - 1$
 (3) multiplication
 $3 + 16 > 35 - 1$
 (4) addition
 $19 > 35 - 1$
 (5) subtraction
 $19 > 34$
 (6) greater than
false

Example 3

Type Conversions

- **Widening or Automatic Type Conversion** – lower data types are automatically convert into higher data types. It is also known as **implicit conversion**.
- Automatic type conversion will take place if the following **two conditions** are met:
 - The two types are compatible.
 - The destination type is larger than the source type

Type Conversions

- Widening primitive conversions:

byte → short, int, long, float, double

short → int, long, float, double

char → int, long, float, double

int → long, float, double

long → float, double

float → double

- **Generally safe** because they tend to go from a small data type to a larger one
- No automatic conversion is supported from **numeric type to char or boolean**

Type Conversions

```
/**  
 * The ConversionAutomatic class implements an application that  
 * Illustrate the automatic type conversion  
 */  
class ConversionAutomatic {  
    public static void main(String[] args) {  
        int i = 100;  
        long l = i; // automatic type conversion  
        float f = l; // automatic type conversion  
        System.out.println("Int value "+i);  
        System.out.println("Long value "+l);  
        System.out.println("Float value "+f);  
    }  
}
```

Type Conversions

- **Narrowing or Explicit Conversion** - If we want to assign a value of **larger data type to a smaller data type** we perform explicit type casting or narrowing.
- Useful for incompatible data types **where automatic conversion cannot be done**.
- Here, target-type specifies the desired type to convert the specified value to.
- **Narrowing primitive conversions:**
 - byte → char**
 - short → byte, char**
 - char → byte, short**
 - int → byte, short, char**
 - long → byte, short, char, int**
 - float → byte, short, char, int, long**
 - double → byte, short, char, int, long, float**

Type Conversions

```
/**  
 * The ConversionExplicit class implements an application that  
 * Illustrate the explicit type conversion  
 */  
class ConversionExplicit {  
    public static void main(String[] args) {  
        double d = 100.04;  
        long l = (long)d; //convert double into long  
        int i = (int)l; // long convert into int  
        System.out.println("Double value "+d);  
        System.out.println("Long value "+l);  
        System.out.println("Int value "+i);  
    }  
}
```

Type Conversions

- **Type promotion in Expressions** - While evaluating expressions, the intermediate value may **exceed the range of operands** and hence the expression value will be promoted.
- **Some conditions for type promotion are:**
 1. Java automatically promotes each byte, short, or char operand to int when evaluating an expression.
 2. If one operand is a long, float or double the whole expression is promoted to long, float or double respectively.

Type Conversions

```
/**  
 * The TypePromotion class implements an application that  
 * Illustrate the type promotion */  
  
class TypePromotion{  
    public static void main(String[] args){  
        byte b = 50;  
        b = (byte)(b * 2); //promote into int  
        System.out.println(b);  
    }  
}
```

Type Conversions

```
/**  
 * The TypePromotion1 class implements an application that  
 * Illustrate the type promotion */  
  
class TypePromotion1{  
    public static void main(String[] args){  
        byte b = 42;  
        char c = 'a';  
        short s = 1024;  
        int i = 50000;  
        float f = 5.67f;  
        double d = .1234;  
        double result = (f * b) + (i / c) - (d * s); //promote into double  
        System.out.println("result = " + result);  
    }  
}
```

Special Symbols

- Special symbols in Java are a **few characters which have special meaning** known to Java compiler and cannot be used for any other purpose.
- In the table we have listed down the special symbols supported in Java along with their description.

Special Symbols

Symbols	Description
<i>brackets []</i>	These are used as an array element reference and also indicates single and multidimensional subscripts
<i>Parentheses()</i>	These indicate a function call along with function parameters
<i>Braces{} </i>	The opening and ending curly braces indicate the beginning and end of a block of code having more than one statement
<i>Comma (,)</i>	This helps in separating more than one statement in an expression
<i>Semi-Colon (;)</i>	This is used to invoke an initialization list

Read User Input



Introduction

- There are **three different ways** to read input from user:
 1. Buffered Reader Class (**Will discuss later**)
 2. Console Class (**Will discuss later**)
 3. Scanner Class
- **Scanner Class**: It is a class in **java.util package** used for obtaining the user input of the primitive types like int and double. It is the **easiest way to read input** in a Java program.
- **Scanner object** is constructed from **Scanner Class** and **System.in (input stream)** object is passed as a parameter while creating a scanner object.
- **Constructing a Scanner object to read console input:**
Scanner scannerObject = new Scanner(System.in);

Read User Input

Methods

- After creating the scanner object, we can use below **Scanner class methods** for reading the **respective primitive data types from the console**.

Method	Description
<code>boolean nextBoolean()</code>	This method reads the boolean value from the user.
<code>byte nextByte()</code>	It reads the byte value from the user.
<code>double nextDouble()</code>	It accepts the input in double datatype from the user.
<code>float nextFloat()</code>	It takes the float value from the user.
<code>int nextInt()</code>	It reads the integer value from the user.

Read User Input

Methods

Method	Description
String nextLine()	This method reads the String value from the user.
long nextLong()	This method reads the long type of value from the user.
short nextShort()	It reads the short type of value from the user.
String next()	It reads the character input from the user.

Methods

Note:

- Scanner class **no specific method** to read character type.
- To read a single character, we use **next().charAt(0)**. **next() function** returns the next token/word in the **input as a string** and **charAt(0) function** returns the **first character** in that string.

Read User Input

Example: #1

```
/**  
 * The ReadSomeInput class implements an application that  
 * Illustrate reading a console input */  
  
import java.util.Scanner; //import Scanner class from util package  
  
public class ReadSomeInput {  
    public static void main(String[] args) {  
        Scanner console = new Scanner(System.in);  
        System.out.print("Enter your Name : ");  
        String name = console.next();  
        System.out.println("Hi, " + name + " . Welcome to the Training Program  
        ...");  
        console.close();  
    }  
}
```

Output:

Enter your Name : Arun

Hi, Arun . Welcome to the Training Program ...

Read User Input

Example: #2

```
/**
 * The ReadMovie class implements an application that
 * Illustrate reading a different types of input
 */
import java.util.Scanner;

public class ReadMovie {

    public static void main(String[] args) throws ParseException {

        Scanner console = new Scanner(System.in);

        System.out.println("Enter Movie ID : ");

        int movieid = console.nextInt();

        System.out.println("Enter Movie Name : ");

        String moviename = console.next();

        System.out.println("Enter Movie Description : ");

        String moviedescription = console.next();

        System.out.println("Enter Movie Language : ");

        String movielanguage = console.next();

        System.out.println("Enter Movie Genre : ");
```

Read User Input

Example: #2

```
String moviegenre = console.next();
System.out.println("Enter Movie release date (dd/mm/yyyy) : ");
String date = console.next();
SimpleDateFormat moviereleasedate = new SimpleDateFormat("dd/MM/yyyy");
Date moviedate = moviereleasedate.parse(date);
System.out.println("Enter Movie Seat Cost : ");
float movieseatcost = console.nextFloat();
System.out.println("ENTERED MOVIE DETAILS ARE");
System.out.println("Movie ID : "+movieid);
System.out.println("Movie Name : "+moviename);
System.out.println("Movie Description : "+moviedescription);
System.out.println("Movie Language : "+movielanguage);
System.out.println("Movie Genre : "+moviegenre);
System.out.println("Movie Date : "+moviedate);
System.out.println("Movie Seat Cost : "+movieseatcost);
}}
```

Output:

```
Enter Movie ID : 1231
Enter Movie Name : AAA
Enter Movie Description : Dramaof1956
Enter Movie Language : English
Enter Movie Genre : ACTION
Enter Movie release date (dd/mm/yyyy) : 23/03/2022
Enter Movie Seat Cost : 120.0
ENTERED MOVIE DETAILS ARE
Movie ID : 1231
Movie Name : AAA
Movie Description : Dramaof1956
Movie Language : English
Movie Genre : ACTION
Movie Date : Wed Mar 23 00:00:00 IST 2022
Movie Seat Cost : 120.0
```

Quiz



1) Which of the following are primitive data types?

a) int

b) float

c) double

d) boolean

e) All the above

e)All the Above

Quiz



2) A local variable stores temporary state; it is declared inside
a

a) Class

b) Method

c) Block

d) Object

b) & c)

Quiz



3) A _____ is a value that should not be altered by the program during normal execution.

a) Variable

b) int

c) Identifiers

d) Constant

d) Constant

Quiz



4) Which of these can not be used for a variable name in Java?

a) Identifier

b) Keyword

c) Both a and b

d) None of the above

b) keyword

Quiz



5) Which conversion also called Automatic Type Conversion?

a) Widening

b) Narrowing

c) Both A and B

d) All the above

a) Widening

Quiz



6) Long Literals in java must be appended by which of these?

a) L

b) l

c) D

d) 0x

a) & b)

Quiz



7) Which variables are created when an object is created and destroyed when the object is destroyed?

a) Local variables

b) Instance variables

c) Class Variables

d) Static variables

b) Instance variables

Quiz



8) Which of the following are not Java keyword ?

a) double

b) switch

c) instanceof

d) then

d) then

Quiz



9) Which of these is not a bitwise operator?

a) &

b) |

c) ^

d) <=

d) <=

Quiz



10) Which operator is used to invert all the digits in binary representation of a number?

a) ~

b) <<

c) ^

d) >>>

a) ~

Quiz



11) It is time to buy a new phone when at least one of the following situations occurs:

- the phone breaks
- the phone is at least 3 years old

```
int phoneAge;    // in years
boolean isBroken;
.....          //code initializes variables
boolean needPhone = _____
```

```
(isBroken == true) || (phoneAge >= 3);
```

Quiz



12) Evaluate the following expression:

$(17 < 4 * 3 + 5) \parallel (8 * 2 == 4 * 4) \&\& !(3 + 3 == 6)$

a) true

b) false

b)false

Quiz



13) Assume num1=10. Which of the following is not a valid statement?

a) num1>>2

b) num1<<<2

c) num1%=2;

d) num1<<2;

b) num1<<<2

Quiz



14) In Java, after executing the following code what are the values of x, y and z?

```
int x,y=10; z=12;
```

```
x=y++ + z++;
```

a) x=22, y=10, z=12

b) x=24, y=10, z=12

c) x=24, y=11, z=13

d) x=22, y=11, z=13

d) x=22, y=11, z=13

Quiz



15) Which Scanner class method is used to read integer value from the user?

a) next()

b) nextInt()

c) nextInteger()

d) readInt()

b) nextInt()

Quiz



16) Which is the correct syntax to declare Scanner class object?

a) `Scanner obj=Scanner();`

b) `Scanner obj=new Scanner();`

c) `Scanner obj= Scanner
(System.in);`

d) `Scanner obj=new Scanner
(System.in);`

d) `Scanner obj=new Scanner
(System.in);`

Quiz



17) Consider the following object declaration statement

`Scanner obj= new Scanner(System.in).`

What is `System.in` in this declaration?

a)Class which point input device

b) Reference to Input stream

c) Reference to Computer System

d)None of these

b) Reference to Input stream

Control Flow Statements



Contents

- Introduction
- Branching - Conditional
- Looping
- Nested Loop
- Branching – Unconditional
- Quiz

Control Flow Statements

Introduction

- Control flow is the **order in which individual statements or instructions** of a program are executed or evaluated.

Control flow statements

- **Branching (Conditional) /Decision Making** – To make **decisions** based on a given condition. If the condition evaluates to **True**, a **set of statements** is executed, **otherwise another set** of statements is executed. In Java, we have **five types** of decision making statements:
 - if
 - if...else
 - if-else-if
 - nested if
 - switch

Control Flow Statements

Decision-Making : simple if

- **Simple if:** If the condition is true, the body of the if statement is executed. If it is false, the body of the if statement is skipped.

- **Syntax**

if is a Java reserved word

The **condition** must be a boolean expression. It must evaluate to either **true** or **false**.

```
if ( condition ){
    statement(s);
}
```

If the **condition** is **true**, the **statement(s)** is/are executed.

If it is **false**, the **statement(s)** is/are skipped.

Control Flow Statements

Decision-Making : simple if

```
/**
 * The IfControlStructure class implements an application that
 * Illustrate the if decision Making control statement
 */
class IfControlStructure{
    public static void main(String[] args){
        boolean isMoving=true;
        int currentSpeed=10;
        if(isMoving){
            System.out.println(currentSpeed);
        }
    }
}
```

Output:

10

Control Flow Statements

Decision-Making : simple if

```
/**
 *The SimpleIf class implements an application that checks the seat availability status for movie ticket booking using
 *simple if decision Making control statement
 */
import java.util.Scanner;
public class SimpleIf {
    public static void main(String[] args){
        boolean seatAvailable = true;           //Seat Available Status
        Scanner input = new Scanner(System.in);  //Scanner class object creation
        System.out.println("Enter the Seat Number : ");
        String SeatNumber = input.next();        //get Seat Number from User
    }
}
```

Control Flow Statements

Decision-Making : simple if

```
        if(seatAvailable){                                //Check the availability of seat
            System.out.println("Your have booked the seat number : "+SeatNumber);
        }
        input.close();
    }
}
```

Output:

Enter the Seat Number : A32

Your have booked the seat number : A32

Control Flow Statements

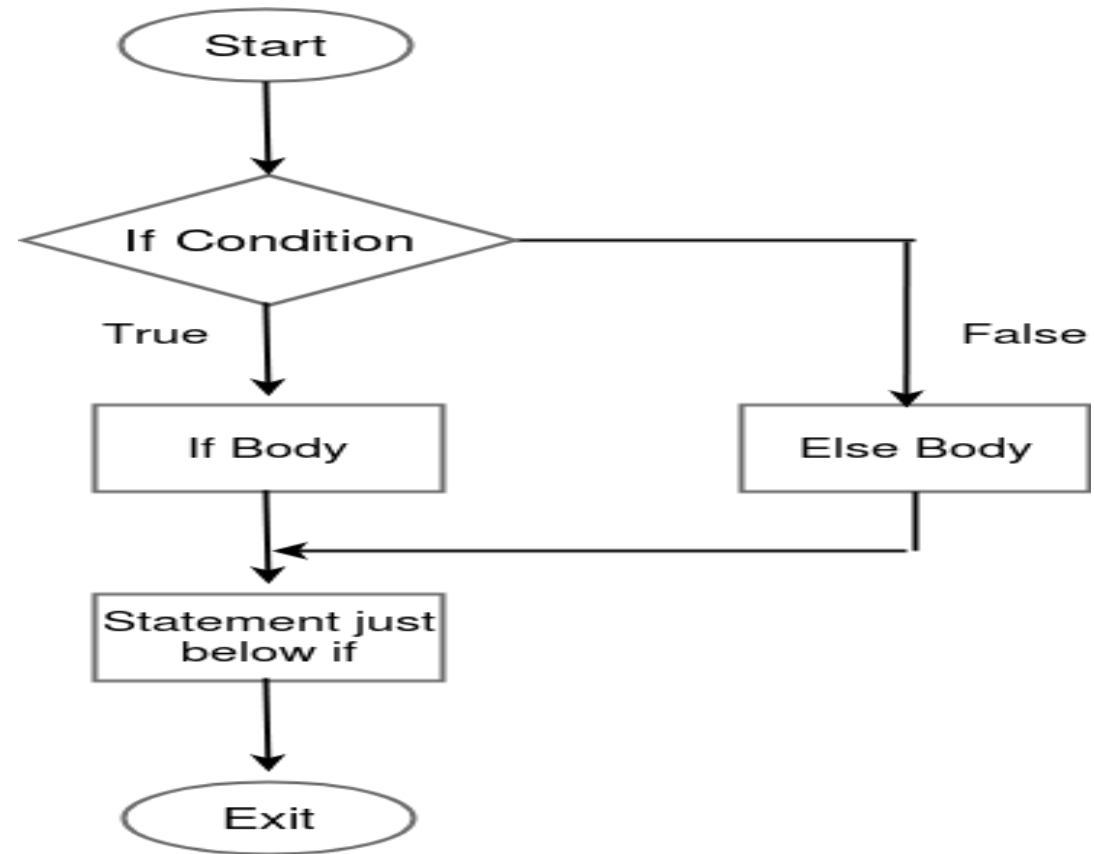
Decision-Making : simple if - else

- **if –else:** If the *condition* is true, the body of the if *statement* is executed. If it is false, the body of the false *statement* is executed.

- **Syntax**

```

if ( condition ){
    if Body;
}
else{
    else Body;
}
    
```



Control Flow Statements

Decision-Making : simple if - else

```
/** * The IfElseControlStructure class implements an application that Illustrate  
the if ..else decision Making control statement */  
class IfElseControlStructure{  
    public static void main(String[] args){  
        boolean isMoving=true;  
        int currentSpeed=10;  
        if(isMoving){  
            currentSpeed--;  
            System.out.println("The bicycle speed got reduced!");  
        }  
        else{  
            System.out.println("The bicycle has already stopped!");  
        }  
    }  
}
```

Output:

The bicycle speed got reduced!

Control Flow Statements

Decision-Making : simple if - else

```
/** * The SimpleIfElse class implements an application that checks the seat availability status for movie ticket booking using the if ..else decision-Making control statement */
```

```
public class SimpleIfElse {  
    public static void main(String[] args){  
        boolean seatAvailable = false;           //Seat Available Status  
        Scanner input = new Scanner(System.in); //Scanner class object creation  
        System.out.println("Enter the Seat Number : ");  
        String SeatNumber = input.next();         // get Seat Number from User  
        if(seatAvailable){                        //Check the availability of seat  
            System.out.println("Your have booked the seat number : "+SeatNumber);  
        }  
    }  
}
```

Control Flow Statements

Decision-Making : simple if - else

```
else {                                //if seat not available then display
    System.out.println("Seat Numer "+SeatNumber+" is already booked");
}
input.close();
}
```

Output:

Enter the Seat Number : A22

Seat Numer A22 is already booked

Control Flow Statements

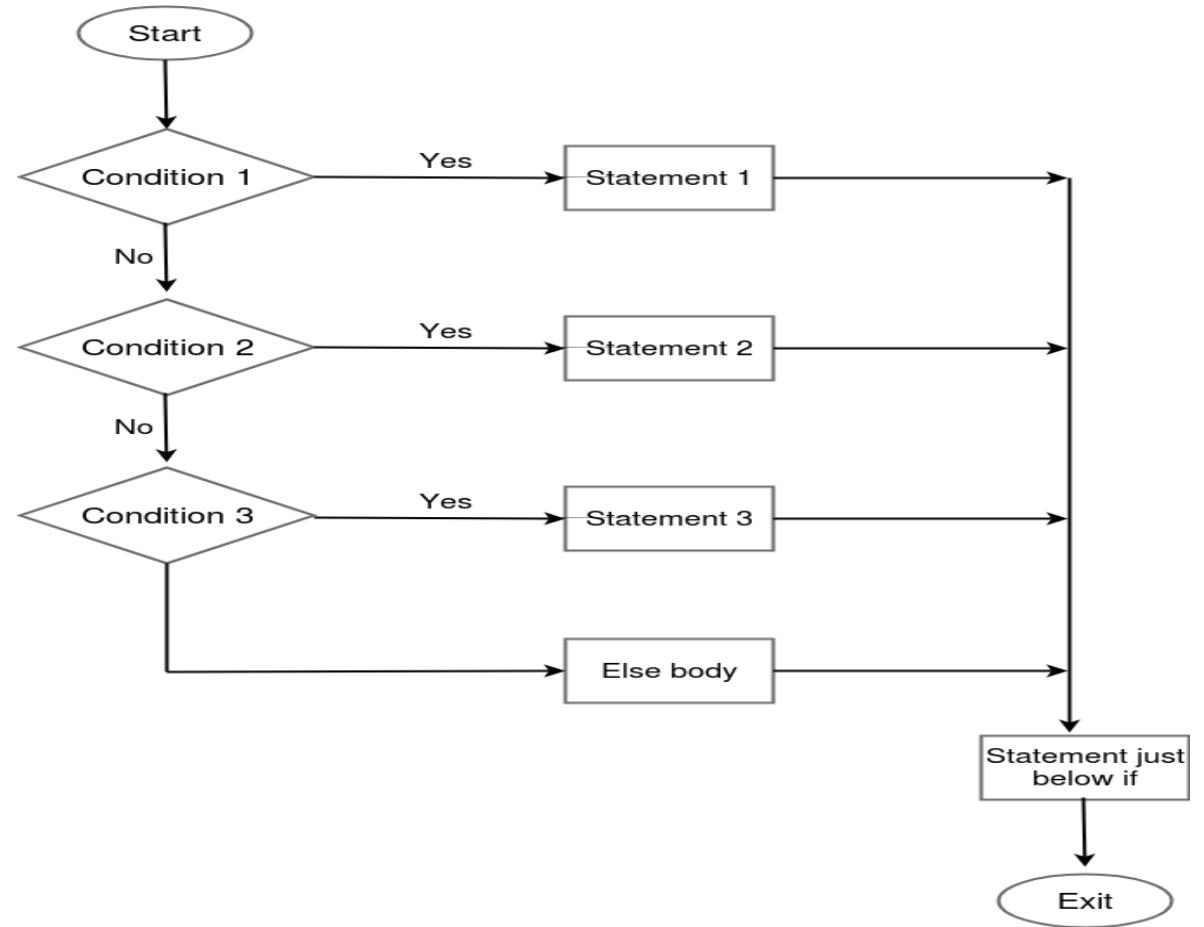
Decision-Making : simple if – else – if

- **if-else-if:** It is also called **else-if ladder**. Here execute any one block of statements among many blocks.

- **Syntax**

```

if ( condition 1){
    statement 1;
}else if ( condition 2 ){
    statement 2;
}else if (condition 3){
    statement 3;
} else {
    else Body
}
    
```



Control Flow Statements

Decision-Making : simple if – else - if

```
/**
 * The IfElseIfControlStructure class implements an application that
 * Illustrate the if ..elseif decision Making control statement */
class IfElseIfControlStructure{
    public static void main(String[] args){
        int colorValue=2;
        if(colorValue==1)
            System.out.println("Color Blue!");
        else if(colorValue==2)
            System.out.println("Color Red!");
        else
            System.out.println("Color Green!");
    }
}
```

Output:
Color Red!

Control Flow Statements

Decision-Making : simple if – else - if

```
/**
 * The IfElseIf class implements an application that display the cost of the specific seat category in movie ticket
   booking using if ..elseif decision Making control statement for fixing the cost based on Movie Type */
import java.util.Scanner;
public class IfElseIf {
    public static void main(String[] args){
        String seattype;
        System.out.println("Type of seats Available\nREGULAR\nPREMIUM\nEXECUTIVE\nVIP\ choose any one of the
option : ");
        Scanner input = new Scanner(System.in); //Scanner class object creation
        seattype = input.next(); // get Seat Number from User
        if (seattype.equals("REGULAR")) { //Display detail for REGULAR type
            System.out.println("You have selected Executive Seat and cost is Rs.80");
        }
    }
}
```

Control Flow Statements

Decision-Making : simple if – else - if

```
else if (seattype.equals("PREMIUM")) {                //Display detail for PREMIUM type
    System.out.println("You have selected Premium Seat and cost is Rs.100");
}
else if (seattype.equals("EXECUTIVE")) {              //Display detail for EXECUTIVE type
    System.out.println("You have selected Regular Seat and cost is Rs.120");
} else if (seattype.equals("VIP")) {                  //Display detail for VIP type
    System.out.println("You have selected VIP Seat and cost is Rs.150");
} else {
    System.out.println("You have not selected any type");
}
input.close();
}
```

Control Flow Statements

Decision-Making : simple if – else - if

Output:

Type of seats Available

REGULAR

PREMIUM

EXECUTIVE

VIP

choose any one of the option : PREMIUM

You have selected Premium Seat and cost Rs.100

Control Flow Statements

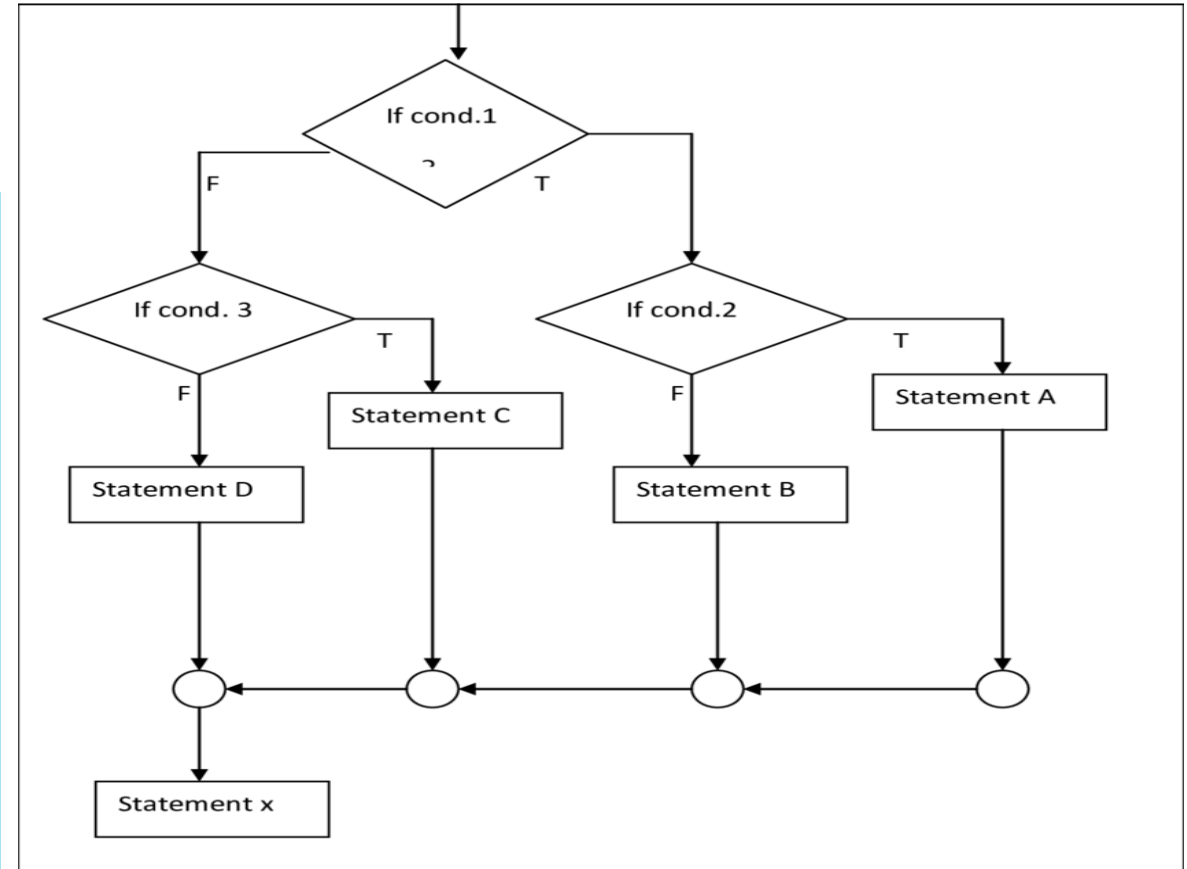
Decision-Making : Nested if

- **Nested if:** Use more than if statement inside another if statement. The outer if statement condition true means inside if statements execute.

- **Syntax**

```

if ( condition 1 ){
    if ( condition 2 ){
        Nested if Body
    }else{
        Nested else Body
    }
}else
    else Body
}
    
```



Control Flow Statements

Decision-Making : Nested if

```
/* * The NestedIfControlStructure class implements an application that
 * Illustrate the nestedif decision Making control statement */
class NestedIFControlStructure{
    public static void main(String[] args){
        int age=15;
        int weight=50;
        if(age>18) {
            if(weight>50)
                System.out.println("You are eligible to donate blood");
            else
                System.out.println("Not eligible because you are under weight");
        } else
            System.out.println("Not eligible because you are under age");
    }
}
```

Output:

Not eligible because
you are underage

Control Flow Statements

Decision-Making : Nested if

```
/* * The Booking class implements an application that validates the login and check for the seat availability
 * using nestedif decision Making control statement */
import java.util.*;
public class NestedIf {
    public static void main(String[] args){
        String username = "Sarvesh",password = "sarvesh@123",usernameentered,passwordentered;
        boolean seatAvailable = true;
        String seatNumber;
        Scanner input = new Scanner(System.in);           //Scanner class object creation
        System.out.println("Enter the Username : ");
        username = input.next();                           //getting the username
        System.out.println("Enter the Password : ");
        password = input.next();                           //getting the username
    }
}
```


Control Flow Statements

Decision-Making : Nested if

```
if (usernameentered.equals(username) && password.equals(password)) {    //validate the user login
    System.out.println("You have logged in and you can book a ticket now");
    System.out.println("Enter the Seat Number : ");
    seatNumber = input.next();                // get the seat number from the user
    if (seatAvailable) {                    // check for seat availability status
        System.out.println("Seat Number "+seatNumber+" you have chosen is available");
    } else{
        System.out.println("Your expected Seat Number "+seatNumber+" is not available");
    }
} else{
    System.out.println("You have to login for booking the ticket");
}      input.close();
}}
```

Control Flow Statements

Decision-Making : Nested if

Output:

Enter the Username : Sarvesh

Enter the Password : sarvesh@123

You have logged in and you can book a ticket now

Enter the Seat Number : A10

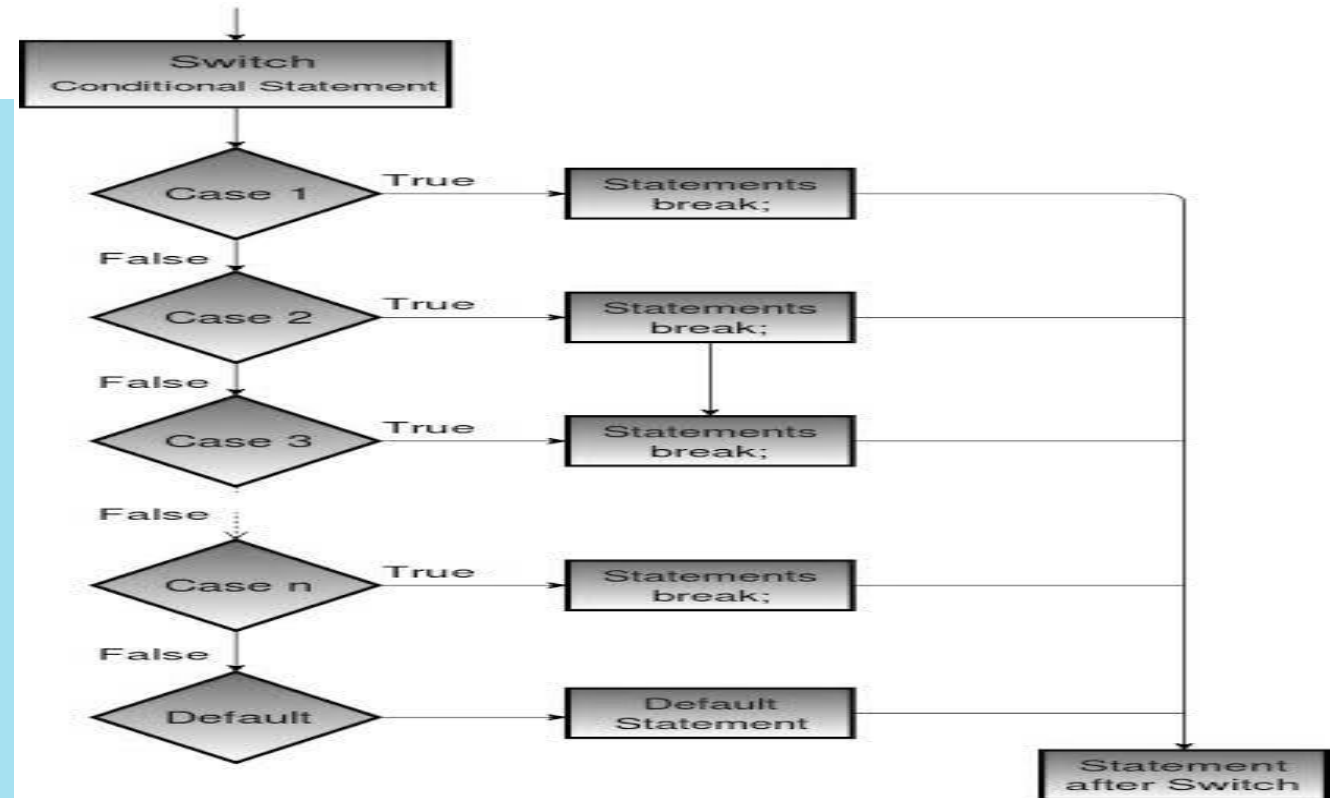
Seat Number A10 you have chosen is available

Control Flow Statements

Decision-Making : switch - case

- **switch-case:** Select one of many possible statements to execute. It gives alternate for long if..else..if ladders which improves code readable.
- **Syntax**

```
switch ( expression ) {
    case value1 :
        statement-list1;
        break;
    case value2 :
        statement-list 2;
        break;
    default:
        statement-list 3;
        break;
}
```



Control Flow Statements

Decision-Making : switch - case

Note:

- In switch, Case value must be in **expression type** only and case values must be **unique**.
- Expression must be of **byte, short, int, long, enums and string**.
- Each case **break statement is optional**. It helps **terminate from switch expression**. If a break statement is not found, it executes the next case.
- The case value can have a **default label** which is **optional**.

Control Flow Statements

Decision-Making : switch - case

```
/** The SwitchControlStructure class implements an application that
 * Illustrate switch decision Making control statement */
class SwitchControlStructure{
    public static void main(String[] args){
        int letter='A';

        switch(letter){
            case 'a':
                System.out.println("Lowercase Letter");
                break;
            case 'A':
                System.out.println("Uppercase Letter");
                break;
```

Control Flow Statements

Decision-Making : switch - case

```
        default:
            System.out.println("Invalid Letter");
            break;
    }
}
```

Output:

Uppercase Letter

Control Flow Statements

Decision-Making : switch - case

```
/** The SwitchCase class implements an application that demonstrate the movie searching by different types of languages
 * using switch decision Making control statement Searching the Movie detail by Title, Language, ReleaseDate, Genre*/
import java.util.Scanner;

public class SwitchCase {
    public static void main(String[] args){
        System.out.println("Enter the type to be search \n1. Search by Title \n2. Search by Language \n3. Search by Release
Date \n4. Search by Genre \nEnter the Choice (1/2/3/4)");

        Scanner input = new Scanner(System.in); //Scanner class object creation
        int choice = input.nextInt();           //getting the choice from user
        switch(choice){
            case 1:
                System.out.println("Your searching choice is Movies by Title");
                break;
            case 2:
```

Control Flow Statements

Decision-Making : switch - case

```
        System.out.println("Your searching choice is Movies by Language");
        break;
    case 3:
        System.out.println("Your searching choice Movies by Release Date");
        break;
    case 4:
        System.out.println("Your searching choice Movies by Genre");
        break;
    default:
        System.out.print("Your choice is wrong. Please enter the correct choice ");
    }
    input.close();
}
```


Control Flow Statements

Decision-Making : switch - case

Output:

Enter the type to be search

1. Search by Title
2. Search by Language
3. Search by Release Date
4. Search by Genre

2

Your searching choice is Movies by Language

Control Flow Statements

Looping - Introduction

- **Loop/ iterative** statements are used for **executing a block of statements repeatedly** until a particular condition is satisfied.
- **In Java, we have following loop statements:**
 - while
 - do...while
 - for
 - for...each
 - Nested loops (for, while, do...while)

Control Flow Statements

Looping - Introduction

Four Looping Elements

- **Initialization:** To **set initial value** to the iteration variable at the very start of the loop.
- **Condition:** Test condition is evaluate and give **boolean** (0 or 1) value as a result.
- **Loop-body:** If the test condition is true, the **body of the loop runs once**.
- **Updation:** **Increment/decrement** statement executes just after executing the body, and then the program goes back to the test condition.

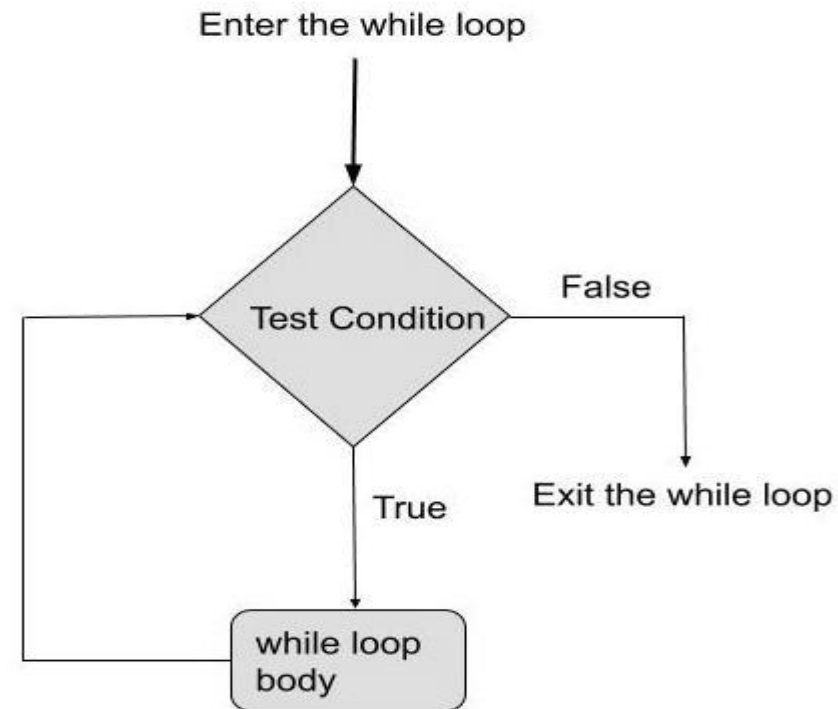
Control Flow Statements

Looping - while

- **while:** It is used to repeat a **specific block of code**. It is **preferable** while we do **not know the exact number of iterations** of loop beforehand.

- Syntax

```
while (condition){
    statement(s)
}
```



Control Flow Statements

Looping - while

```
/**
 * The WhileStructure class implements an application that
 * Illustrate While Looping control statement
 */
class WhileStructure{
    public static void main(String[] args){
        int counter = 1;
        while (counter < 11)
        {
            System.out.println("Count is: " + counter);
            counter++;
        }
    }
}
```

Control Flow Statements

Looping - while

Output:

Count is: 1

Count is: 2

Count is: 3

Count is: 4

Count is: 5

Count is: 6

Count is: 7

Count is: 8

Count is: 9

Count is: 10

Control Flow Statements

Looping - while

```
/**
 * The ShowSeat class implements an application that display the current seat availability
 * using While Looping control statement
 */
public class ShowSeat {
    public static void main (String args[]){
        int MaxSeatCount = 10, seatCount = 0;
        while(seatCount < MaxSeatCount){
            System.out.println("Current Seat Availability : "+(MaxSeatCount-seatCount));
            seatCount++;
        }
        System.out.println("Seats are Filled");
    }
}
```

Control Flow Statements

Looping - while

Output:

Current Seat Availability : 10

Current Seat Availability : 9

Current Seat Availability : 8

Current Seat Availability : 7

Current Seat Availability : 6

Current Seat Availability : 5

Current Seat Availability : 4

Current Seat Availability : 3

Current Seat Availability : 2

Current Seat Availability : 1

Seats are Filled

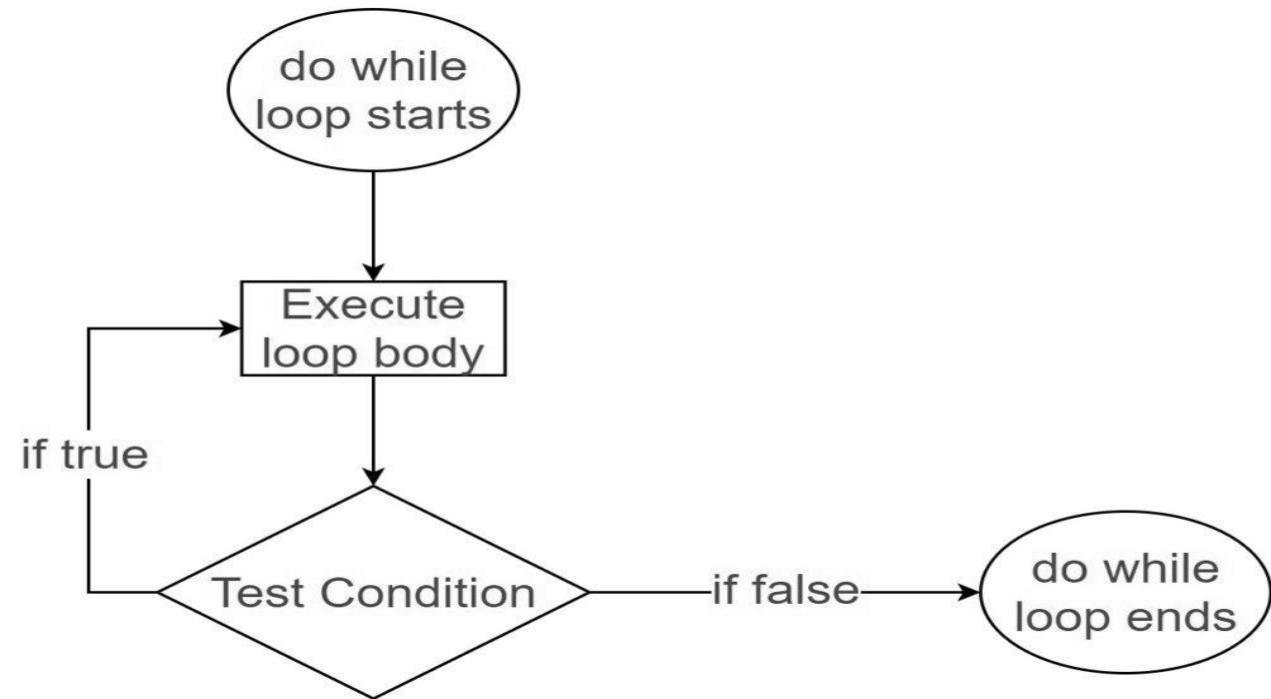
Control Flow Statements

Looping : do - while

- **do-while:** Executes the statement first and then checks for the condition. It is also called an **exit-controlled loop**.

- **Syntax**

```
do {
    statement(s)
} while (condition);
```



Control Flow Statements

Looping : do - while

```
/**
 * The DoWhileStructure class implements an application that checks the seat availability
 * status for movie ticket booking and Illustrate do..while Looping control statement
 */
class DoWhileStructure{
    public static void main(String[] args){
        int counter = 1;
        do {
            System.out.println("Count is: " + counter);
            counter++;
        } while (counter < 11)
    }
}
```

Control Flow Statements

Looping : do - while

Output:

Count is: 1

Count is: 2

Count is: 3

Count is: 4

Count is: 5

Count is: 6

Count is: 7

Count is: 8

Count is: 9

Count is: 10

Control Flow Statements

Looping : do - while

```
/**
 * The ShowSeat class implements an application that checks the seat
 * availability status for movie ticket booking using do.. while Looping control statement */
public class ShowSeat {
    public static void main (String args[]){
        int MaxSeatCount = 5, seatCount = 0;
        do{
            System.out.println("Current Seat Availability : "+(MaxSeatCount-seatCount));
            seatCount++;
        } while(seatCount < MaxSeatCount);
        System.out.println("Seats are Filled");
    }
}
```

Control Flow Statements

Looping : do - while

Output:

Current Seat Availability : 5

Current Seat Availability : 4

Current Seat Availability : 3

Current Seat Availability : 2

Current Seat Availability : 1

Seats are Filled

Control Flow Statements

Looping : for

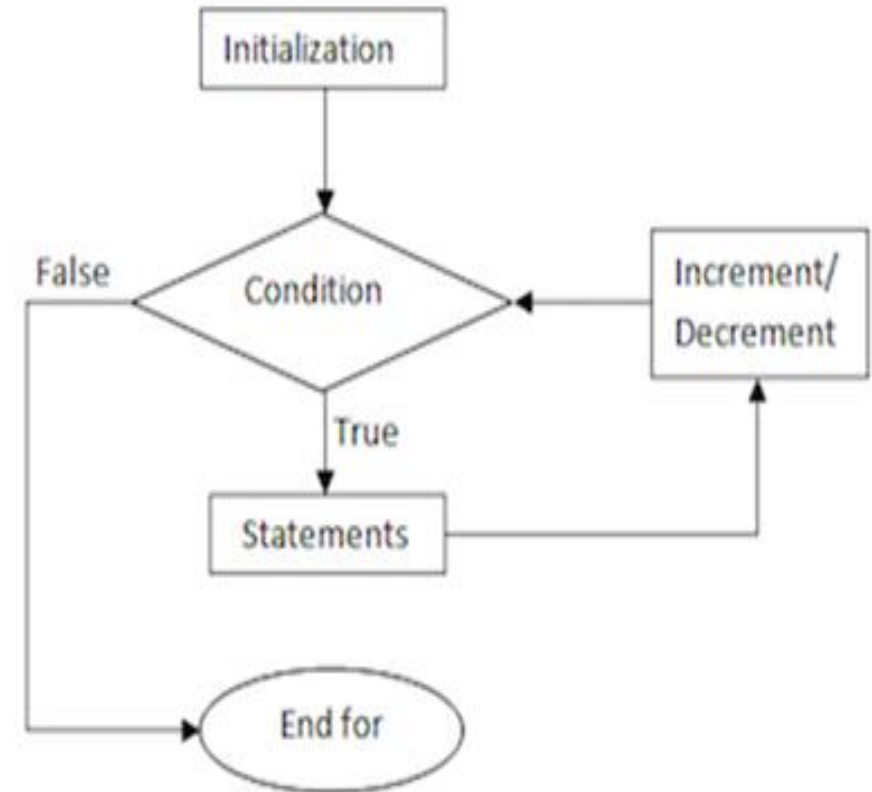
- **for:** When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop
- **Syntax**

```
for (initialization; condition ; updation) {  
    statement(s)  
}
```

Initialization : Initializes the loop; it's executed once.

Condition : Evaluates till the condition becomes false.

Updation : Increment / Decrement the value. Executed (every time) after the code block is executed.



Control Flow Statements

Looping : for

```
/**  
 * The ForStructure class implements an application that  
 * Illustrate ForLooping control statement  
 */  
class ForStructure{  
    public static void main(String[] args) {  
        for(int count = 1; count<10; count++) {  
            System.out.println("Count is: " + count);  
        }  
    }  
}
```

Control Flow Statements

Looping : for

Output:

Count is: 1

Count is: 2

Count is: 3

Count is: 4

Count is: 5

Count is: 6

Count is: 7

Count is: 8

Count is: 9

Control Flow Statements

Looping : for

```
/**
 * The ShowSeat class implements an application that that checks the seat availability status for
 * movie ticket booking using For Looping control statement
 */
class ShowSeat{
    public static void main (String args[]){
        int MaxSeatCount = 5, seatCount = 0;
        for(seatCount=0;seatCount < MaxSeatCount;seatCount++){
            System.out.println("Current Seat Availability : "+(MaxSeatCount-seatCount));
        }
        System.out.println("Seats are Filled");
    }
}
```

Control Flow Statements

Looping : for

Output:

Current Seat Availability : 5

Current Seat Availability : 4

Current Seat Availability : 3

Current Seat Availability : 2

Current Seat Availability : 1

Seats are Filled

Control Flow Statements

Looping : for - each

- **for-each:** It's **commonly used** to iterate over an **array or a Collections** class. It is also called **enhanced for loop**.
- **Syntax:**

```
for (type var : arrayName) {  
    statements;  
}
```
- It is used **to iterate over the elements** of a collection **without** knowing the **index** of each element.
- It is **immutable** which means the values which are **retrieved during the execution** of the loop are read only

Control Flow Statements

Looping : for - each

```
/**  
 * The ForEachApp class implements an application that  
 * illustrate foreach looping Structure  
 */  
class ForEachApp{  
    public static void main (String args[]){  
        int[] marks = { 125, 132, 95, 116, 110 };  
        int maxSoFar = marks[0];  
        //for each loop
```

Control Flow Statements

Looping : for - each

```
        for (int indexValue : marks){  
            if (indexValue > maxSoFar){  
                maxSoFar = indexValue;  
            }  
        }  
        System.out.println("The highest score is " +maxSoFar);  
    }  
}
```

Output:

The highest score is 132

Control Flow Statements

Looping : for - each

```
/**
 * The ForEach class implements an application that list the movie based on their Genre
 * using foreach looping Structure
 */
import java.util.Scanner;

public class ForEach {
    public static void main(String[] args) {
        String MovieName[] = {"AAA","BBB","CCC","DDD"};
        String MovieGenre[] = {"ACTION","COMEDY","THRILLER","ACTION"};
        Scanner input = new Scanner(System.in);           //Scanner class object creation
        System.out.println("Enter the Genre to be searched : ");
        String Genre = input.next();
        int counter = 0;
```

Control Flow Statements

Looping : for - each

```
System.out.println(Genre + " Movies are");  
for (String M : MovieGenre ) {  
    if(M.equals(Genre)) {  
        System.out.println(MovieName[counter]);  
    }  
    counter++;  
}  
input.close();  
}  
}
```

Output:

```
Enter the Genre to be searched :  
ACTION  
ACTION Movies are  
AAA  
DDD
```

Control Flow Statements

Nested Loop

- **Nested Loop:** A loop inside another loop is called a nested loop. The number of loops depend on the requirement of a problem. It contains outer loop and inner loop. For each iteration of outer loop the inner loop executes completely.
- **Syntax**

```
while(condition) {  
    statements(s)  
    while(condition) {  
        statement(s)  
        .....  
    }  
    .....  
}
```

```
for (initialization; condition; updation) {  
    .....  
    for (initialization; condition; updation) {  
        statement(s)  
        .....  
    }  
    .....  
}
```

```
do {  
    .....  
    statement(s)  
    do {  
        statement(s)  
        .....  
    } while(condition);  
    .....  
} while(condition);
```


Control Flow Statements

Nested Loop

```
/**
 * The NestedWhileApp class implements an application that
 * illustrate nested while loop */
class NestedWhileAPP{
    public static void main (String args[]){
        int outerLoop=1,innerLoop=1;
        while(outerLoop<=5){
            while(innerLoop<=5){
                System.out.print("*");
                innerLoop++;
            }
        }
    }
}
```

Control Flow Statements

Nested Loop

```
        System.out.println(" ");
        outerLoop++;
        innerLoop=1;
    }
}
```

Output:

```
*****
*****
*****
*****
*****
```

Control Flow Statements

Nested Loop

```
/**  
 * The NestedWhile class implements an application that demonstrate the seat availability while the seats are getting  
   booked in multiple screens using nested while loop */  
class NestedWhile{  
    public static void main (String args[]){  
        int MaxSeatCount = 5, TotalScreenCount = 2, seatCount = 0, screenCount = 0;  
        while(screenCount < TotalScreenCount){  
            seatCount = 0;  
            System.out.println("Screen "+(screenCount+1)+" Availability details");  
            while(seatCount < MaxSeatCount){  
                System.out.println("Current Seat Availability : "+(MaxSeatCount-seatCount));  
                seatCount++;  
            }  
        }  
    }  
}
```

Control Flow Statements

Nested Loop

```
        System.out.println("Seats Filled in Screen "+(screenCount+1));
        screenCount++;
    }
}
```

Output:

Screen 1 Availability details

Current Seat Availability : 5

Current Seat Availability : 4

Current Seat Availability : 3

Current Seat Availability : 2

Current Seat Availability : 1

Seats Filled in Screen 1

Screen 2 Availability details

Current Seat Availability : 5

Current Seat Availability : 4

Current Seat Availability : 3

Current Seat Availability : 2

Current Seat Availability : 1

Seats Filled in Screen 2

Control Flow Statements

Nested Loop

```
/**
 * The Nested DoWhileApp class implements an application that illustrate nested do...while loop */
class NestedDoWhileApp {
    public static void main (String args[]){
        int outerLabel= 1;
        do {
            int innerLabel = 1;
            do{
                System.out.print(outerLabel);
                innerLabel++;
            } while (innerLabel <= 3);
            outerLabel++;
        } while (outerLabel <= 3);
    }
}
```

Output:

111222333

Control Flow Statements

Nested Loop

```
/**
 * The NestedDoWhile class implements an application that demonstrate the seat availability while the seats are
 * getting booked in multiple screens using nested do...while loop */
class NestedDoWhile{
    public static void main (String args[]){
        int MaxSeatCount = 5, TotalScreenCount = 2, seatCount = 0, screenCount = 0;
        do{
            System.out.println("Screen "+(screenCount+1)+" Availability details");
            seatCount = 0;
            do{
                System.out.println("Current Seats Availability : "+(MaxSeatCount-seatCount));
                seatCount++;
            } while(seatCount < MaxSeatCount);
        }
```

Control Flow Statements

Nested Loop

```
        System.out.println("Seats Filled in Screen "+(screenCount+1));
        screenCount++;
    } while(screenCount < TotalScreenCount);
}
```

Output:

```
Screen 1 Availability details
Current Seat Availability : 5
Current Seat Availability : 4
Current Seat Availability : 3
Current Seat Availability : 2
Current Seat Availability : 1
Seats Filled in Screen 1
Screen 2 Availability details
Current Seat Availability : 5
Current Seat Availability : 4
Current Seat Availability : 3
Current Seat Availability : 2
Current Seat Availability : 1
Seats Filled in Screen 2
```

Control Flow Statements

Nested Loop

```
/**  
 * The Nested ForApp class implements an application that  
 * illustrate nested for looping structure  
 */  
class NestedForApp  
{  
    public static void main (String args[])  
    {  
        int rows = 5;  
        // outer loop
```


Control Flow Statements

Nested Loop

```
for (int i = 1; i <= rows; ++i){  
    // inner loop to print the numbers  
    for (int j = 1; j <= i; ++j) {  
        System.out.print(j + " ");  
    }  
    System.out.println("");  
}  
}
```

Output:

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

Control Flow Statements

Nested Loop

```
/**
 * The NestedFor class implements an application that demonstrate the seat availability while the seats are getting
 * booked in multiple screens and illustrate nested for looping structure
 */

class NestedFor{

    public static void main (String args[]){

        int MaxSeatCount = 5, TotalScreenCount = 2, seatCount = 0, screenCount = 0;

        System.out.println("Screen "+(screenCount+1)+" Availability details");

        for(screenCount=0;screenCount < TotalScreenCount;screenCount++){

            for(seatCount=0;seatCount < MaxSeatCount;seatCount++){

                System.out.println("Current Seat Availability "+(MaxSeatCount-seatCount));
```

Control Flow Statements

Nested Loop

```
        }  
        System.out.println("Seats Filled in Screen "+(screenCount+1));  
    }  
}  
}
```

Output:

```
Screen 1 Availability details  
Current Seat Availability : 5  
Current Seat Availability : 4  
Current Seat Availability : 3  
Current Seat Availability : 2  
Current Seat Availability : 1  
Seats Filled in Screen 1  
Screen 2 Availability details  
Current Seat Availability : 5  
Current Seat Availability : 4  
Current Seat Availability : 3  
Current Seat Availability : 2  
Current Seat Availability : 1  
Seats Filled in Screen 2
```

Control Flow Statements

Branching - Unconditional

- **Branching (Unconditional)** - To **transfers** the control from **one part of the program** to another part **without any condition**.
- In Java, we have the following **unconditional statements**:
 - break
 - labelled break
 - continue
 - labelled continue

Control Flow Statements

Branching - Unconditional

- **break:** When a **break** statement is encountered inside a loop, the **loop** is immediately **terminated** and the program control resumes at the **next statement** following the **loop**.
- The Java break is used to **break loop or switch statement**. It **breaks the current flow of the program** at specified condition. In case of **inner loop**, it **breaks only inner loop**.
- Break statement use **all types of loops** such as for loop, while loop and do-while loop.
- **Syntax**

```
break;
```

Control Flow Statements

Branching - Unconditional

```
/**
 * The BreakApp class implements an application that
 * illustrate Break branching statement
 */
class BreakApp{
    public static void main (String args[]){
        for(int count = 1;count<10;count++) {
            if(count ==5)
                break;    //exit from the current loop
            System.out.println("Count is: " + count);
        }
    }
}
```

Output:

Count is: 1

Count is: 2

Count is: 3

Count is: 4

Control Flow Statements

Branching - Unconditional

```
/**  
 * The UnconditionalBreak class implements an application that demonstrate the seat availability while the seats are  
 * getting booked assuming that the VIP seats are already reserved using Break branching statement  
 */  
class UnconditionalBreak {  
    public static void main (String args[]){  
        int premiumSeat = 5, vipSeat = 5, seatBooked = 0;  
        int totalSeat = premiumSeat + vipSeat;  
        for(seatBooked = 0;seatBooked < totalSeat;seatBooked++) {  
            if(seatBooked > premiumSeat) {
```

Control Flow Statements

Branching - Unconditional

```
        System.out.println("All PREMIUM Seats Booked ");
        System.out.println("All VIP Seats 1 to 5 are Reserved ");
        break;    //exit from the current loop
    }
    else {
        System.out.println("PREMIUM Seat Availability : "+(premiumSeat - seatBooked));
    }
}
}
```


Control Flow Statements

Branching - Unconditional

Output:

PREMIUM Seat Availability : 5

PREMIUM Seat Availability : 4

PREMIUM Seat Availability : 3

PREMIUM Seat Availability : 2

PREMIUM Seat Availability : 1

All PREMIUM Seats Booked

All VIP Seats 6 to 10 are Reserved

Control Flow Statements

Branching - Unconditional

- **continue:** When you need to **jump to the next iteration** of the loop immediately.
- It **continues the current flow** of the program and **skips the remaining code** at the specified condition.

In case of an inner loop, it continues the inner loop only.

- Continue statement use **all types of loops** such as for loop, while loop and do-while loop.
- Syntax:

```
continue;
```

Control Flow Statements

Branching - Unconditional

```
/**
 * The ContinueApp class implements an application that
 * illustrate Continue branching statement
 */
class ContinueApp {
    public static void main (String args[]) {
        for(int count = 1;count<10;count++) {
            if(count ==5)
                continue;
            System.out.println("Count is: " + count);
        }
    }
}
```

Output:

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 6
Count is: 7
Count is: 8
Count is: 9
```

Control Flow Statements

Branching - Unconditional

```
/**
 * The UnconditionalContinue class implements an application that demonstrate the seat availability while the seats are
 * getting booked assuming that the VIP seats are already reserved using Continue branching statement
 */
class UnconditionalContinue {
    public static void main (String args[]){
        int executiveSeat = 5, premiumSeat = 5, vipSeat = 5, seatBooked = 0;
        int totalSeat = regularSeat + executiveSeat + premiumSeat + vipSeat;
        for(seatBooked = 0;seatBooked < totalSeat;seatBooked++) {
            if(seatBooked < (vipSeat)){
                System.out.println("All VIP Seats 1 to 5 are Reserved ");
                continue;
            }
        }
    }
}
```

Control Flow Statements

Branching - Unconditional

```
if(seatBooked < vipSeat + premiumSeat) {  
    System.out.println("PREMIUM Seat No : "+(seatBooked+1));  
}  
if(seatBooked < (vipSeat + premiumSeat + executiveSeat)) {  
    System.out.println("EXECUTIVE Seat No : "+(seatBooked+1));  
}  
}  
}  
}
```

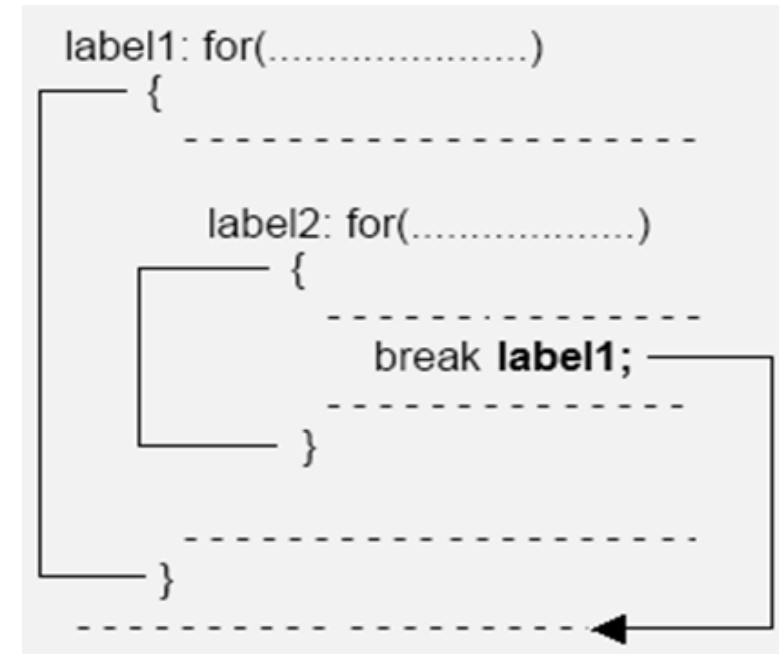
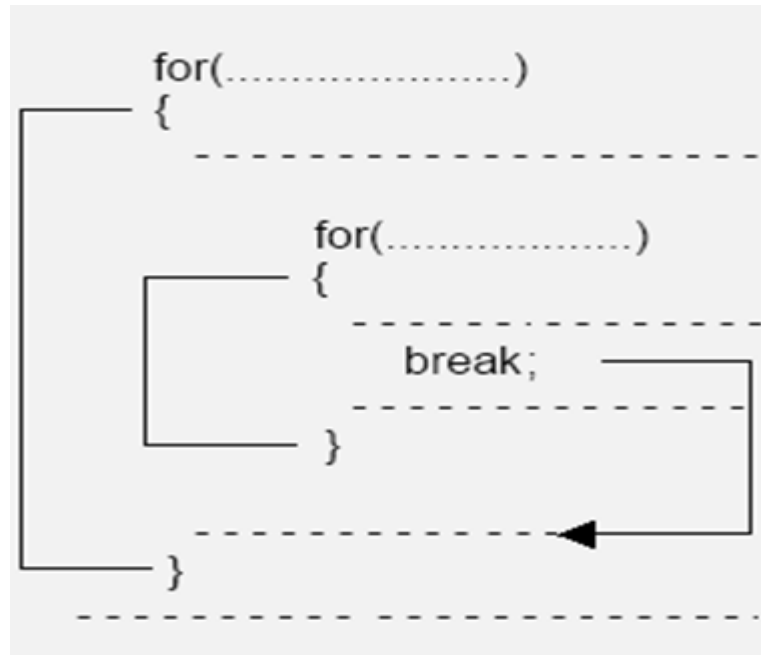
Output:

All VIP Seats 1 to 5 are Reserved
All VIP Seats 1 to 5 are Reserved
All VIP Seats 1 to 5 are Reserved
All VIP Seats 1 to 5 are Reserved
All VIP Seats 1 to 5 are Reserved
PREMIUM Seat No : 6
PREMIUM Seat No : 7
PREMIUM Seat No : 8
PREMIUM Seat No : 9
PREMIUM Seat No : 10
Executive Seat No : 11
Executive Seat No : 12
Executive Seat No : 13
Executive Seat No : 14
Executive Seat No : 15

Control Flow Statements

Branching - Unconditional

- Labeled Loop:** In java, use labels with break and continue. A Label is used to **identify a block of code**. In case of multiple loop involved use labeled loop to **transfer any specific loop with help of labels**.



Control Flow Statements

Branching - Unconditional

```
/** * The LabeledBreakApp class implements an application that * illustrate Labeled Break */
class LabeledBreakApp{
    public static void main(String args[]){
        first: // First label
        for (int i = 0; i < 3; i++) {
            second: // Second label
            for (int j = 0; j < 3; j++) {
                if (1== i && 1 == j) {
                    // Using break statement with label
                    break first;
                }
                System.out.println(i + " " + j);
            }
        }
    }
}
```

Output:

```
0 0
0 1
0 2
1 0
```

Control Flow Statements

Branching - Unconditional

```
/** * The LabeledBreakApp class implements an application that illustrate Labeled Break */
public class LabelledBreak {
    public static void main (String args[]){
        int MaxSeatCount = 10, TotalScreenCount = 2, seatCount = 0, screenCount = -1;
        Start:
        //System.out.println(screenCount);
        while(screenCount < TotalScreenCount){
            screenCount++;
            System.out.println("Screen "+(screenCount+1)+" Seat Booked detail");
            seatCount=0;
            while(seatCount < MaxSeatCount){
                if(seatCount >3 && screenCount == 1) {
                    System.out.println("Seat No 4 & 5 are Reserved");
                }
            }
        }
    }
}
```


Control Flow Statements

Branching - Unconditional

```
        break Start;
    }
    else
        System.out.println("Seats No Booked : "+(seatCount+1));
    seatCount++;
}
System.out.println("All Seats Filled in Screen "+(screenCount+1));
}
}
}
```

Control Flow Statements

Branching - Unconditional

Output:

Screen 1 Seat Booked detail

Seats No Booked : 1

Seats No Booked : 2

Seats No Booked : 3

Seats No Booked : 4

Seats No Booked : 5

All Seats Filled in Screen 1

Screen 2 Seat Booked detail

Seats No Booked : 1

Seats No Booked : 2

Seats No Booked : 3

Seat No 4 & 5 are Reserved

Control Flow Statements

Branching - Unconditional

```
/** The LabeledContinueApp class implements an application that illustrate LabeledContinue loop */
class LabeledContinueApp {
    public static void main(String args[]){
        first: // First label
        for (int i = 0 ; i < 3; i++) {
            second: // Second label
            for (int j = 0; j < 3; j++) {
                if (1 == i && 1 == j) {
                    // Using continue statement with label
                    continue second;
                }
                System.out.println(i + " " + j);
            }
        }
    }
}
```

Output:

```
0 0
0 1
0 2
1 0
2 0
2 1
2 2
```

Control Flow Statements

Branching - Unconditional

```
/** The LabeledContinueApp class implements an application that illustrate LabelledContinue loop */
```

```
public class LabelledContinue {
```

```
public static void main (String args[]){
```

```
    int MaxSeatCount = 5, TotalScreenCount = 2, seatCount = 0, screenCount = 0;
```

```
    SkipScreen:
```

```
    //System.out.println(screenCount);
```

```
    while(screenCount < TotalScreenCount){
```

```
        System.out.println("Screen "+(screenCount)+" Ticket Booking detail");
```

```
        seatCount=0;
```

```
        SkipSeat:
```

```
        while(seatCount < MaxSeatCount){
```

```
            seatCount++;
```

Control Flow Statements

Branching - Unconditional

```
if((seatCount == 2 || seatCount == 3) && screenCount == 1) {  
    System.out.println("Seat No "+seatCount+" is Reserved");  
    continue SkipSeat;  
}  
else  
    System.out.println("Seats No Booked : "+(seatCount));  
  
}  
System.out.println("All Seats Filled in Screen "+(screenCount+1));  
screenCount++;  
}  
}  
}
```

Control Flow Statements

Branching - Unconditional

Output:

Screen 1 Ticket Booking detail

Seats No Booked : 1

Seats No Booked : 2

Seats No Booked : 3

Seats No Booked : 4

Seats No Booked : 5

All Seats Filled in Screen 1

Screen 2 Ticket Booking detail

Seats No Booked : 1

Seat No 2 is Reserved

Seat No 3 is Reserved

Seats No Booked : 4

Seats No Booked : 5

All Seats Filled in Screen 2

Control Flow Statements

Quiz



1. What do you call a statement that executes a certain statement(s) ,if the condition is true, and executes a different statement(s) if the condition is false?

a) if statement

b) if else statement

c) if else if statement

d) None of the Above

b) if else statement

Control Flow Statements

Quiz



2. From the given if-else statement, what will be displayed if the value of num is 6?

```
if (num>0)
    System.out.println("Positive Number\n");
else
    System.out.println("Negative Number\n");
    System.out.println("The number is %d",num);
```

a) Positive Number

b) Negative Number

c) Positive Number
The number is 6

d) The number is 6

c) Positive Number
The number is 6

Control Flow Statements

Quiz



3. In switch case statement, what will be executed if there is no case matched?

a) case

b) break

c) default

d) All the above

c) default

Control Flow Statements

Quiz



4. What is a java keyword used in switch statements that causes immediate exit or that terminates the switch statement?

a) The case keyword

b) The switch keyword

c) The default keyword

d) The break keyword

d) The break keyword

Control Flow Statements

Quiz



5. Which of the following loops will execute the statements at least once, even though the condition is false initially?

a) while

b) do...while

c) for

d) All the options

b) do...while

Control Flow Statements

Quiz



6. What value is stored in index at the end of this loop?

```
for(int index =1;index<=10;index++)
```

a) 1

b) 9

c) 10

d) 11

b) 11

Control Flow Statements

Quiz



7. Which of the following are true about the enhanced for loop?

- A. It can iterate over an array or a Collection but not a Map.**
- B. Using an enhanced for loop prevents the code from going into an infinite loop**
- C. Using an enhanced for loop on an array may cause infinite loop.**
- D. An enhanced for loop can iterate over a Map.**
- E. You cannot find out the number of the current iteration while iterating.**

a) A,B,E

b) A,B,C

c) B,C,E

d) A,D,E

a) A,B,E

Control Flow Statements

Quiz



8. What is printed as a result of the following code segment?

```
for (int k = 0; k < 20; k+=2)
{
    if (k % 3 == 1)
        System.out.print(k + " ");
}
```

a) 0 2 4 6 8 10 12 14 16 18

b) 4,6

c) 4,10,16

d) 0,6,12,18

c) 4,10,16

Control Flow Statements

Quiz



9. What is the output after the following code has been executed?

```
class FlowControl1 {
    public static void main(String[] args){
        int value1= 10, value2 = 20;
        if (value1 < value2 ){
            if (value2 > value2 )
                System.out.println("Hello Friend");
            else
                System.out.println(" Happy Day!");
        }
    }
}
```

Control Flow Statements

Quiz



10. What is stored in the variable result after the following code has been executed?

```
class FlowControl2{
    public static void main(String args[]){
        int index = 0;
        int result = 1;
        while ( true ){
            ++index;
            if ( index % 2 == 0 )
                continue;
            else if ( index % 5 == 0 )
                break;
            result *= 3;
        }
    }
}
```


Control Flow Statements

Quiz



11. What is the output after the following code has been executed?

```
class FlowControl3 {
    public static void main(String[] args){
        boolean b = true;
        if(!b) {
            System.out.println("HELLO");
        } else {
            System.out.println("BYE");
        }
    }
}
```

Control Flow Statements

Quiz



12. What is the output after the following code has been executed?

```
class FlowControl4 {
    public static void main(String[] args){
        for (int i = 0; ; i++) {
            System.out.println("HIII");
        }
        System.out.println("BYE");
    }
}
```

Control Flow Statements

Quiz



13. What is stored in the result after the following code has been executed?

```
class Test {
    public static void main(String[] args){
        do {
            System.out.print(1);
            do {
                System.out.print(2);
            } while (false);
        } while (false);
    }
}
```

Arrays



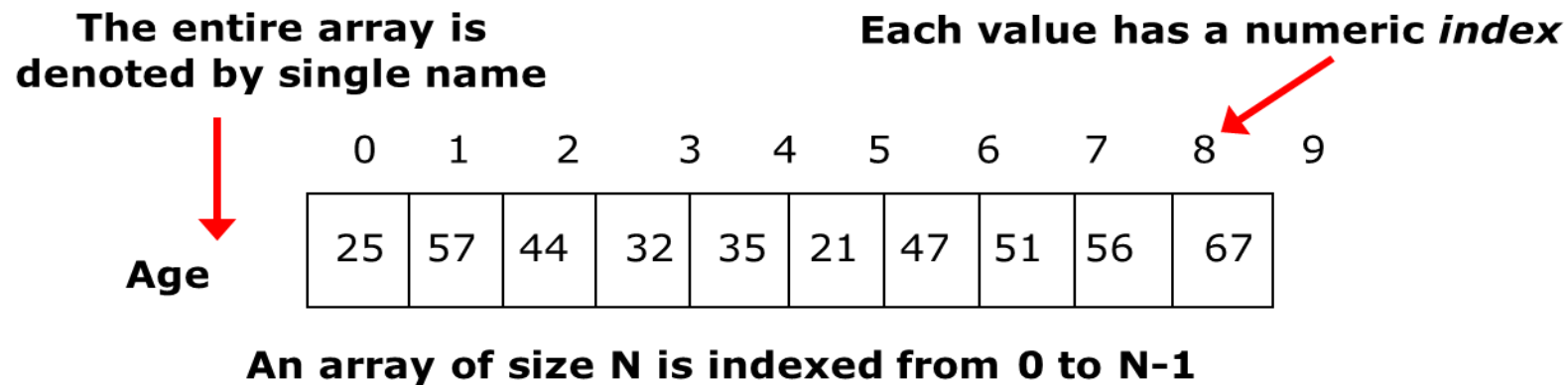
Contents

- Introduction
- Single Dimensional Array
- Multi Dimensional Array
- Multi Dimensional Array: Jagged Array
- Quiz

Arrays

Introduction

- An **array** is a **container** that holds a **fixed number of values** of a **same data type**.
- **Need of Array: Difficult to manage** large number of variable in normal way. The **idea of array** is to represent many instances in one variable.
- Length of the **array is fixed** and index **starts with 0**. **Can't access** the elements **beyond the array limit**.



Arrays

Types

- There are **three types of arrays** in Java:
 - Single Dimensional Array
 - Multidimensional Array
 - Jagged Array

Arrays

Single Dimensional Array

- **Single dimensional array** is a collection of items under **common name** in **linear array** fashion.
- **Declaration**
- **Syntax**

```
dataType[] arrayName; (or)
```

```
dataType []arrayName; (or)
```

```
dataType arrayName[];
```

Instantiation of an Array in Java

```
arrayReferenceVariable=new dataType[size]
```

Example

```
int []salary; //declaration
```

```
salary=new int[10]; //Instantiation
```

Declaration and Instantiation

```
int salary[]=new int[10];
```

Note:

- Array **declaration** only create **reference of array variable**.
- To **create or give memory to array only at** that **time of array instantiation**.

Arrays

Single Dimensional Array

Create Array

- `int [] marks=new int[10];` **//declaration and instantiation**

(OR)

- `int marks[]=new int[10];`
- The `[]` **operator** is the **indication to the compiler** we are **naming an array object** and not an ordinary variable.
- Java Array is object, we use the **new operator** to create an array.

Arrays

Single Dimensional Array

Define or initialize Array values:

- There are **two ways** to define the array values:

1. During Declaration: `int [] marks={89, 90, 67,35, 99, 80};`

2. Using index : `int marks[]=new int[10]; // declaration`

`marks[0]=89;`

`marks[1]=90;`

`.....`

- **Array Length:** To find the array length java provides **inbuilt length variable**

`int len=marks.length; //assigns the size of marks array in to the variable len`

Arrays

Single Dimensional Array

Retrieving and Processing Array Elements

- Array elements are accessed using index value. The index starts with 0.

`element=marks[3]; //retrieve 4th element of array`

`marks[4]= 100; // update 100 into 5th position`

- Since more number of elements are there in an array, **loops are more convenient way** to process.
- In general, array elements are **accessed and processed** using **for loop** and **for....each loop**.

Arrays

Single Dimensional Array

```
/**
 * The ArrayDemo1 class implements an application that
 * Illustrate the access array elements
 */
public class ArrayDemo1 {
    public static void main(String[] args) {
        int[] marks = new int[3]; ; // create array
        marks[0] = 89; //assign values
        marks[1] = 90;
        System.out.println("Element at index 0: " + marks[0]); //access elements
        System.out.println("Element at index 1: " + marks[1]);
        System.out.println("Element at index 2: " + marks[2]);

    }
}
```

Output:

Element at index 0: 89

Element at index 1: 90

Element at index 2: 0

Arrays

Single Dimensional Array

```
/**
 * The CustomerDetail class implements an application that
 * Illustrate the access array elements
 */
public class CustomerDetail {
    public static void main(String[] args) {
        String[] custName = new String[5]; ; // create array
        custName[0] = "Aaron"; //assign values
        custName[1] = "Kavin";
        custName[2] = "Jesicca";
        custName[3] = "Rishabh";
        custName[4] = "Vinitha";
    }
}
```

Arrays

Single Dimensional Array

```
        System.out.println("*****CUSTOMER DETAIL*****");  
        System.out.println(custName[0]); //access elements  
        System.out.println(custName[1]);  
        System.out.println(custName[2]);  
        System.out.println(custName[3]);  
        System.out.println(custName[4]);  
    }  
}
```

Output:

```
*****CUSTOMER DETAIL*****  
  
Aaron  
Kavin  
Jesicca  
Rishabh  
Vinitha
```

Arrays

Single Dimensional Array

```
/**
 * The ArrayDemoForEach class implements an application that
 * Illustrate the access array elements using for-each statements
 */
Class ArrayDemoForEach {
    public static void main(String[] args) {
        int[] marks = {56, 84, 52, 90, 100};
        System.out.println("Using for each Loop:");
        for(int i:marks) {
            System.out.println(i);
        }
    }
}
```

Output:

Using for each Loop:

56

84

52

90

100

Arrays

Single Dimensional Array

```
/**  
 * The CustomerDetailForEach class implements an application that  
 * Illustrate the access array elements using for-each statements  
 */  
public class CustomerDetailForEach {  
    public static void main(String[] args) {  
        String[] custName = new String[5]; ; // create array  
        custName[0] = "Aaron"; //assign values  
        custName[1] = "Kavin";  
        custName[2] = "Jesicca";  
        custName[3] = "Rishabh";  
        custName[4] = "Vinitha";  
    }  
}
```


Arrays

Single Dimensional Array

```
System.out.println("*****CUSTOMER DETAIL*****");  
for(String name : custName) {  
    System.out.println(name);  
}  
}
```

Output:

```
*****CUSTOMER DETAIL*****  
Aaron  
Kavin  
Jesicca  
Rishabh  
Vinitha
```

Arrays

Multi Dimensional Array

- In general, **more than one dimension** refer to the multi dimensional array. Its is **array of arrays**. The retrieve and processing like single dimensional array.
- **Declaration and Instantiation**
- **Two-Dimensional Array:** In the **form of matrix** which represents collection of **rows and columns**.

Syntax:

```
DataType[][] ArrayName = new int[RowSize][ColumnSize];  
  
int bookCount[][] = new int[3][3];
```

- **Three Dimensional Array:** In the **form of table** and each table contains number of rows and columns.

Syntax:

```
DataType[][][] ArrayName = new int[TableSize][RowSize][ColumnSize];  
  
int bookCount[][][] = new int[3][3][3];
```

Arrays

Multi Dimensional Array

Need for Multi-dimensional Array:

- Data is stored in the form of **table or matrix form**. It is used to represents the data in **different dimension like row and column wise**.
- It is used to represents **Graph and database** like data structure.
- Specifically used to **find minimum spanning tree** and **connectivity checking between nodes**.

Arrays

Multi Dimensional Array

```
/**
 * The ArrayDemo3 class implements an application that illustrate the access of multidimensional
 * array elements */
Class ArrayDemo3{
    public static void main(String[] args){
        int [][] x = new int[][] {{1,2},{3,4},{5,6}}; // initialize values
        for(int i=0; i < x.length; i++) {
            // print array elements
            for (int j=0; j < x[i].length; j++) {
                System.out.print(x[i][j]);
            }
            System.out.println();
        }
    }
}
```

Output:

```
1 2
3 4
5 6
```

Arrays

Multi Dimensional Array

```
/**
 * The MovieSeat class implements an application that illustrate the access of multidimensional array elements */
public class MovieSeat {
    public static void main(String[] args){
        String [][] seatType = new String[][] {{"B","B","A","A","A"}, {"A","A","A","B","B"}, {"A","B","B","B","B"}, {"B","A","A","B","A"}};
        int vipcount = 0, premiumcount = 0, regularcount = 0, viptotal = 5, premiumtotal = 10, regulartotal = 5;
        System.out.println("--MOVIE SEAT ARRANGEMENT--");
        for(int i=0; i < seatType.length; i++) {
            if (i==0)
                System.out.println("-----VIP SEATS-----");
            else if(i==1)
                System.out.println("-----PREMIUM SEATS-----");
            else if(i==3)
                System.out.println("-----REGULAR SEATS-----");
        }
    }
}
```

Arrays

Multi Dimensional Array

```
for (int j=0; j < seatType[i].length; j++) {  
    System.out.print(" "+seatType[i][j]+" ");  
    if(i==0 && seatType[i][j].equalsIgnoreCase("B"))  
        vipcount++;  
    else if(i>0 && i<3 && seatType[i][j].equalsIgnoreCase("B"))  
        premiumcount++;  
    else if(i==3 && seatType[i][j].equalsIgnoreCase("B"))  
        regularcount++;  
}  
    System.out.println();  
}  
System.out.println("----SEAT BOOKED DETAIL----");  
System.out.println("-----VIP SEATS-----");  
System.out.println("BOOKED : "+vipcount+" AVAILABLE : "+(viptotal-vipcount)+" TOTAL : "+viptotal);
```

Arrays

Multi Dimensional Array

```
        System.out.println("-----PREMIUM SEATS-----");
        System.out.println("BOOKED : "+premiumcount+" AVAILABLE : "+(premiumtotal-
premiumcount)+" TOTAL : "+premiumtotal);
        System.out.println("-----REGULAR SEATS-----");
        System.out.println("BOOKED : "+regularcount+" AVAILABLE : "+(regulartotal-
regularcount)+" TOTAL : "+regulartotal);
    }
}
```

Output:

```
--MOVIE SEAT ARRANGEMENT--
-----VIP SEATS-----
B  B  A  A  A
-----PREMIUM SEATS-----
A  A  A  B  B
A  B  B  B  B
-----REGULAR SEATS-----
B  A  A  B  A
----SEAT BOOKED DETAIL----
-----VIP SEATS-----
BOOKED : 2 AVAILABLE : 3 TOTAL : 5
-----PREMIUM SEATS-----
BOOKED : 6 AVAILABLE : 4 TOTAL : 10
-----REGULAR SEATS-----
BOOKED : 2 AVAILABLE : 3 TOTAL : 5
```

Arrays

Multi Dimensional Array : Jagged Array

- Java supports jagged array. It is **array of arrays** with **different number of columns**.
- **Declaration and Instantiation**
- **Jagged Array:** In the **form of matrix** which represents **fixed number of rows and different size columns**.

Syntax:

```
DataType[][] Array_Name = new int[Row_Size][];  
int bookNo[][] = new int[3][]; //variable size column  
//adding column  
bookNo[0] = new int[3]; //0th row contains 3 columns  
bookNo[1] = new int[5]; //1st row contains 5 columns  
bookNo[2] = new int[1]; //2nd row contains 1 column
```


Arrays

Multi Dimensional Array : Jagged Array

Need for Jagged Array:

- It improves the performance when working with multi-dimensional arrays.
- In a jagged array, which is an array of arrays, each inner array can be of a different size. It helps the efficient **memory management**.
- **For example:**
 - Course registration count based on different category.
 - Ticket reservation details based on different category. Etc.

Arrays

Multi Dimensional Array : Jagged Array

```
/**
 * The JaggedArray class implements an application that
 * Illustrate the jagged array
 */
public class JaggedArray {
    public static void main(String[] args) {
        int bookNo[][] = new int[3][];
        bookNo[0] = new int[] {1,2,3};
        bookNo[1] = new int[] {4,5};
        bookNo[2] = new int[] {6,7,8,9,10};
        System.out.println("Two Dimensional Jagged Array");
    }
}
```

Arrays

Multi Dimensional Array : Jagged Array

```
for(int i=0;i<bookNo.length;i++) {  
    for(int j=0;j<bookNo[i].length;j++) {  
        System.out.println(bookNo[i][j]+" ");  
    }  
    System.out.println();  
}  
}
```

Output:

Two Dimensional Jagged Array

1
2
3
4
5
6
7
8
9
10

Arrays

Multi Dimensional Array

```
/**
 * The MovieSeat class implements an application that illustrate the access of multidimensional array elements */
public class MovieSeatJaggedArray {
    public static void main(String[] args){
        String [][] seatType = new String[][] {{ "B","A","A"}, {"A","A","A","B","B"}, {"A","B","B","B","B"}, {"B","A","A","A"} };
        int vipcount = 0, premiumcount = 0, regularcount = 0, viptotal = 5, premiumtotal = 10, regulartotal = 5;
        System.out.println("--MOVIE SEAT ARRANGEMENT--");
        for(int i=0; i < seatType.length; i++) {
            if (i==0)
                System.out.println("-----VIP SEATS-----");
            else if(i==1)
                System.out.println("-----PREMIUM SEATS-----");
            else if(i==3)
                System.out.println("-----REGULAR SEATS-----");
        }
    }
}
```

Arrays

Multi Dimensional Array

```
for (int j=0; j < seatType[i].length; j++) {  
    System.out.print(" "+seatType[i][j]+" ");  
    if(i==0 && seatType[i][j].equalsIgnoreCase("B"))  
        vipcount++;  
    else if(i>0 && i<3 && seatType[i][j].equalsIgnoreCase("B"))  
        premiumcount++;  
    else if(i==3 && seatType[i][j].equalsIgnoreCase("B"))  
        regularcount++;  
}  
System.out.println();  
}  
System.out.println("----SEAT BOOKED DETAIL----");  
System.out.println("-----VIP SEATS-----");  
System.out.println("BOOKED : "+vipcount+" AVAILABLE : "+(viptotal-vipcount)+" TOTAL : "+viptotal);
```

Arrays

Multi Dimensional Array

```
        System.out.println("-----PREMIUM SEATS-----");
        System.out.println("BOOKED : "+premiumcount+" AVAILABLE : "+(premiumtotal-
premiumcount)+" TOTAL : "+premiumtotal);
        System.out.println("-----REGULAR SEATS-----");
        System.out.println("BOOKED : "+regularcount+" AVAILABLE : "+(regulartotal-
regularcount)+" TOTAL : "+regulartotal);
    }
}
```

Output:

```
--MOVIE SEAT ARRANGEMENT--
-----VIP SEATS-----
B  A  A
-----PREMIUM SEATS-----
A  A  A  B  B
A  B  B  B  B
-----REGULAR SEATS-----
B  A  A  A
----SEAT BOOKED DETAIL----
-----VIP SEATS-----
BOOKED : 1 AVAILABLE : 2 TOTAL : 3
-----PREMIUM SEATS-----
BOOKED : 6 AVAILABLE : 4 TOTAL : 10
-----REGULAR SEATS-----
BOOKED : 1 AVAILABLE : 3 TOTAL : 4
```

Arrays

Quiz



1. Which of the following is FALSE about Java array?

a) A java array is always an object

b) Length of array can be changed after the creation of array

c) Arrays in Java are always allocated on heap

d) Array is the example for Non-Primitive type

b) Length of array can be changed after the creation of array

Arrays

Quiz



2. In java array supports,

a) Primitive type only

b) Object type only

c) Both

d) None of these above

C) Both

Arrays

Quiz



3. Java supports variable size column in multidimensional array

a) Yes

b) No

a) Yes

Functions



Contents

- Introduction
- Function elements
- Quiz

Functions

Introduction

- A **function/Method** is a **block of code** which perform **specific task** and the task is executed when the function is invoked or called.
- Primary **uses of functions**:
 - It allows code **reusability** (define once and use multiple times)
 - You can **break a complex program** into smaller chunks of code
 - Reducing duplication** of code
 - Make program shorter and **increases code readability**

Functions

Types

- There are **two types** of functions:
 - **Built-in function:** Java has several functions that are **readily available for use**. In Java, every built-in function should be **part of some class**.
 - **User defined function:** Function created **by user** based on the **need of application**.
- With methods (functions), there are **2 major points** of view
 - **Builder of the method** - responsible for creating the *declaration* and the *definition* of the method (i.e. how it works)
 - **Caller** - somebody (i.e. some portion of code) that *uses* the method to perform a task

Functions

Function Elements

- There are **two** important **elements** of function:

1. Function Definition

- It define the **operation of a function**. It consists of the **function signature** followed by the **function body**. The function prototype includes **function name, parameter list and return type**.

2. Function Call

- It means **call or invoke** the specific function. When a function is invoked, the program **control jumps** to the **called function** to execute the statements that are in the part of that function. Once the called function is executed the program **control passes back to the calling function**.

Functions

Function Elements

- **Function Definition**

Syntax:

```
modifier returnType methodName(parameterlists) // Function signature
{
    //Function Body
}
```

- **Modifiers / Specifiers:** It defines the **visibility of the method** i.e. from where it can be accessible in the application.

Functions

Function Elements

- In Java, there **4 type of the access specifiers**.
 - **public**: accessible in all class in your application.
 - **protected**: accessible within the class in which it is defined and, in its subclass,(es).
 - **private**: accessible only within the class in which it is defined.
 - **default (declared/defined without using any modifier)** : accessible within same class and package within which its class is defined.

Functions

Function Elements

- **Function Definition**

- **The return type** : The data type of the value returned by the method or void if does not return a value.
- **Method Name** : Name of the function should specify using valid identifier
- **Java Naming Convention**: It is a **single word** that should be a verb in lowercase or **multi-word**, that begins with a verb in lowercase followed by adjective or noun. After the first word, **first letter of each word should be capitalized. Example**: computeAddition, setName
- **Parameter list** : Comma separated list of the **input parameters** are defined, preceded with their data type, within the enclosed parenthesis. If there are no parameters, you must use **empty parentheses ()**.
- **Method body** : It is enclosed between braces. The code specifies the **task to be done**.

Functions

Function Elements

- **Function Call**
 - **Static method** is invoked by the **class name** or **using object**. (Refer Example)
 - **Example:** `className.methodName(argumentList)`
 - **Non static method** is invoked by the **instance/object** of the class (**We Will discuss later**)
 - **Example:** `objectName.methodName(argumentList)`

Functions

Return Values

- **Function return types:** Function return values of **any valid types**. It must **return data** that **matches** their return type.
- **Example:**

```
public int addTwoInt(int a, int b){  
    return a+b;  // return integer  
}
```

```
//void method return nothing  
public void printName(String name){  
    System.out.println("Hello World!!!"); // return void  
}
```

Functions

Arguments and Parameters

- **Arguments and Parameters:** The terms parameter and argument can be used for the same thing information that are passed into a function.
 - **Arguments** –An argument is a value that is passed during a **function call or calling function**.
 - **Parameters** – Parameters used in the function definition or called function. A parameter is a variable defined in the **function definition or called function**.

Note:

- During program execution, the **values in the actual arguments is assigned to the formal parameter**.

Functions

Function Elements

//Called Function

public int addTwoInt(int a, int b){

return a+b; //Function Body

}

//Calling Function

public static void main (String[] args){

int sum; Function Call

sum=addTwoInt(5,6);

System.out.println("Sum of two integer values :"+ sum);

}

**Here,
a & b is formal parameters**

**Here,
5 & 6 is actual arguments**

Functions

Function Elements

```
/**  
 * The MethodDeclareDemo class implements an application that  
 * Illustrate the user defined methods(static) */  
class MethodDeclareDemo{  
    //User Defined Method  
    public int static addTwoInt(int a, int b){ //a, b is parameters  
        return a+b;  
    }  
    public static void main (String[] args){  
        int sum;  
        sum=addTwoInt(5,6); //5, 6 is arguments  
        System.out.println("Sum of two integer values :"+ sum);  
    }  
}
```

Output:

Sum of two integer
values :11

Functions

Function Elements

```
/**  
* The FunctionDeclare class implements an application that display the Movie details  
* using the user defined methods(static) */  
public class FunctionDeclare {  
    static void getMovieDetail(String moviename,String moviedescription,int movieduration,String movielanguage,  
    String moviereleasedate,String moviecountry,String moviegenre) {  
        System.out.println("Movie Title : "+moviename);  
        System.out.println("Movie Description : "+moviedescription);  
        System.out.println("Movie Duration : "+movieduration);  
        System.out.println("Movie Language : "+movielanguage);  
        System.out.println("Movie Release Date : "+moviereleasedate);  
        System.out.println("Movie Country : "+moviecountry);  
        System.out.println("Movie Genre : "+moviegenre);  
  
    }
```

Functions

Function Elements

```
public static void main(String[] args) {  
    String moviename = "AAA";  
    String moviedescription = "Dramaof1945";  
    int movieduration = 3;  
    String movielanguage = "English";  
    String moviereleasedate = "25/03/2022";  
    String moviecountry = "XYZ";  
    String moviegenre = "THRILLER";  
    System.out.println("-----Movie Detail-----");  
    getMovieDetail(moviename, moviedescription, movieduration, movielanguage, moviereleasedate,  
moviecountry, moviegenre);  
    System.out.println("-----");  
}
```


Functions

Function Elements

Output:

-----Movie Detail-----

Movie Title : AAA

Movie Description : Dramaof1945

Movie Duration : 3

Movie Language : English

Movie Release Date : 25/03/2022

Movie Country : XYZ

Movie Genre : THRILLER

Functions

Quiz



1. Java method signature is a combination of ____.

a) returnType

b) methodName

c) Argumentlist

d) All the above

d) All the above

Functions

Quiz



2. What the naming convention should be for Methods in Java?

a) It should start with lowercase letter.

b) If name contains many words then first word's letter start with lowercase only and other words will start with uppercase

c) It should be a verb such as `show()`, `count()`, `describe()`

d) All the above

d) All the above

Functions

Quiz



3. Which is not an advantage of the function?

a) Re-usability

b) Makes problem simple to solve

c) Efficiency

d) Easy to Understand the Solution

c) Efficiency

THANK YOU